CHAPTER 2

Modeling Complex Systems

Recommended sections for a first reading: 2.1 through 2.5

2.1 INTRODUCTION

In Chap. 1 we looked at simulation modeling in general, and then modeled and coded two specific systems. Those systems were very simple, and it was possible to program them directly in a general-purpose language, without using any special simulation software or support programs (other than a random-number generator). Most real-world systems, however, are quite complex, and coding them without supporting software can be a difficult and time-consuming task.

In this chapter we first discuss an activity that takes place in most simulations: list processing. A group of ANSI-standard C support functions, simlib, is then introduced, which takes care of some standard list-processing tasks as well as several other common simulation activities, such as processing the event list, accumulating statistics, generating random numbers and observations from a few distributions, as well as calculating and writing out results. We then use simlib in four example simulations, the first of which is just the single-server queueing system from Sec. 1.4 (included to illustrate the use of simlib on a familiar model); the last three examples are somewhat more complex.

Our purpose in this chapter is to illustrate how more complex systems can be modeled, and to show how list processing and the simlib utility functions can aid in their programming. Our intention in using a package such as simlib is purely pedagogical; it allows the reader to move quickly into modeling more complex systems and to appreciate how real simulation-software packages handle lists and other data. We do not mean to imply that simlib is as comprehensive or efficient as, or in any other way comparable to, the modern commercial simulation software discussed in

Chaps. 3 and 14; in fact, the complete source code for simlib is given in App. 2A and Fig. 7.5, and can be downloaded from www.mhhe.com/law (along with the code for all the examples in this chapter). A FORTRAN 77 version of simlib, along with some documentation specific to it and code for the four example models discussed in this chapter, can also be downloaded from this website.

2.2 LIST PROCESSING IN SIMULATION

The simulation models considered in Chap. 1 were really quite simple in that they contained either one or no *lists* of *records* other than the event list. Furthermore, the records in these lists consisted of a single *attribute* and were always processed in a first-in, first-out (FIFO) manner. In the queueing example, there was a FIFO list containing the records of all customers waiting in queue, and each customer record consisted of a single attribute, the time of arrival. In the inventory example there were no lists other than the event list. However, most complex simulations require many lists, each of which may contain a large number of records, consisting in turn of possibly many attributes each. Furthermore, it is often necessary to process these lists in a manner other than FIFO. For example, in some models one must be able to remove that record in a list with the smallest value for a specified attribute (other than the time the record was placed in the list). If this large amount of information is not stored and manipulated efficiently, the model may require so much execution time or so many storage locations that the simulation study would not be feasible.

In Sec. 2.2.1 we discuss two approaches to storing lists of records in a computer—sequential and linked allocation—and then explain why the latter approach is preferable for complex simulations. In Sec. 2.2.2 we present a treatment of linked storage allocation that is sufficient for the development of a simple C-based simulation "language," simlib, in Sec. 2.3. This language, which can be completely mastered in just a few hours of study, provides considerable insight into the nature of the special-purpose simulation software discussed in Chaps. 3 and 14, which requires much more time to learn. More important, simlib provides a vehicle for explaining how to simulate systems that are considerably more complicated than those discussed in Chap. 1.

2.2.1 Approaches to Storing Lists in a Computer

There are two principal approaches to storing lists of records in a computer. In the *sequential-allocation* approach, used in Chap. 1, the records in a list are put into physically adjacent storage locations, one record after another. This was the approach taken in Sec. 1.4 with the list of arrival times for customers in the queue.

In the *linked-allocation* approach, each record in a list contains its usual attributes and, in addition, *pointers* (or *links*) giving the *logical* relationship of the record to other records in the list. Records in a list that follow each other logically need not be stored in physically adjacent locations. A detailed discussion of linked allocation

is given in Sec. 2.2.2. Linked allocation of lists has several advantages for simulation modeling:

- The time required to process certain kinds of lists can be significantly reduced. For the queueing example of Sec. 1.4, every time a customer completed service (and left a nonempty queue behind) we had to move each entry in the arrival-time array up one storage location; this would be quite inefficient if the queue were long, in which case the array would contain a large number of records. As we shall see in Example 2.1, linked allocation expedites processing of such arrays.
- For simulation models where the event list contains a large number of event records simultaneously, we can speed up event-list processing considerably; see Example 2.2 and Sec. 2.8 for further discussion.
- For some simulation models, the amount of computer memory required for storage can be reduced; see the discussion at the end of Sec. 2.2.2.
- Linked allocation provides a general framework that allows one to store and manipulate many lists simultaneously with ease, whereby records in different lists may be processed in different ways. This generality is one of the reasons for the use of the linked-allocation approach by all major simulation software.

2.2.2 Linked Storage Allocation

In this section we present a discussion of linked storage allocation sufficient for development of the simple C-based simulation "language" simlib, described in the next section. For a more complete and general discussion of list-processing principles, see, for example, Knuth (1997, chap. 2).

Suppose that a list of records is to be stored in an array, that the rows of the array correspond to the records, and that the columns of the array correspond to the attributes (or data fields) that make up the records. For the queueing simulation of Sec. 1.4, each customer waiting in the queue had a record in the arrival-time list, and each record consisted of a single attribute, the corresponding customer's time of arrival. In general, a customer's record might have additional attributes such as age, a priority number, service requirement, etc.

A list of records is called *doubly linked* if each record has associated with it a predecessor link and a successor link. The *successor link* (or *forward pointer*) for a particular record gives the *physical* row number in the array of the record that *logically* succeeds the specified record. If no record succeeds the specified record, the successor link is set to zero. The *predecessor link* (or *backward pointer*) for a particular record gives the physical row number in the array of the record that logically precedes the specified record. If no record precedes the specified record, the predecessor link is set to zero. The number of the physical row in the array that contains the record that is logically first in the list is identified by a *head pointer*, which is set to zero when the list contains no records. The physical row number of the record that is logically last in the list is identified by a *tail pointer*, which is set to zero when the list is empty.

At any given time a list will probably occupy only a subset of the rows of the array in which it is physically stored. The "empty" rows of the array that are

available for future use are linked together in a *list of available space* (LAS). The LAS is usually processed in a LIFO (last-in, first-out) manner: This means that when a row is needed to store an additional record, the row is taken from the head of the LAS; and when a row is no longer needed to store a record, the row is returned to the head of the LAS. Since all operations are done at the head of the LAS, it requires neither a tail pointer nor predecessor links. (We call such a list *singly linked.*) At time 0 in a simulation, all rows in the array are members of the LAS, the successor link of row i is set to $i + 1$ (except for that of the last row, which is set to 0), all predecessor links are set to 0, and the head of the LAS is set to 1. (The predecessor link for a particular row is set to a positive integer only when that row is occupied by a record.) In languages that support dynamic storage allocation (like C), the LAS can be thought of as all memory available for dynamic allocation.

EXAMPLE 2.1. For the queueing simulation of Sec. 1.4, consider the list containing the customers waiting in *queue* to be served. Each record in this list has the single attribute, "time of arrival." Suppose that at time 25 in the simulation there are three customers in queue, with times of arrival 10, 15, and 25, and that these records are stored in (physical) rows 2, 3, and 1 of an array with 5 rows and 1 column. (To make the figures below manageable, we assume that there will never be more than five customers in queue at any time.) Rows 4 and 5 are members of the LAS. The situation is depicted in Fig. 2.1. Note that the head pointer of the list is equal to 2, the successor link for the record in row 2 is equal to 3, the predecessor link for the record in row 3 is equal to 2, etc.

Suppose that the next event in the simulation (after time 25) is the arrival of a customer at time 40 and that we would like to add an appropriate record to the list, which is to be processed in a FIFO manner. Since the head pointer for the LAS is equal to 4,

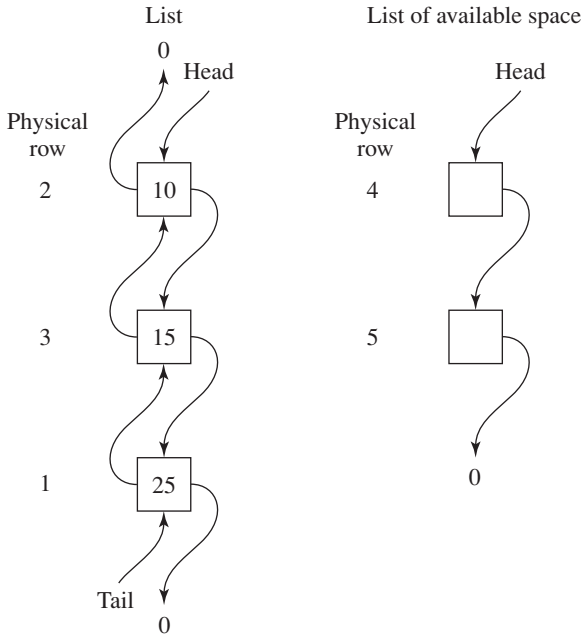


FIGURE 2.1
State of the lists for the queueing simulation at time 25.

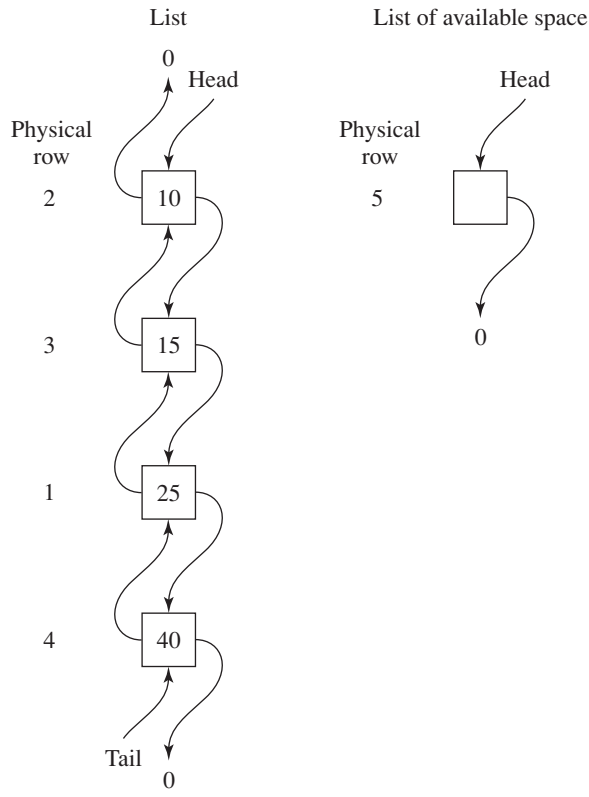


FIGURE 2.2
State of the lists for the queueing simulation at time 40.

the record for the arriving customer will be placed in physical row 4 of the array and the head pointer of the LAS is now set to 5, which is the value of the successor link for row 4. Since the new record will be added to the tail of the list and the tail pointer for the list is now equal to 1, the successor link for the record in row 1 is set to 4, the predecessor link for the new record, i.e., the one in (physical) row 4, is set to 1, the successor link for the new record is set to 0, and the tail pointer for the list is set to 4. The state of both lists after these changes have been made is shown in Fig. 2.2.

Suppose that the next event in the simulation (after time 40) is the service completion at time 50 of the customer who was being served (at least since time 25) and that we want to remove the record of the customer at the head of the list so that this customer can begin service. Since the head pointer for the list is equal to 2 and the successor link for the record in (physical) row 2 is equal to 3, the time of arrival of the record in row 2 is used to compute the delay of the customer who will enter service (this delay is $50 - 10$), the head pointer for the list is set to 3, the predecessor link for the record in row 3 is set to 0, and row 2 (which is no longer needed) is placed at the head of the LAS by setting its head pointer to 2 and the successor link for row 2 to 5 (the previous head of the LAS). The state of both lists after these changes have been made is shown in Fig. 2.3.

Thus, removing a record from the head of the list always requires setting only four links or pointers. Contrast this with the brute-force approach of Chap. 1, which requires moving each record in the (sequential) list up by one location. If the list contained, say,

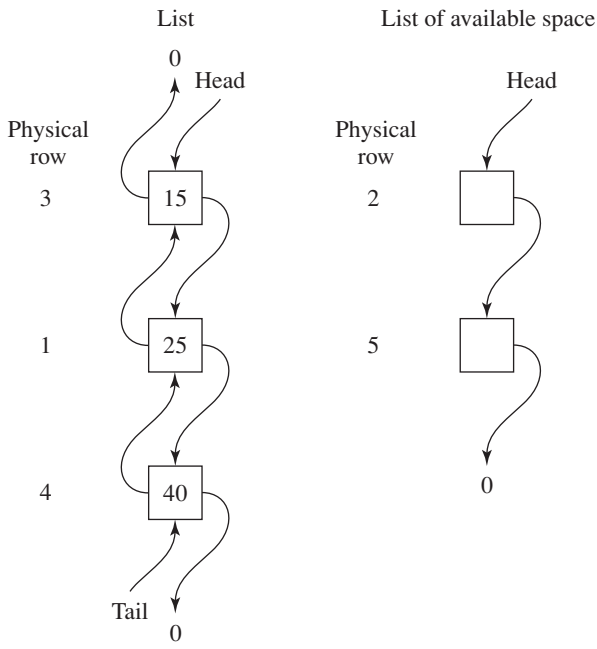


FIGURE 2.3
State of the lists for the queueing simulation at time 50.

100 records, this would be a much more time-consuming task than with the linked-list approach.

While storing a queue as a linked list as in the above example seems fairly natural, the next example illustrates how the event list can also be processed as a linked list.

EXAMPLE 2.2. For the inventory simulation of Sec. 1.5, the event list was stored in an array with each of the four event types having a dedicated physical location. If an event was not currently scheduled to occur, its entry in the list was set to ∞ (represented as 10^{30} in the computer). However, for many complex simulations written in a general-purpose language and for simulations using the special-purpose simulation software described in Chaps. 3 and 14, the event list is stored as a linked list ranked in increasing order on event time. Now, events having an event time of ∞ are simply not included in the event list. Moreover, since the event list is kept ranked in increasing order on the event times (attribute 1), the next event to occur will always be at the head of the list, so we need only remove this record to determine the next event time (attribute 1) and its type (attribute 2). For instance, suppose that the event list for the inventory simulation is to be stored in an array of 4 rows and 2 columns, column 1 being for the attribute "event time" and column 2 being for the attribute "event type," i.e., 1, 2, 3, or 4. Suppose that at time 0 we know that the first demand for the product (event type 2) will occur at time 0.25, the first inventory evaluation (event type 4) will occur immediately at time 0, the simulation will end (event type 3) at time 120, and there is no outstanding order scheduled to arrive (event type 1). The state of the event list and the LAS just after initialization at time 0 is shown in Fig. 2.4. Note that event type 1 is not included in the event list, and that event type 2 is in (physical) row 1 of the array since it was the first event record to be placed in the event list.

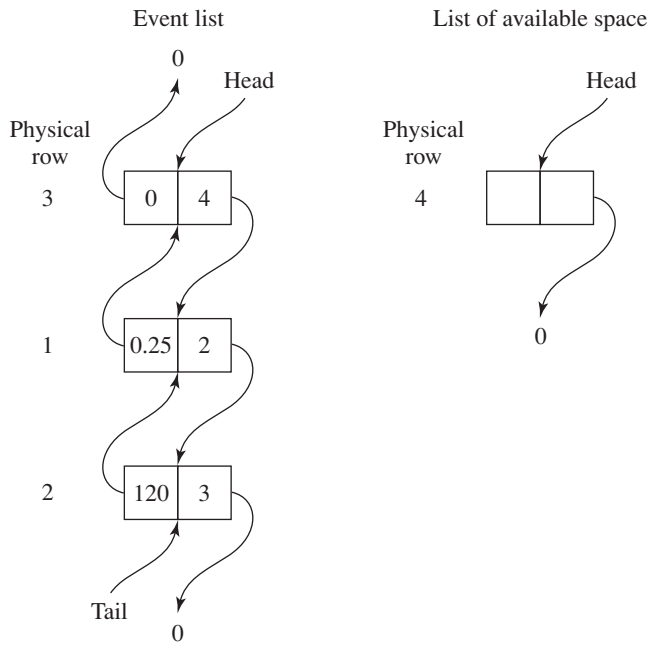


FIGURE 2.4
State of the lists for the inventory simulation just after initialization at time 0.

To determine the next (first) event to occur at time 0, the first record is removed from the event list, the simulation clock is updated to the first attribute of this record, i.e., the clock is set to 0, the event type of the next event to occur is set to the second attribute of this record, i.e., is set to 4, and row 3, which contained this record, is placed at the head of the LAS. Since the next event type is 4, an inventory-evaluation event will occur next (at time 0). Suppose that an order is placed at time 0 and that it will arrive from the supplier at time 0.6. To place this order-arrival event in the event list, first 0.6 and 1 are placed in columns 1 and 2, respectively, of row 3 (the head of the LAS), and then this new record is added to the event list by logically proceeding down the event list (using the successor links) until the correct location is found. In particular, attribute 1 of the new record (0.6) is first compared with attribute 1 of the record in row 1 (0.25). Since $0.6 > 0.25$, the new record should be farther down the event list. Next, 0.6 is compared with attribute 1 of the record in row 2 (120). (Note that the successor link of the record in row 1 is equal to 2.) Since $0.6 < 120$, the new record is logically placed between the records in physical rows 1 and 2 by adjusting the successor and predecessor links for the three records. After this has been done, another inventory-evaluation event is scheduled at time 1 and placed in the event list in a manner similar to that for the order-arrival event. The state of both lists after all processing has been done at time 0 is shown in Fig. 2.5.

In the above discussion, a single list was stored in an array where the empty rows were members of the LAS, but we could just as well store many different lists simultaneously in the same physical array. There is a single LAS, and the beginning

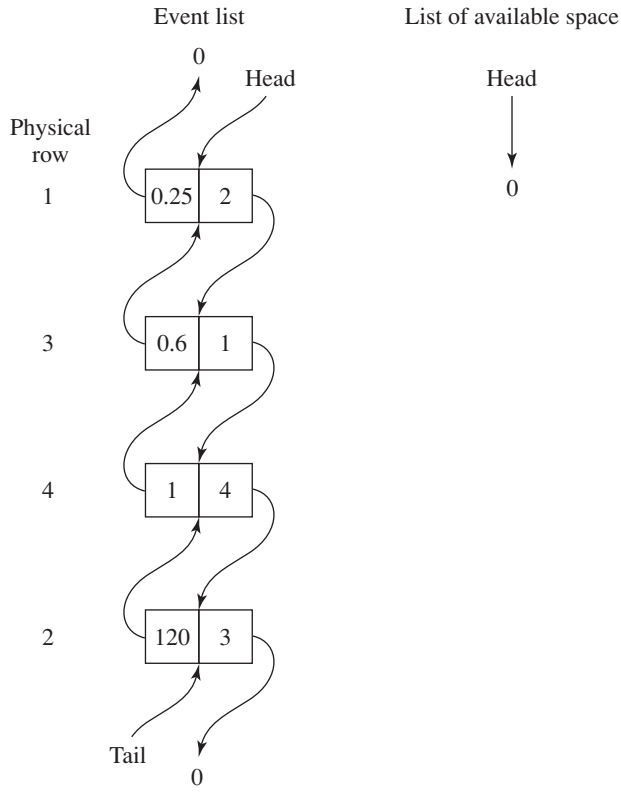


FIGURE 2.5
State of the lists for the inventory simulation after all processing has been done at time 0.

and end of each list are identified by separate head and tail pointers. This approach can lead to significant savings in storage space for some applications. For example, suppose that a simulation requires 20 lists, each containing up to 100 records of 10 attributes each. Using the sequential storage method (as in Chap. 1), 20 arrays with 100 rows and 10 columns each would be required, for a total storage requirement of 20,000 locations. Suppose, however, that at any given time an average of only 25 percent of all available rows are actually being used. Then an alternative approach might be to store all 20 lists in one array consisting of 1000 rows and 10 columns. This approach would require 10,000 locations for the array plus an additional 2040 locations for the links and pointers, for a total of 12,040 storage locations. Furthermore, at a particular point in a simulation, some lists may be occupying more than their "fair" share of the available rows without causing a memory overflow condition.

The simple simulation language simlib, developed in the next section, stores all lists (including the event list) in dynamic memory, which is allocated when needed to store a new record in a list, and is freed for other uses after a record is removed from a list.

2.3
A SIMPLE SIMULATION LANGUAGE: simlib

In this section we describe an easy-to-understand C-based simulation "language," simlib, which implements the concept of linked storage allocation presented in Sec. 2.2.2. The language makes it easy to file a record in a list (the record may be filed first in the list, last in the list, or so that the list is kept ranked in increasing or decreasing order on a specified attribute), to remove a record from a list (either the first record or the last record in the list may be removed), to process the event list, to compute discrete-time statistics on variables of interest (e.g., the average delay in queue in a queueing system), to compute continuous-time statistics on variables of interest (e.g., the time-average number of items in an inventory system), to generate the random variates used in the examples in this chapter and in Chap. 1, and to provide output in a "standard" format if desired. Although simlib provides many of the important features found in special-purpose simulation software (see Chaps. 3 and 14), it is designed for neither completeness nor computational efficiency. Our reasons for presenting it here are to provide some insight into the operation of simulation software and to provide a vehicle for understanding how to simulate systems that are much more complicated than those in Chap. 1.

The heart of simlib is a collection of doubly linked lists, all residing together in dynamic memory, with space allocated as new records are filed into the lists, and space freed as records are removed from the lists. There is a maximum of 25 lists, and the records in each list can have up to 10 attributes, with all data stored as type float. Since simlib uses dynamic storage allocation, the total number of records in all the lists is limited only by the amount of memory available on the computer.

List 25 is always reserved for the event list, with attribute 1 being the event time and attribute 2 being the event type. This list is furthermore kept sorted in increasing order on attribute 1 (event time) so that the top (first) record always refers to the next event.

To use simlib, the file simlib.h must be #included by the user. This file in turn #includes the file simlibdefs.h. These files are shown in Figs. 2.47 and 2.48 in App. 2A, and they contain (among other things) declarations and definitions of the following variables and constants germane to the user:

sim_time	The simulation clock, a float variable, updated by simlib function timing (see the discussion of function timing below)
next_event_type	The type of the next event, an int variable, determined by simlib function timing (see the discussion of function timing below)
transfer[i]	A float array with indices i = 1, 2, . . . , 10 (index i = 0 is not used) for transferring into and out of the lists used in the simulation, where transfer[i] refers to attribute i of a record. The transfer array is also used by simlib as a place to transfer to the user the values of certain summary statistics.
maxatr	The maximum number of attributes for any record in any list in the simulation model, an int variable. If maxatr is not set in the main function by the user, a default value of 10 is used; maxatr cannot be greater than 10.

	Setting maxatr to a value less than 10, if possible, will result in faster program execution. Because of the way simlib is written, the user must specify maxatr to be at least 4.
list_size[list]	The current number of records in list "list", an int array, which is automatically updated by simlib. The user should not need to alter the value of list_size[list], but only query its value if appropriate. For example, if list 3 has been set up to represent a queue, then list_size[3] will always be the number in this queue, so that we can tell if it is empty by asking if list_size[3] is equal to 0.
list_rank[list]	The attribute number, if any, according to which the records in list "list" are to be ranked (in increasing or decreasing order) by simlib function list_file; this is an int array. For example, if list 4 is to be kept so that the records are ranked on attribute 2, then the user would need to set list_rank[4] to be equal to 2 before using list_file to file a record in list 4; one of the arguments in function list_file controls whether the ranking is increasing or decreasing. Typically, list_rank[list] is set by the user in the main function. If list "list" is not ranked (i.e., records are inserted only at the end or the beginning), then list_rank[list] need not be set at all. Note that list_rank[25] is set to 1 in the simlib initialization routine init_simlib, since attribute 1 of the event list (list number 25) is always the event time, and we wish to keep the event list ranked in increasing order of the event times.
FIRST	A symbolic constant for the option of filing or removing a record at the beginning of a list, automatically set to 1 in simlibdefs.h
LAST	A symbolic constant for the option of filing or removing a record at the end of a list, automatically set to 2 in simlibdefs.h
INCREASING	A symbolic constant for the option of keeping a list ranked in increasing order according to the attribute specified in the list_rank array, automatically set to 3 in simlibdefs.h
DECREASING	A symbolic constant for the option of keeping a list ranked in decreasing order according to the attribute specified in the list_rank array, automatically set to 4 in simlibdefs.h
LIST_EVENT	A symbolic constant for the number of the event list, 25, automatically set to 25 in simlibdefs.h
EVENT_TIME	A symbolic constant for the attribute number of the event time in the event list, automatically set to 1 in simlibdefs.h
EVENT_TYPE	A symbolic constant for the attribute number of the event type in the event list, automatically set to 2 in simlibdefs.h

These variables and arrays are used or set by the user during the simulation, as appropriate, and must have the names and types given above, as declared in simlib.h.

There are some 19 functions composing simlib, each designed to perform a frequently occurring simulation activity:

- **init_simlib.** This function, to be invoked from the user-written main function at the beginning of each simulation run, allocates storage for the lists, initializes the successor and predecessor links as well as the head and tail pointers for each list, initializes the simulation clock to 0, sets list_rank[LIST_EVENT] to EVENT_TIME to keep the event list ranked on event times, and sets maxatr to a

default value of 10. Also, the statistical accumulators for functions sampst and timest (see their discussion below) are set to 0.

- **list_file(option, list).** This function places the attributes of a record that the user has placed in the transfer array into list "list" ("list" is an int) in a location controlled by the int argument "option". That is, when list_file is invoked, transfer[i] is placed in list "list" as attribute i of the new record, for i = 1, 2, . . . , maxatr. The following options are available:

option	Action
1 (or FIRST)	File the record in transfer before the first record currently in list "list".
2 (or LAST)	File the record in transfer after the last record currently in list "list".
3 (or INCREASING)	File the record in transfer into list "list" so that the list is kept ranked in increasing order on attribute list_rank[list], which must have been given a value previously. (If two records have the same value of attribute list_rank[list], the rule is FIFO.)
4 (or DECREASING)	File the record in transfer into list "list" so that the list is kept ranked in decreasing order on attribute list_rank[list], which must have been given a value previously. (If two records have the same value of attribute list_rank[list], the rule is FIFO.)

Thus, list_file(l, 3) would file transfer[1], . . . , transfer[maxatr] into list 3, with this becoming the first record in the list; list_file(FIRST, 3) would do the same thing. If we want list 2 to be kept ranked in decreasing order according to attribute 4 of the records in that list, then we could execute list_file(DECREASING, 2), making sure that we had set list_rank[2] to 4 previously, probably in the main function. Finally, we could schedule an event into the event list by executing list_file(INCREASING, LIST_EVENT) after setting (at least) transfer[1] to the time of occurrence of this event, and transfer[2] to the type of this event (see the description of simlib function event_schedule below, however, for an easier way to schedule events into the event list). Storage is dynamically allocated by list_file for the new record in list "list".

- **list_remove(option, list).** Invoking this function removes a record from list "list" (an int), and copies it (i.e., its attributes) into the transfer array. The int argument "option" determines which record is removed:

option	Action
1 (or FIRST)	Remove the first record from list "list" and place it into the transfer array.
2 (or LAST)	Remove the last record from list "list" and place it into the transfer array.

After invoking list_remove, the elements in transfer, now being equal to those in the record just removed, are typically used for some purpose in the simulation. For example, list_remove(2, 1) removes the last record in list 1 and places it in the transfer array; list_remove(LAST, 1) does the same thing. Storage is dynamically

freed by list_remove since space in list "list" for the record just removed is no longer needed.

- **timing.** This function, invoked by the user from the main function, accomplishes what the timing functions of Chap. 1 did, except now timing is an internal simlib function, and it uses the list structure of the event list exactly as described in Example 2.2. The type of the next event, next_event_type (a simlib int variable declared in simlib.h), is determined, and the clock, sim_time, is updated to the time of this next event. Actually, timing simply invokes list_remove(FIRST, LIST_EVENT) to remove the first record from list 25, the event list; since the event list is kept in increasing order on the event time, we know that this will be the next event to occur. Thus, attributes 1 through maxatr of the first event record are placed in transfer, and can be used if desired. In particular, it is sometimes advantageous to make use of attributes *other* than 1 and 2 in event records; see Secs. 2.6 and 2.7 as well as Prob. 2.3.
- **event_schedule(time_of_event, type_of_event).** The user can invoke this function to schedule an event to occur at time time_of_event (a float) of type type_of_event (an int) into the event list. Normally, event_schedule will be invoked to schedule an event in the simulated future, so time_of_event would be of the form sim_time + time_interval, where time_interval is an interval of time from now until the event is to happen. If attributes other than 1 (event time) and 2 (event type) of event records are being used in the event list, it is the user's responsibility to place their values in the appropriate locations in the transfer array before invoking event_schedule.
- **event_cancel(event_type).** This function cancels (removes) the first event in the event list with event type event_type (an int), if there is such an event, and places the attributes of the canceled event record in the transfer array. If the event list does not have an event of type event_type, no action is taken by event_cancel. If event_cancel finds an event of type event_type and cancels it, the function returns an int value of 1; if no such event is found, the function returns an int value of 0.
- **sampst(value, variable).** This function accumulates and summarizes discrete-time data, such as customer delays in a queue. There is provision for up to 20 "sampst variables," maintained and summarized separately, and indexed by the int argument "variable". For example, a model could involve three separate queues, and sampst variables 1, 2, and 3 could be used to accumulate and summarize customer delays in each of these queues, separately. There are three different typical uses of sampst:

  During the simulation. Each time a new value of sampst variable "variable" is observed (e.g., the end of a delay in queue), its value is placed in the float argument "value", and sampst is invoked. For instance, if we have defined sampst variable 2 to be the delays in queue 2 of a model, we could execute sampst(delay2, 2) after having placed the desired delay in the float variable delay2. The function sampst internally maintains separate registers for each variable, in which statistics are accumulated to produce the output described below.

At the end of the simulation. The user can invoke sampst with the *negative* of the variable "variable" desired, to produce summary statistics that are placed into the transfer array as follows:

i	transfer[i]
1	Mean (average) of the values of variable "variable" observed
2	Number of values of variable "variable" observed
3	Maximum value of variable "variable" observed
4	Minimum value of variable "variable" observed

In addition, sampst returns as a float the mean (in its name), the same thing that is placed into transfer[1], as a convenience since means are often desired. For example, executing sampst(0.0, −2) would place the summary statistics on sampst variable 2 in the transfer array, as described above, and return the mean; the desired summary statistics would then typically be written out by the user, or perhaps used in some other way. Note that in this use of sampst, the value of "value" is ignored. (A technicality: If no values for variable "variable" were observed, the mean, maximum, and minimum are undefined. In this case, sampst returns the mean as 0, the maximum as -10^{30}, and the minimum as 10^{30}.)

To reset all sampst variable accumulators. The accumulators for all sampst variables are reinitialized, as at the start of the simulation, by executing sampst(0.0, 0); note that this is done in init_simlib at time 0. This capability would be useful *during* a simulation if we wanted to start observing data only after the simulation had "warmed up" for some time, as described in Sec. 9.5.1; see also Prob. 2.7.

- **timest(value, variable).** This function is similar to sampst, but operates instead on continuous-time data such as the number-in-queue function; see Sec. 1.4.1. Again, the int argument "variable" refers to one of up to 20 "timest variables" on which data are accumulated and summarized when desired. For example, timest variables 1, 2, and 3 could refer to the number of customers in queues 1, 2, and 3, respectively. As with sampst, there are three "typical" uses:

  During the simulation. Each time timest variable "variable" attains a new value, we must execute timest(value, variable), where the float argument "value" contains the *new* (i.e., *after* the change) value of the variable. For example, if the length of queue 2 changes as a result of an arrival or departure to the float variable q2, we would execute timest(q2, 2) to do the proper accumulation. The accumulators for timest are initialized in init_simlib under the assumption that all continuous-time functions being tracked by timest are initially zero; this can be overridden by executing timest(value, variable) just after invoking init_simlib, where value contains the desired (nonzero) initial value of timest variable "variable". (This would be done for each desired timest variable.)

At the end of the simulation. The user can invoke timest with the *negative* of the variable "variable" desired, to produce summary statistics that are placed into the transfer array as follows:

i	transfer[i]
1	Time average of the values of variable "variable" observed, *updated to the time of this invocation*
2	Maximum value of variable "variable" observed up to the time of this invocation
3	Minimum value of variable "variable" observed up to the time of this invocation

In addition, timest returns as a float the time average (in its name), the same thing that is placed into transfer[1], as a convenience since means are often desired. For example, executing timest(0.0, −2) would place the summary statistics on timest variable 2 in the transfer array, as described above, and return the time average; the desired summary statistics would then typically be written out by the user, or perhaps used in some other way.

To reset all timest variable accumulators. The accumulators for all timest variables are reinitialized to zero, as at the start of the simulation, by executing timest(0.0, 0); this is done in init_simlib at time 0. Note that this assumes that all timest variables should have the value 0, which can be overridden by immediately executing timest(value, variable) to reset timest variable "variable" to the value "value".

- **filest(list).** This function, typically invoked only at the end of a simulation run, provides summary data on the number of records in list "list", placing them into the transfer array in a manner similar to timest, as follows:

i	transfer[i]
1	Time-average number of records in list "list", *updated to the time of this invocation*
2	Maximum number of records in list "list", up to the time of this invocation
3	Minimum number of records in list "list", up to the time of this invocation

In addition, filest returns as a float the time average (in its name), the same thing that is placed into transfer[1], as a convenience since means are often desired. Internally, simlib treats the number of records in a list as a continuous-time function whose value may rise or fall only at the times of events, so that it is sensible to speak of the time-average number of records in a list, etc. In addition, simlib automatically tracks each list in this way, and can produce these statistics when filest is invoked. Who cares about the history of list lengths? This capability turns out to be quite convenient, since the number of records in a list often has some physical meaning. For example, a queue will usually be represented in a simulation by a list, and the number of records in that list is thus identical to the number of customers in the queue; hence the time-average and maximum number of

customers in the queue are simply the time average and maximum of the number of records in the list. Another example is a list being used to represent a server, where the list will have one record in it if the server is busy and will be empty if the server is idle; the server utilization is thus the time-average number of records in this list, since the only possibilities for its length are 0 and 1. In this way, we can often (but not always) avoid explicitly tracking a continuous-time function via timest. However, timest should probably be used instead of filest if the corresponding list is set up merely for convenience in statistics collection, especially when the function being tracked rises or falls by increments other than 1 (e.g., the inventory level in the model of Sec. 1.5), due to the overhead in filing and removing many dummy records. Moreover, timest would have to be used instead of filest if the function being tracked can take on noninteger values. [Internally, simlib treats the number of records in list "list" as timest variable 20 + list, so that there are actually 45 timest variables, but only the first 20 are accessible to the user. Then filest simply invokes timest with variable = −(20 + list) to get statistics on list "list".]

- **out_sampst(unit, lowvar, highvar).** If desired, this function may be invoked to produce the summary statistics on sampst variables lowvar through highvar (inclusively) and write them to file "unit"; lowvar and highvar are both int arguments.This produces "standard" output format (which fits within an 80-character line), and obviates the need for the *final* invocation of sampst (but *not* the invocations during the course of the simulation) and also eliminates the need for fprintf statements, formatting, etc. The disadvantage of using out_sampst is that the annotation and layout of the output cannot be controlled or customized. For example, out_sampst(outfile, 1, 3) would write summary statistics to file outfile on sampst variables 1, 2, and 3; sampst(outfile, 4, 4) would write summary statistics on sampst variable 4. In the simlib simulations later in this chapter, we show examples of using (and ignoring) this standard-format output option.
- **out_timest(unit, lowvar, highvar).** Similar to out_sampst, this optional function may be used to produce standard-format output on file "unit" for timest variables lowvar through highvar.
- **out_filest(unit, lowfile, highfile).** This function uses filest to produce summary statistics on the number of records in files lowfile through highfile, written to file "unit"; lowfile and highfile are both int arguments.
- **expon(mean, stream).** This function returns a float with an observation from an exponential distribution with mean "mean" (a float argument). The int argument stream is the user-specified random-number stream number, discussed more fully in Sec. 7.1 and App. 7A. For now, we can think of the stream number as a separate, independent random-number generator (or list of random numbers) to be used for the purpose of generating the desired observations from the exponential distribution. It is generally a good idea to "dedicate" a random-number stream to a particular source of randomness in a simulation, such as stream 1 for interarrivals and stream 2 for service times, etc., to facilitate the use of *variance-reduction techniques* (see Chap. 11). These techniques can often provide a great improvement in the statistical precision of simulations. Furthermore, using dedicated streams can facilitate program verification (debugging). Except for the

stream specification, this is the same function used in Sec. 1.4.4 for generating observations from an exponential distribution. There are 100 separate streams available in simlib; i.e., "stream" must be an int between 1 and 100 inclusively, and the length of each stream is 100,000 random numbers.

- **random_integer(prob_distrib[], stream).** This function returns an int with an observation from a discrete probability distribution with *cumulative* distribution-function values specified by the user in the float array prob_distrib. For i a positive integer between 1 and 25, prob_distrib[i] should be specified by the user, before invoking random_integer, to be the desired probability of generating a value *less than or equal to* i. If the range of random integers desired is 1, 2, . . . , k, where k $<$ 25, then prob_distrib[k] should be set to 1.0, and it is then not necessary to specify prob_distrib[j] for j $>$ k. The int argument stream, between 1 and 100, gives the random-number stream to be used. Except for the random-number stream specification, this is the same function used for the inventory model in Sec. 1.5.3.
- **uniform(a, b, stream).** This function returns a float with an observation from a (continuous) uniform distribution between a and b (both float arguments). As before, stream is an int between 1 and 100 giving the random-number stream to be used.
- **erlang(m, mean, stream).** This function returns a float with an observation from an m-Erlang distribution with mean "mean" using random-number stream "stream"; m is an int, mean is a float, and stream is an int. This distribution will be discussed in Sec. 2.7.
- **lcgrand(stream).** This is the random-number generator used by simlib, a function returning a float with an observation from a (continuous) uniform distribution between 0 and 1, using stream "stream" (an int argument). Its code is given in Fig. 7.5 in App. 7A, instead of in App. 2A. When using simlib, and in particular #including the file simlib.h, it is not necessary to #include lcgrand.h from Fig. 7.6 in App. 7A, since simlib.h contains the required declarations.
- **lcgrandst(zset, stream).** This function "sets" the random-number *seed* for stream "stream" to the long argument zset. It is shown in Fig. 7.5 in App. 7A.
- **lcgrandgt(stream).** This function returns a long with the current underlying integer for the random-number generator for stream "stream"; it is shown in Fig. 7.5 in App. 7A, and discussed more fully in Chap. 7. It could be used to restart a subsequent simulation (using lcgrandst) from where the current one left off, as far as random-number usage is concerned.

This completes the description of simlib, but before proceeding with concrete examples of its use, we conclude this section with an overview of how simlib's components are typically used together in a simulation. It is still up to the user to write a C main function and event functions, but the simlib functions will make the coding much easier. First, we must determine the events and decide what lists will be used for what purpose; the numbering of lists and their attributes is in large measure arbitrary, but it is essential to be consistent. Also, any sampst and timest variables to be used must be defined, as should the usage of random-number streams. In addition to the global variables defined by simlib (via the header file simlib.h), the user will generally need to declare some global and perhaps local variables through

the model. In the main function, the following activities take place, roughly in the order listed:

1. Read and write (for confirmation) the input parameters.
2. Invoke init_simlib to initialize the simlib variables.
3. (If necessary) Set list_rank[list] to the attribute number on which list "list" is to be ranked, for lists that need to be kept in some sort of order according to the value of a particular attribute. (If no lists other than the event list are to be ranked, then this step is skipped.)
4. Set maxatr to the maximum number of attributes used in any list. Note that maxatr must be at least 4 for proper operation of simlib. If this is skipped, maxatr defaults to 10 and the simulation will run correctly, but setting maxatr to a smaller value will make the simulation faster, since it avoids repeated copying of unused attributes into and out of the lists.
5. (If necessary) Invoke timest to initialize any timest variables that are not to be zero initially.
6. Initialize the event list by invoking event_schedule for each event to be scheduled at time 0. If event attributes beyond the first (event time) and second (event type) are used in the data structure, it is the user's responsibility to set transfer[3], transfer[4], etc., before invoking event_schedule for that event. Events that cannot occur are simply not placed into the event list.
7. Invoke timing to determine next_event_type and update sim_time to the time of that event.
8. Invoke the appropriate event function (user-written, but using simlib variables and functions where possible), as determined by next_event_type. This is typically done with a case statement, routing control to one of several event-function-invocation statements, as was done in the C programs of Chap. 1.
9. When the simulation ends, invoke a report-generator function (user-written), which in turn will invoke sampst, timest, or filest and then write out the desired summary statistics. Alternatively, the report generator could invoke out_sampst, out_timest, or out_filest to write out the summary statistics in standard format.

While the simulation is running, lists are maintained by using list_file and list_remove, together with the transfer array to transfer the data in the attributes of records into and out of lists. When needed, sampst and timest are used to gather statistics on variables of interest.

A final note on simlib's capabilities concerns error checking. While no software package can detect all errors and suggest how to fix them, there are special opportunities to do some of this in simulation programs, as discussed in Chap. 1. Accordingly, simlib contains several such error checks, and will write out a message (to standard output) indicating the nature of the error and the clock value when it occurred. For example, simlib function timing checks for a "time reversal," i.e., an attempt to schedule an event at a time earlier than the present. Also, there are checks for illegal list numbers, illegal variable numbers, etc., and for attempting to remove a record from a list that is empty.

In Secs. 2.4 through 2.7 we show how to use simlib to simulate systems of varying complexity.

2.4 SINGLE-SERVER QUEUEING SIMULATION WITH simlib

2.4.1 Problem Statement

In this section we show how to simulate the single-server queueing system from Sec. 1.4, using simlib. The model is exactly the same, so that we can concentrate on the use of simlib without having to worry about the structure of a new model. We will use the 1000-delay stopping rule, as originally described in Sec. 1.4.3.

2.4.2 simlib Program

The first step is to identify the events; they are the same as before—an arrival is a type 1 event, and a departure (service completion) is a type 2 event.

Next, we must define the simlib lists and the attributes in their records. It is important to write this down, as it will be referenced while the program is developed:

List	Attribute 1	Attribute 2
1, queue	Time of arrival to queue	—
2, server	—	—
25, event list	Event time	Event type

Note that list 1 (representing the queue) is quite similar to the array time_arrival used in Sec. 1.4.4, except that now we are taking advantage of simlib's list-processing capabilities; list 1 has only a single attribute. List 2 represents the server and either will be empty (if the server is idle) or will contain a single record (if the server is busy); a record in list 2 when the server is busy is a "dummy" record, in that it has no actual attributes. The purpose for defining such a list is to allow the use of filest at the end of the simulation to get the server utilization. Also, we can tell whether the server is busy by asking whether list_size[2] is equal to 1. This use of a dummy list is convenient in that the server_status variable is eliminated, and we need not use timest during or at the end of the simulation to get the utilization. However, it is not the most computationally efficient approach, since all the machinery of the linked lists is invoked whenever the server changes status, rather than simply altering the value of a server_status variable. (This is a good example of the tradeoff between computation time and analyst's time in coding a model.) Finally, list 25 is the event list, with attribute 1 being event time and attribute 2 being event type; this is required in all simlib programs, but for some models we will use additional attributes for the event record in list 25.

Next, we should identify all sampst and timest variables used. Our only sampst variable is as follows:

sampst variable number	Meaning
1	Delays in queue

```
/* External definitions for single-server queueing system using simlib. */

#include "simlib.h"             /* Required for use of simlib.c. */

#define EVENT_ARRIVAL         1  /* Event type for arrival. */
#define EVENT_DEPARTURE       2  /* Event type for departure. */
#define LIST_QUEUE            1  /* List number for queue. */
#define LIST_SERVER           2  /* List number for server. */
#define SAMPST_DELAYS         1  /* sampst variable for delays in queue. */
#define STREAM_INTERARRIVAL   1  /* Random-number stream for interarrivals. */
#define STREAM_SERVICE        2  /* Random-number stream for service times. */

/* Declare non-simlib global variables. */

int   num_custs_delayed, num_delays_required;
float mean_interarrival, mean_service;
FILE  *infile, *outfile;

/* Declare non-simlib functions. */

void init_model(void);
void arrive(void);
void depart(void);
void report(void);
```

FIGURE 2.6
C code for the external definitions, queueing model with simlib.

Since we can obtain both the number-in-queue and utilization statistics using filest (or out_filest if standard output format is acceptable), we do not need any timest variables for this model.

Finally, we allocate separate random-number streams for generating the interarrival and service times, as follows:

Stream	Purpose
1	Interarrival times
2	Service times

Figure 2.6 shows the external declarations, which are at the top of the file mm1smlb.c. The first thing we do is #include the header file simlib.h, which is required for all programs using simlib. To make the code more readable and more general, we define symbolic constants for the event types, list numbers, the sampst variable, and the random-number streams. We must still declare some non-simlib variables for the model, though far fewer than in Sec. 1.4.4 since much of the information is held internally by simlib. And we must still have our own functions, init_model to initialize this particular model, arrive and depart for the events, and a report generator, but we no longer need a timing function or the expon function since they are provided by simlib.

Figure 2.7 shows the main function, which must still be written by the user. After opening the input and output files, we read the input parameters and then immediately write them out for confirmation that they were read properly (and to document our output). Invoking init_simlib initializes simlib, after which we set maxatr to 4; while we have no records with more than two attributes, maxatr must be at least 4 for simlib to operate properly. We are not using any ranked lists (other

```
main()  /* Main function. */
{
    /* Open input and output files. */

    infile  = fopen("mm1smlb.in",  "r");
    outfile = fopen("mm1smlb.out", "w");

    /* Read input parameters. */

    fscanf(infile, "%f %f %d", &mean_interarrival, &mean_service,
           &num_delays_required);

    /* Write report heading and input parameters. */

    fprintf(outfile, "Single-server queueing system using simlib\n\n");
    fprintf(outfile, "Mean interarrival time%11.3f minutes\n\n",
            mean_interarrival);
    fprintf(outfile, "Mean service time%16.3f minutes\n\n", mean_service);
    fprintf(outfile, "Number of customers%14d\n\n\n", num_delays_required);

    /* Initialize simlib */

    init_simlib();

    /* Set maxatr = max(maximum number of attributes per record, 4) */

    maxatr = 4;  /* NEVER SET maxatr TO BE SMALLER THAN 4. */

    /* Initialize the model. */

    init_model();

    /* Run the simulation while more delays are still needed. */

    while (num_custs_delayed < num_delays_required) {

        /* Determine the next event. */

        timing();

        /* Invoke the appropriate event function. */

        switch (next_event_type) {
            case EVENT_ARRIVAL:
                arrive();
                break;
            case EVENT_DEPARTURE:
                depart();
                break;
        }
    }

    /* Invoke the report generator and end the simulation. */

    report();

    fclose(infile);
    fclose(outfile);

    return 0;
}
```

FIGURE 2.7
C code for the main function, queueing model with simlib.

```
void init_model(void)  /* Initialization function. */
{
    num_custs_delayed = 0;

    event_schedule(sim_time + expon(mean_interarrival, STREAM_INTERARRIVAL),
                   EVENT_ARRIVAL);
}
```

FIGURE 2.8
C code for function init_model, queueing model with simlib.

than the event list), so we do not need to set anything in the list_rank array; also, both of the continuous-time functions (queue length and server status) are initially zero, so we need not override their default values. The user-written function init_model is then invoked to set up our own, non-simlib modeling variables. The rest of the main function is similar to Fig. 1.11 for the non-simlib version of this model, except that we need not update the continuous-time statistical accumulators since simlib takes care of that internally.

Figure 2.8 shows init_model, which begins by setting the num_custs_delayed counter to 0 for the number of delays observed. The first arrival event is then scheduled by invoking event_schedule with the desired event time (a float) as the first argument and the event type (an int) as the second argument; note that adding sim_time to the generated exponential interarrival time in the first argument is not strictly necessary here since sim_time is now zero, but we write it this way to show the general form and to emphasize that the first argument of event_schedule is the (absolute) time in the simulated future when the event is to occur, not the interval of time from now until then. In Chap. 1 we had to set the time of impossible events to ∞ (actually, 10^{30}), but now we simply leave them out of the event list, ensuring that they cannot be chosen to happen next. Thus, we just do not schedule a departure event at all here.

In Fig. 2.9 is the code for event function arrive, which begins by using event_schedule to schedule the next arrival event, in a manner similar to that in init_model (here, adding sim_time to the generated exponential interarrival time *is* necessary since sim_time will be positive). We then check to see whether the server is busy, by asking whether the server list contains a (dummy) record; this is done by checking whether list_size[LIST_SERVER] is equal to 1. If so, the arriving customer must join the end of the queue, which is done by placing the time of arrival (the current clock value, sim_time) into the first location of the transfer array, and by filing this record at the end (option = LAST = 2) of the queue list (list = LIST_QUEUE = 1). Note that we do not have to check for overflow of the queue here since simlib is automatically allocating storage dynamically for the lists as it is needed. On the other hand, if the server is idle, the customer experiences a delay of 0, which is noted by invoking sampst; this *is* necessary even though the delay is 0, since sampst will also increment the number of observations by 1. We increment num_custs_delayed since a delay is being observed, and a departure event is scheduled into the event list; note that we are dedicating stream EVENT_DEPARTURE (=2) to generating service times.

Event function depart, in Fig. 2.10, checks whether the queue is empty, by looking at the length of the queue list, held by simlib in list_size[LIST_QUEUE]. If so,

```
void arrive(void)  /* Arrival event function. */
{
    /* Schedule next arrival. */

    event_schedule(sim_time + expon(mean_interarrival, STREAM_INTERARRIVAL),
                   EVENT_ARRIVAL);

    /* Check to see whether server is busy (i.e., list SERVER contains a
       record). */

    if (list_size[LIST_SERVER] == 1) {

        /* Server is busy, so store time of arrival of arriving customer at end
           of list LIST_QUEUE. */

        transfer[1] = sim_time;
        list_file(LAST, LIST_QUEUE);
    }

    else {

        /* Server is idle, so start service on arriving customer, who has a
           delay of zero.  (The following statement IS necessary here.) */

        sampst(0.0, SAMPST_DELAYS);

        /* Increment the number of customers delayed. */

        ++num_custs_delayed;

        /* Make server busy by filing a dummy record in list LIST_SERVER. */

        list_file(FIRST, LIST_SERVER);

        /* Schedule a departure (service completion). */

        event_schedule(sim_time + expon(mean_service, STREAM_SERVICE),
                       EVENT_DEPARTURE);
    }
}
```

FIGURE 2.9
C code for function arrive, queueing model with simlib.

the server is made idle by removing the (dummy) record from list LIST_SERVER, the only action needed; note that we are removing the first record in the list, but we could also have removed the last one since there is only one record there. On the other hand, if there is a queue, the first customer is removed from it, and that customer's time of arrival is placed in transfer[1] by list_remove. The delay in queue of that customer is thus sim_time − transfer[1], which is computed and tallied in sampst, and the number of delays observed is incremented; as in the examples of Chap. 1, if the simulation were to be run for a long period of simulated time, it might be necessary to make both sim_time and transfer of type double to avoid loss of precision in the subtraction to compute the delay in queue. Finally, the service completion of this customer is scheduled by invoking event_schedule. Note that we need no longer move the queue up, since this is done internally by simlib, using linked lists as discussed in Example 2.1.

The report-generation function is shown in Fig. 2.11, and uses the standard output formatting in out_sampst for the delay-in-queue measure and out_filest for the

```
void depart(void)  /* Departure event function. */
{
    /* Check to see whether queue is empty. */

    if (list_size[LIST_QUEUE] == 0)

        /* The queue is empty, so make the server idle and leave the departure
           (service completion) event out of the event list. (It is currently
           not in the event list, having just been removed by timing before
           coming here.) */

        list_remove(FIRST, LIST_SERVER);

    else {

        /* The queue is nonempty, so remove the first customer from the queue,
           register delay, increment the number of customers delayed, and
           schedule departure. */

        list_remove(FIRST, LIST_QUEUE);
        sampst(sim_time - transfer[1], SAMPST_DELAYS);
        ++num_custs_delayed;
        event_schedule(sim_time + expon(mean_service, STREAM_SERVICE),
                       EVENT_DEPARTURE);
    }
}
```

FIGURE 2.10
C code for function depart, queueing model with simlib.

```
void report(void)  /* Report generator function. */
{
    /* Get and write out estimates of desired measures of performance. */

    fprintf(outfile, "\nDelays in queue, in minutes:\n");
    out_sampst(outfile, SAMPST_DELAYS, SAMPST_DELAYS);
    fprintf(outfile, "\nQueue length (1) and server utilization (2):\n");
    out_filest(outfile, LIST_QUEUE, LIST_SERVER);
    fprintf(outfile, "\nTime simulation ended:%12.3f minutes\n", sim_time);
}
```

FIGURE 2.11
C code for function report, queueing model with simlib.

number-in-queue and utilization measures. Note that we write out brief headers before invoking out_sampst and out_filest to make the report a little more readable.

2.4.3 Simulation Output and Discussion

The output file mm1smlb.out is given in Fig. 2.12, and illustrates the standard format produced by out_sampst and out_filest. We use general formatting of the numerical results to avoid the possibility of overflowing the field widths. We get all characteristics of the output measures, i.e., average, maximum, and minimum, as well as the number of observations for the discrete-time variables used by sampst. We also write out the final clock value, as a check.

An important point is that these numerical results are *not* the same as those in Fig. 1.19 for the non-simlib version of this same model; in fact, they are quite a bit different, with the average delay in queue changing from 0.430 in Chap. 1 to 0.525

```
Single-server queueing system using simlib

Mean interarrival time      1.000 minutes

Mean service time           0.500 minutes

Number of customers          1000


Delays in queue, in minutes:

 SAMPST                          Number
variable                           of
 number       Average            values          Maximum          Minimum
-------------------------------------------------------------------------------

    1      0.5248728E+00    0.1000000E+04    0.5633087E+01    0.0000000E+00
-------------------------------------------------------------------------------


Queue length (1) and server utilization (2):

  File          Time
 number        average           Maximum          Minimum
-------------------------------------------------------------------

    1      0.5400774E+00    0.8000000E+01    0.0000000E+00

    2      0.5106925E+00    0.1000000E+01    0.0000000E+00
-------------------------------------------------------------------


Time simulation ended:     971.847 minutes
```

FIGURE 2.12
Output report, queueing model with simlib.

here, a difference of some 22 percent. The reason for this is that we are now using the concept of "dedicating" a random-number stream to a particular source of randomness, while in Chap. 1 we used the same stream (number 1) for everything. Both programs are correct, and this illustrates the need for careful statistical analysis of simulation output data, as discussed in Chaps. 9 through 12.

While using simlib did simplify the coding of this model considerably, the value of such a package becomes more apparent in complex models with richer list structures. Such models are considered next, in Secs. 2.5 through 2.7.

2.5 TIME-SHARED COMPUTER MODEL

In this section we use simlib to simulate a model of a time-shared computer facility considered by Adiri and Avi-Itzhak (1969).

2.5.1 Problem Statement

A company has a computer system consisting of a single central processing unit (CPU) and n terminals, as shown in Fig. 2.13. The operator of each terminal "thinks"

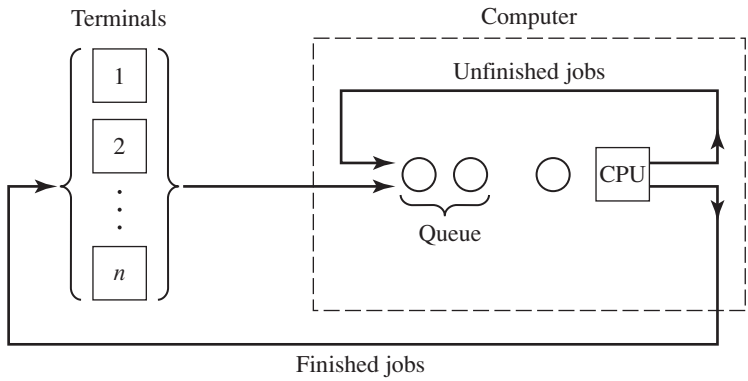


FIGURE 2.13
Time-shared computer model.

for an amount of time that is an exponential random variable with mean 25 seconds, and then sends to the CPU a job having service time distributed exponentially with mean 0.8 second. Arriving jobs join a single queue for the CPU but are served in a *round-robin* rather than FIFO manner. That is, the CPU allocates to each job a maximum service *quantum* of length $q = 0.1$ second. If the (remaining) service time of a job, s seconds, is no more than q, the CPU spends s seconds, plus a fixed swap time of $\tau = 0.015$ second, processing the job, which then returns to its terminal. However, if $s > q$, the CPU spends $q + \tau$ seconds processing the job, which then joins the end of the queue, and its remaining service time is decremented by q seconds. This process is repeated until the job's service is eventually completed, at which point it returns to its terminal, whose operator begins another think time.

Let R_i be the response time of the ith job to finish service, which is defined as the time elapsing between the instant the job leaves its terminal and the instant it is finished being processed at the CPU. For each of the cases $n = 10, 20, \ldots, 80$, we use simlib to simulate the computer system for 1000 job completions (response times) and estimate the expected average response time of these jobs, the expected time-average number of jobs waiting in the queue, and the expected utilization of the CPU. Assume that all terminals are in the think state at time 0. The company would like to know how many terminals it can have on its system and still provide users with an average response time of no more than 30 seconds.

2.5.2 simlib Program

The events for this model are:

Event description	Event type
Arrival of a job to the CPU from a terminal, at the end of a think time	1
End of a CPU run, when a job either completes its service requirement or has received the maximum processing quantum q	2
End of the simulation	3

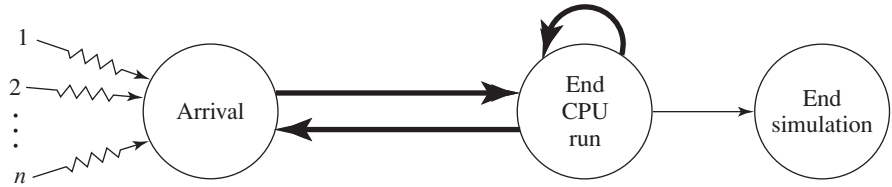


FIGURE 2.14
Event graph, computer model.

Note that we have defined an end-simulation event, even though the stopping rule for this model is not a fixed point in simulated time. The end-simulation event is scheduled at the time the 1000th response time is observed, and is scheduled to occur immediately, i.e., at that time. Clearly, there are other ways in which this stopping rule could be implemented, as discussed below.

An event graph (see Sec. 1.4.7) for this model is shown in Fig. 2.14. The n separate initializations of the arrival (i.e., end-think-time) event refer to the fact that each of the n terminals will be initially scheduled to have such an event. Note also that the arrival and end-CPU-run events can potentially schedule each other; an arrival can schedule the end of a CPU run if the arriving job finds the CPU idle, and an end-CPU-run event can schedule an arrival if the job exiting the CPU is finished and returns to its terminal. Also, an end-CPU-run event can schedule itself in the case of a job's leaving the CPU before its total processing is finished, and looping back to the queue only to find that the queue is empty and the CPU is idle since all other jobs are at their terminals. Finally, note that the end-simulation event can be scheduled only from the end-CPU-run event and in zero time, in the case that a finished job leaves the CPU and supplies the last (1000th) response time required; as discussed in Sec. 1.4.7, an event having incoming arcs that are all thin and smooth (representing a scheduling of the event in zero time from the event from which the smooth arrow emanates) can be eliminated from the model and its action incorporated elsewhere. Problem 2.2 deals with this issue for this model.

Three lists of records will be used, one corresponding to the jobs in queue (list 1), one for the job being served by the CPU (list 2), and one for the event list (list 25, as usual). These lists have the following attributes:

List	Attribute 1	Attribute 2
1, queue	Time of arrival of job to computer	Remaining service time
2, CPU	Time of arrival of job to computer	Remaining service time after the present CPU pass (negative if the present CPU pass is the last one needed for this job)
25, event list	Event time	Event type

As in Sec. 2.4, we are using a list to represent a server (list 2 for the CPU), thereby facilitating estimation of the CPU utilization via filest at the end of the simulation. Here, however, this "server" list's attributes are *not* dummies; they carry necessary information about the job in service, since it may have to revisit the CPU several

times, and its time of arrival and remaining service time must be carried along for it to be processed correctly. Note also that we have matched up the attributes in lists 1 and 2, so that transferring a record between these lists can be done simply by invoking list_remove and then list_file without having to rearrange the attributes in the transfer array. Finally, we are not explicitly keeping track of the terminal of origin for a job, so that when its processing is complete in the computer we do not know what terminal to send it back to in order for its next think time to begin. This would certainly not do in reality, since the terminal operators would be getting each others' output back, but in the *simulation* we need not represent the ownership of each job by a particular terminal since we want only overall (i.e., over all the terminals) performance measures for the response times. Furthermore, all the terminals are probabilistically identical, as the think-time and CPU-time distributions are the same. Problem 2.3 asks that the model be enriched by collecting separate response-time statistics for each terminal, and allowing the terminal characteristics to vary; any of these changes would require that the terminal of origin of a job be carried along with it while it is inside the computer.

Since there is only a single discrete-time statistic of interest (the response times), we need only a single sampst variable:

sampst variable number	Meaning
1	Response times

For each of the continuous-time statistics desired (number in queue and CPU utilization), there is a corresponding list whose length represents the desired quantity, so we can again obtain the output via filest, and we do not need any of our own timest variables.

There are two types of random variables for this model, and we use the following stream assignments:

Stream	Purpose
1	Think times
2	Service times

Figure 2.15 shows the global external definitions for this model, which are at the top of the file tscomp.c. After #including the header file simlib.h, we define symbolic constants for the event types, list numbers, sampst variable number, and random-number streams. The non-simlib variables are declared, including ints for the minimum and maximum number of terminals across the simulations (min_terms = 10 and max_terms = 80), the increment in the number of terminals across the simulations (incr_terms = 10), the number of terminals for a particular simulation (num_terms), the number of response times observed in the current simulation (num_responses), and the number of responses required (num_responses_required = 1000); floats are required only for the input parameters for mean think and service times, the quantum q, and the swap time τ. The user-written, non-simlib functions are declared, including event functions for arrival and end-CPU-run events; we have written a non-event function, start_CPU_run, to process a particular activity that

```
/* External definitions for time-shared computer model. */

#include "simlib.h"                /* Required for use of simlib.c. */

#define EVENT_ARRIVAL          1  /* Event type for arrival of job to CPU. */
#define EVENT_END_CPU_RUN      2  /* Event type for end of a CPU run. */
#define EVENT_END_SIMULATION   3  /* Event type for end of the simulation. */
#define LIST_QUEUE             1  /* List number for CPU queue. */
#define LIST_CPU               2  /* List number for CPU. */
#define SAMPST_RESPONSE_TIMES  1  /* sampst variable for response times. */
#define STREAM_THINK           1  /* Random-number stream for think times. */
#define STREAM_SERVICE         2  /* Random-number stream for service times. */

/* Declare non-simlib global variables. */

int   min_terms, max_terms, incr_terms, num_terms, num_responses,
      num_responses_required, term;
float mean_think, mean_service, quantum, swap;
FILE  *infile, *outfile;

/* Declare non-simlib functions. */

void arrive(void);
void start_CPU_run(void);
void end_CPU_run(void);
void report(void);
```

FIGURE 2.15
C code for the external definitions, computer model.

may occur when either an arrival or an end-CPU-run event occurs, avoiding having to repeat this same block of code in each of those event functions. There is not a separate model-initialization function since there is little required to initialize this model (beyond what simlib does in init_simlib), so this activity was simply put into the main function. For this model we chose not to use the standard output formatting option, since we are really doing eight separate simulations and would like to arrange the output data in a customized table, with one line per simulation; also, we wish to get only the mean of the output performance measures rather than all of their characteristics (maximum, etc.).

The main function is shown in Fig. 2.16. As usual, we open the input and output files, read the input parameters, and then write them out in a report heading. Since we will be doing eight simulations, with one output line per simulation, we also write out column headings for the output at this time. A "for" loop setting num_terms in turn to 10, 20, . . . , 80 then begins, and encompasses the rest of the main function except for closing the files at the very end; a separate simulation is run, including initialization and results writing, within this "for" loop for each value of num_terms. Each simulation begins with a fresh initialization of simlib by invoking init_simlib, then sets maxatr to 4, initializes num_responses to 0, and schedules the first arrival to the CPU from each terminal by invoking event_schedule for each of the num_terms terminals. Note that we will then have num_terms events scheduled in the event list, all of type 1, each one representing the end of the initial think time for a particular terminal. A "do while" loop then starts, invoking the timing function and the appropriate event function as long as the event type is not the end-simulation event; when the event just executed is the end-simulation event (and report has been invoked), the "do while" loop ends and we go back to the enclosing

```
main()  /* Main function. */
{
    /* Open input and output files. */

    infile  = fopen("tscomp.in",  "r");
    outfile = fopen("tscomp.out", "w");

    /* Read input parameters. */

    fscanf(infile, "%d %d %d %d %f %f %f %f",
           &min_terms, &max_terms, &incr_terms, &num_responses_required,
           &mean_think, &mean_service, &quantum, &swap);

    /* Write report heading and input parameters. */

    fprintf(outfile, "Time-shared computer model\n\n");
    fprintf(outfile, "Number of terminals%9d to%4d by %4d\n\n",
            min_terms, max_terms, incr_terms);
    fprintf(outfile, "Mean think time  %11.3f seconds\n\n", mean_think);
    fprintf(outfile, "Mean service time%11.3f seconds\n\n", mean_service);
    fprintf(outfile, "Quantum          %11.3f seconds\n\n", quantum);
    fprintf(outfile, "Swap time        %11.3f seconds\n\n", swap);
    fprintf(outfile, "Number of jobs processed%12d\n\n\n",
            num_responses_required);
    fprintf(outfile, "Number of      Average         Average");
    fprintf(outfile, "       Utilization\n");
    fprintf(outfile, "terminals   response time  number in queue     of CPU");

    /* Run the simulation varying the number of terminals. */

    for (num_terms = min_terms; num_terms <= max_terms;
         num_terms += incr_terms) {

        /* Initialize simlib */

        init_simlib();

        /* Set maxatr = max(maximum number of attributes per record, 4) */

        maxatr = 4;  /* NEVER SET maxatr TO BE SMALLER THAN 4. */

        /* Initialize the non-simlib statistical counter. */

        num_responses = 0;

        /* Schedule the first arrival to the CPU from each terminal. */

        for (term = 1; term <= num_terms; ++term)
            event_schedule(expon(mean_think, STREAM_THINK), EVENT_ARRIVAL);

        /* Run the simulation until it terminates after an end-simulation event
           (type EVENT_END_SIMULATION) occurs. */

        do {

            /* Determine the next event. */

            timing();

            /* Invoke the appropriate event function. */

            switch (next_event_type) {
                case EVENT_ARRIVAL:
                    arrive();
                    break;
```

FIGURE 2.16
C code for the main function, computer model.

```
                case EVENT_END_CPU_RUN:
                    end_CPU_run();
                    break;
                case EVENT_END_SIMULATION:
                    report();
                    break;
            }

        /* If the event just executed was not the end-simulation event (type
           EVENT_END_SIMULATION), continue simulating.  Otherwise, end the
           simulation. */

        } while (next_event_type != EVENT_END_SIMULATION);
    }

    fclose(infile);
    fclose(outfile);

    return 0;
}
```

FIGURE 2.16
(*continued*)

"for" loop on the number of terminals. The end-simulation event is not scheduled initially, but will be scheduled in the function end_CPU_run at the time of the 1000th response-time completion, to occur at that time, whereupon the main function will invoke function report and end the current simulation. When the outer "for" loop ends, we have run all the simulations we want, including producing the output for each, and so the entire program ends by closing the files.

The arrival event is flowcharted in Fig. 2.17 and the code is in Fig. 2.18. While in the computer (i.e., in the queue or in the CPU), each job has its own record with

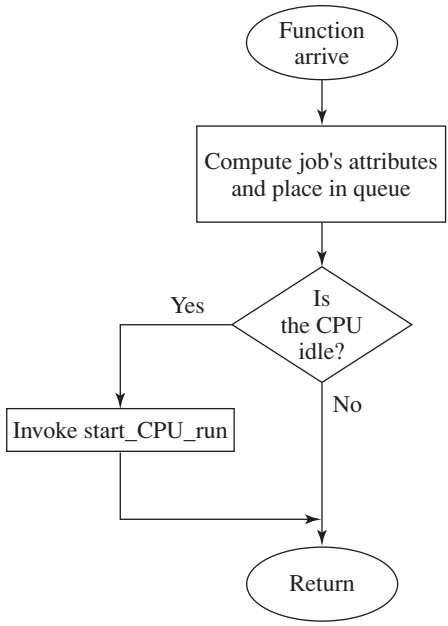


FIGURE 2.17
Flowchart for arrival function, computer model.

```
void arrive(void)  /* Event function for arrival of job at CPU after think
                      time. */
{

    /* Place the arriving job at the end of the CPU queue.
       Note that the following attributes are stored for each job record:
            1. Time of arrival to the computer.
            2. The (remaining) CPU service time required (here equal to the
               total service time since the job is just arriving). */

    transfer[1] = sim_time;
    transfer[2] = expon(mean_service, STREAM_SERVICE);
    list_file(LAST, LIST_QUEUE);

    /* If the CPU is idle, start a CPU run. */

    if (list_size[LIST_CPU] == 0)
        start_CPU_run();
}
```

FIGURE 2.18
C code for function arrive, computer model.

attributes as described earlier. Since this event represents a job's arrival to the computer at the end of a think time, its attributes must be defined now, so the time of arrival is stored in the first attribute and the total service requirement is generated and stored in the second attribute. The record for this newly arrived job is then placed at the end of the queue. It could be, however, that the CPU is actually idle (i.e., the number of records in list LIST_CPU, list_size[LIST_CPU], is equal to 0), in which case the function start_CPU_run is invoked to take this job out of the queue (it would be the only one there) and place it in the CPU to begin its processing. Implicit in the logic of this function is that a job arriving to the computer and finding the CPU idle cannot just go right in, but must first enter the queue and then be removed immediately; this is really a physical assumption that does matter, since there is a swap time incurred whenever a job leaves the queue and enters the CPU, as executed by function start_CPU_run, to be discussed next.

The non-event function start_CPU_run is flowcharted in Fig. 2.19, and its code is in Fig. 2.20. This function is designed to be invoked from the event function arrive, as just discussed, or from the event function end_CPU_run; thus it must be general enough to handle either case. The purpose of the function is to take the first job out of the queue, place it in the CPU, and schedule the time when it will leave the CPU, by virtue of either being completely done or having used up an entire quantum. The first thing to do is to remove the job from the front of the queue, which is done by invoking list_remove. Next, the time that the job will occupy the CPU is computed, being the smaller of a quantum and the remaining service time (held in the job's second attribute, having just been placed in transfer[2] by list_remove), plus a swap time. Before filing the job's record in the CPU list, its remaining service time (in transfer[2]) is decremented by a full quantum, even if it needs only a partial quantum to get done; in this case the second attribute of the job becomes negative, and we use this condition as a flag that the job, when leaving the CPU after the pass just beginning, is done and is to be sent back to its terminal. On the other hand, if the job will not be done after this pass through the CPU, it will be getting a full

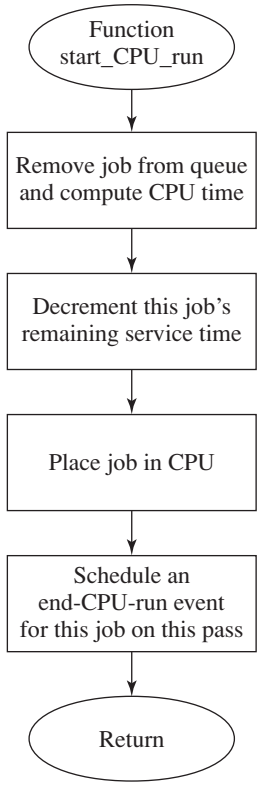


FIGURE 2.19
Flowchart for function start_CPU_run, computer model.

```
void start_CPU_run(void)  /* Non-event function to start a CPU run of a job. */
{
    float run_time;

    /* Remove the first job from the queue. */

    list_remove(FIRST, LIST_QUEUE);

    /* Determine the CPU time for this pass, including the swap time. */

    if (quantum < transfer[2])
        run_time = quantum + swap;
    else
        run_time = transfer[2] + swap;

    /* Decrement remaining CPU time by a full quantum.  (If less than a full
       quantum is needed, this attribute becomes negative.  This indicates that
       the job, after exiting the CPU for the current pass, will be done and is
       to be sent back to its terminal.) */

    transfer[2] -= quantum;

    /* Place the job into the CPU. */

    list_file(FIRST, LIST_CPU);

    /* Schedule the end of the CPU run. */

    event_schedule(sim_time + run_time, EVENT_END_CPU_RUN);
}
```

FIGURE 2.20
C code for function start_CPU_run, computer model.

quantum of service this time, and its second attribute should be decremented by a full quantum, correctly representing the (nonnegative) remaining service time needed after this CPU pass. Finally, the job is placed in the CPU list by invoking list_file (note that transfer[1], the time of this job's arrival to the computer, is already correctly set since it is from the record that was just removed from the queue list, where the first attribute has the same definition), and the end of this CPU run is scheduled into the event list.

Event function end_CPU_run is invoked from the main function when a job completes a pass through the CPU; it is flowcharted in Fig. 2.21, and its code is listed in Fig. 2.22. The job is first removed from the CPU, after which a check is made to see if it still needs more CPU time, i.e., if its second attribute is positive. If so, it is simply put back at the end of the queue (note that the attributes for the queue and CPU lists match up, so that the contents of transfer are correct), and start_CPU_run is invoked to remove the first job from the queue and begin its processing. On the other hand, if the job coming out of the CPU is finished, its response time, sim_time − transfer[1], is registered with sampst; as before for long simulations, both sim_time and transfer might have to be of type double to avoid loss of precision in this subtraction. The end of its next think time is scheduled, and the number of response times observed is incremented. Then a check is made to see whether this response time was the last one required; if so, an end-simulation event is scheduled to occur immediately (the first argument passed to event_schedule is sim_time, the current simulation time), and the timing function will pick off this end-simulation event immediately (i.e., without the passage of any simulated time), and the main function will invoke the report function to end this simulation. If, however, the simulation is not over, start_CPU_run is invoked provided that the queue is not empty; if the queue is empty, no action is taken and the simulation simply continues.

The report generator is listed in Fig. 2.23, and it simply writes to the output file a single line with the number of terminals for the simulation just completed, the average returned by sampst for the response times, and the time averages returned by filest for the queue length and server utilization (recall that sampst, timest, and filest return in their names the averages they compute, in addition to placing the average into transfer[1]).

2.5.3 Simulation Output and Discussion

The output file, tscomp.out, is shown in Fig. 2.24. As expected, congestion in the computer gets worse as the number of terminals rises, as measured by the average response time, average queue length, and CPU utilization. In particular, it appears that this system could handle about 60 terminals before the average response time degrades to a value much worse than 30 seconds. At this level, we see that the average queue length would be around 30 jobs, which could be useful for determining the amount of space needed to hold these jobs (the maximum queue length might have been a better piece of information for this purpose); further, the CPU would be busy nearly all the time with such a system. However, our usual caveat applies to

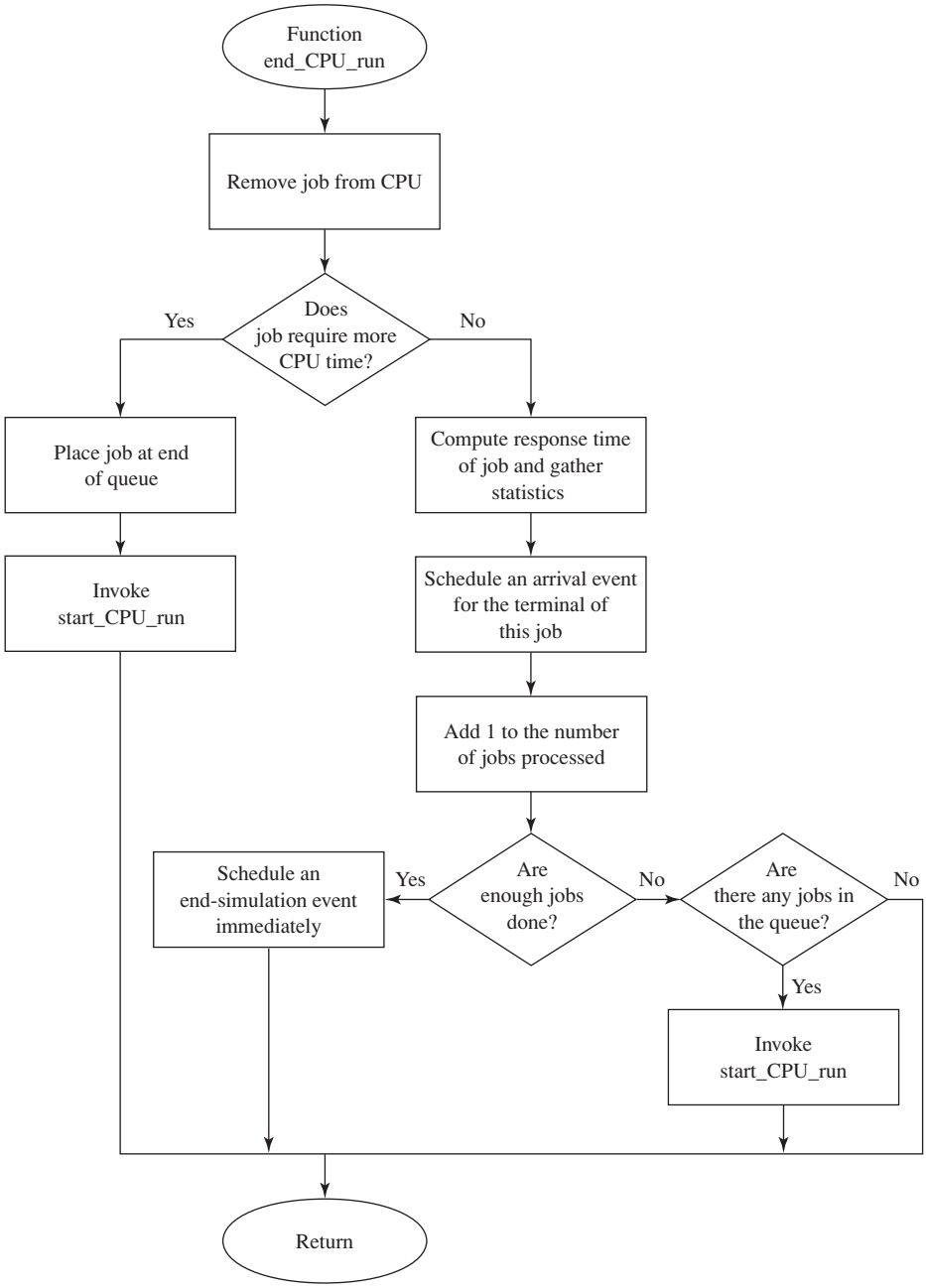


FIGURE 2.21
Flowchart for function end_CPU_run, computer model.

```
void end_CPU_run(void)  /* Event function to end a CPU run of a job. */
{
    /* Remove the job from the CPU. */

    list_remove(FIRST, LIST_CPU);

    /* Check to see whether this job requires more CPU time. */

    if (transfer[2] > 0.0) {

        /* This job requires more CPU time, so place it at the end of the queue
           and start the first job in the queue. */

        list_file(LAST, LIST_QUEUE);
        start_CPU_run();
    }

    else {

        /* This job is finished, so collect response-time statistics and send it
           back to its terminal, i.e., schedule another arrival from the same
           terminal. */

        sampst(sim_time - transfer[1], SAMPST_RESPONSE_TIMES);

        event_schedule(sim_time + expon(mean_think, STREAM_THINK),
                       EVENT_ARRIVAL);

        /* Increment the number of completed jobs. */

        ++num_responses;

        /* Check to see whether enough jobs are done. */

        if (num_responses >= num_responses_required)

            /* Enough jobs are done, so schedule the end of the simulation
               immediately (forcing it to the head of the event list). */

            event_schedule(sim_time, EVENT_END_SIMULATION);

        else

            /* Not enough jobs are done; if the queue is not empty, start
               another job. */

            if (list_size[LIST_QUEUE] > 0)
                start_CPU_run();
    }
}
```

FIGURE 2.22
C code for function end_CPU_run, computer model.

```
void report(void)  /* Report generator function. */
{
    /* Get and write out estimates of desired measures of performance. */

    fprintf(outfile, "\n\n%5d%16.3f%16.3f%16.3f", num_terms,
            sampst(0.0, -SAMPST_RESPONSE_TIMES), filest(LIST_QUEUE),
            filest(LIST_CPU));
}
```

FIGURE 2.23
C code for function report, computer model.

```
Time-shared computer model

Number of terminals       10 to  80 by   10

Mean think time       25.000 seconds

Mean service time      0.800 seconds

Quantum                0.100 seconds

Swap time              0.015 seconds

Number of jobs processed        1000


Number of      Average         Average       Utilization
terminals   response time  number in queue     of CPU

   10           1.324           0.156           0.358

   20           2.165           0.929           0.658

   30           5.505           4.453           0.914

   40          12.698          12.904           0.998

   50          24.593          23.871           0.998

   60          31.712          32.958           1.000

   70          42.310          42.666           0.999

   80          47.547          51.158           1.000
```

FIGURE 2.24
Output report, computer model.

these conclusions: The output data on which they are based resulted from just a single run of the system (of somewhat arbitrary length) and are thus of unknown precision.

2.6 MULTITELLER BANK WITH JOCKEYING

We now use simlib to simulate a multiteller bank where the customers are allowed to jockey (move) from one queue to another if it seems to be to their advantage. This model also illustrates how to deal with another common stopping rule for a simulation.

2.6.1 Problem Statement

A bank with five tellers opens its doors at 9 A.M. and closes its doors at 5 P.M., but operates until all customers in the bank by 5 P.M. have been served. Assume that the interarrival times of customers are IID exponential random variables with mean 1 minute and that service times of customers are IID exponential random variables with mean 4.5 minutes.

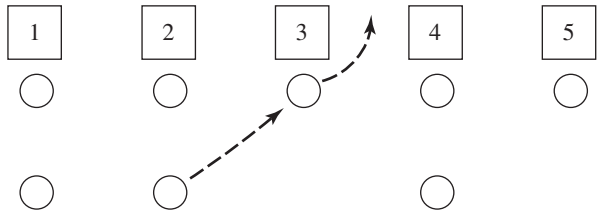


FIGURE 2.25
The customer being served by teller $i = 3$ completes service, causing the customer from the tail of queue $j = 2$ to jockey.

Each teller has a separate queue. An arriving customer joins the shortest queue, choosing the leftmost shortest queue in case of ties. Let n_i be the total number of customers in front of teller i (including customers in queue as well as the customer in service, if any) at a particular instant. If the completion of a customer's service at teller i causes $n_j > n_i + 1$ for some other teller j, then the customer from the tail of queue j jockeys to the tail of queue i. (If there are two or more such customers, the one from the closest, leftmost queue jockeys.) If teller i is idle, the jockeying customer begins service at teller i; see Fig. 2.25.

The bank's management is concerned with operating costs as well as the quality of service currently being provided to customers, and is thinking of changing the number of tellers. For each of the cases $n = 4$, 5, 6, and 7 tellers, we use simlib to simulate the bank and estimate the expected time-average total number of customers in queue, the expected average delay in queue, and the expected maximum delay in queue. In all cases we assume that no customers are present when the bank opens.

2.6.2 simlib Program

The events for this model are:

Event description	Event type
Arrival of a customer to the bank	1
Departure of a customer upon completion of his or her service	2
Bank closes its doors at 5 P.M.	3

An event graph for this model is given in Fig. 2.26. It is identical to that for the single-server queue with fixed run length (see Fig. 1.27), except that the end-simulation event has been replaced by the close-doors event. Even though these two events fit into the event diagram in the same way, the action they must take is quite different.

This model requires $2n + 1$ lists of records, where n is the number of tellers for a particular simulation run. Lists 1 through n contain the records of the customers waiting in the respective queues. Lists $n + 1$ through $2n$ are used to indicate

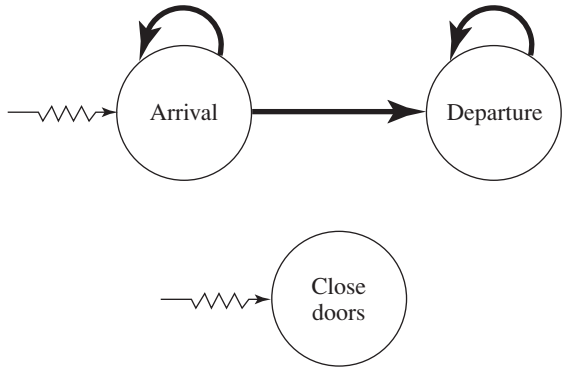


FIGURE 2.26
Event graph, bank model.

whether the tellers are busy. If list $n + i$ (where $i = 1, 2, \ldots, n$) contains one record, teller i is busy; if it contains no records, teller i is idle. Finally, list 25 is the event list, as usual. The attributes for all these lists are as follows:

List	Attribute 1	Attribute 2	Attribute 3
1 through n, queues	Time of arrival to queue	—	—
$n + 1$ through $2n$, tellers	—	—	—
25, event list	Event time	Event type	Teller number if event type = 2

Here again we are using separate lists for the servers in the model; in this case the only reason for doing so is to represent the busy/idle status of the servers, since no meaningful information is stored in the attributes of the records in these lists, and we are not asking for server utilization statistics. Note also that we are taking advantage of the opportunity to store more than just the event time and type in the records in the event list. The reason for this is that in the case of a departure event (type 2), we need to know the teller number from which the departure will occur in order to manage the queues and the jockeying rules correctly. A programming implication of this is that we must remember to define a value for transfer[3] before invoking the simlib function event_schedule for type 2 events, since event_schedule copies the attributes into only transfer[1] and transfer[2] before filing the event record into the event list.

The statistics collection in this model is somewhat different. Since there are several different queues, a customer may experience his or her delay (the time elapsing between arrival to the system and commencement of service at *some* server) in several different queues, due to the possibility of jockeying. The customer carries along the time of his or her arrival (in attribute 1 of the queue lists) regardless of what queue he or she may be in, so that the delay can be computed

when service begins. Thus, we simply lump all the customer delays together into a single sampst variable:

sampst variable number	Meaning
1	Delay in queue (or queues)

By using sampst, we will automatically get the maximum delay in queue (or queues) as well.

We also want to get the time-average total number of customers in queue, which is computed as follows. If we let $Q_i(t)$ be the number of customers in queue i at time t, for $i = 1, 2, \ldots, n$, then

$$Q(t) = \sum_{i=1}^{n} Q_i(t) \tag{2.1}$$

is the total number of customers in all the queues at time t. Thus, what we want to compute is

$$\hat{q} = \frac{\int_0^T Q(t)\, dt}{T} \tag{2.2}$$

where T is the time the simulation ends (as determined by the stopping rule described above). However, if we substitute Eq. (2.1) into Eq. (2.2) and use linearity of integrals, we get

$$\hat{q} = \hat{q}_1 + \hat{q}_2 + \cdots + \hat{q}_n$$

where

$$\hat{q}_i = \frac{\int_0^T Q_i(t)\, dt}{T}$$

is simply the time-average number of customers in queue i. All this really says is that the average of the sum of the individual queue lengths is the sum of their average lengths. Thus, we can use filest (applied to the lists for the individual queues) at the end of the simulation to obtain the $\hat{q}_i$'s, and then just add them together to get $\hat{q}$. To be sure, $\hat{q}$ could be obtained directly by defining a timest variable corresponding to $Q(t)$, incrementing it upon each arrival, and decrementing it with the commencement of each service; but we have to keep the queue lists anyway, so the above approach is preferred. (Problem 2.4 considers an extension of this model where we want to know the maximum total number of customers in the queues as well as the above statistics; the question addressed there concerns whether the maximum of the total is equal to the total of the maxima.)

There are two types of random variables in this model: interarrival times and service times. We use the following stream assignments:

Stream	Purpose
1	Interarrival times
2	Service times

```
/* External definitions for multiteller bank. */

#include "simlib.h"             /* Required for use of simlib.c. */

#define EVENT_ARRIVAL        1  /* Event type for arrival of a customer. */
#define EVENT_DEPARTURE      2  /* Event type for departure of a customer. */
#define EVENT_CLOSE_DOORS    3  /* Event type for closing doors at 5 P.M. */
#define SAMPST_DELAYS        1  /* sampst variable for delays in queue(s). */
#define STREAM_INTERARRIVAL  1  /* Random-number stream for interarrivals. */
#define STREAM_SERVICE       2  /* Random-number stream for service times. */

/* Declare non-simlib global variables. */

int   min_tellers, max_tellers, num_tellers, shortest_length, shortest_queue;
float mean_interarrival, mean_service, length_doors_open;
FILE  *infile, *outfile;

/* Declare non-simlib functions. */

void arrive(void);
void depart(int teller);
void jockey(int teller);
void report(void);
```

FIGURE 2.27
C code for the external definitions, bank model.

Figure 2.27 shows the external definitions and global variables for the model. As usual, we #include the simlib header file simlib.h and then define symbolic constants for the event types, sampst variable, and random-number stream numbers. Next, ints are declared for the minimum and maximum number of tellers (4 and 7, respectively) for the different simulations we will carry out, and for the number of tellers for a given simulation; the "short" ints pertain to queue-selection decisions of arriving customers. The input parameters are declared as floats; length_doors_open is assumed to be read in units of hours, while the time units used elsewhere in the simulation are minutes, so an adjustment must be made in the code for this. The functions are prototyped, with arrive being for type 1 events, depart for type 2 events (with an int argument giving the teller number from which the departure is to occur), jockey being a non-event function with int argument being the teller number where a service is being completed (so is the possible destination for a jockeying customer), and report writing the results when the simulation ends at *or after* 5 P.M.

The main function is shown in Fig. 2.28, and it begins by opening the input and output files, reading the input parameters, writing them back out, and producing a report heading. As in the computer model, there is a for loop around most of the main function, with the index num_tellers representing the number of tellers *n* for the current model variant. Invoking init_simlib initializes simlib (note that this must be done for each model variant, so is inside the for loop), and maxatr is set to 4 (we have no more than 3 attributes in any of our records, but maxatr can be no less than 4 for simlib to work properly). The first arrival is scheduled, and the close-doors event is also scheduled, taking care to change the time units to minutes. A while loop then begins, continuing to run the current simulation so long as the event list is not empty, after which the current simulation is terminated; some explanation is required to argue why this is a valid way to implement the termination rule for

```
main()  /* Main function. */
{
    /* Open input and output files. */

    infile  = fopen("mtbank.in",  "r");
    outfile = fopen("mtbank.out", "w");

    /* Read input parameters. */

    fscanf(infile, "%d %d %f %f %f", &min_tellers, &max_tellers,
           &mean_interarrival, &mean_service, &length_doors_open);

    /* Write report heading and input parameters. */

    fprintf(outfile, "Multiteller bank with separate queues & jockeying\n\n");
    fprintf(outfile, "Number of tellers%16d to%3d\n\n",
            min_tellers, max_tellers);
    fprintf(outfile, "Mean interarrival time%11.3f minutes\n\n",
            mean_interarrival);
    fprintf(outfile, "Mean service time%16.3f minutes\n\n", mean_service);
    fprintf(outfile,
            "Bank closes after%16.3f hours\n\n\n\n", length_doors_open);

    /* Run the simulation varying the number of tellers. */

    for (num_tellers = min_tellers; num_tellers <= max_tellers; ++num_tellers) {

        /* Initialize simlib */

        init_simlib();

        /* Set maxatr = max(maximum number of attributes per record, 4) */

        maxatr = 4;  /* NEVER SET maxatr TO BE SMALLER THAN 4. */

        /* Schedule the first arrival. */

        event_schedule(expon(mean_interarrival, STREAM_INTERARRIVAL),
                       EVENT_ARRIVAL);

        /* Schedule the bank closing.  (Note need for consistency of units.) */

        event_schedule(60 * length_doors_open, EVENT_CLOSE_DOORS);

        /* Run the simulation while the event list is not empty. */

        while (list_size[LIST_EVENT] != 0) {

            /* Determine the next event. */

            timing();

            /* Invoke the appropriate event function. */

            switch (next_event_type) {

                case EVENT_ARRIVAL:
                    arrive();
                    break;
                case EVENT_DEPARTURE:
                    depart((int) transfer[3]);  /* transfer[3] is teller
                                                   number. */
                    break;
                case EVENT_CLOSE_DOORS:
                    event_cancel(EVENT_ARRIVAL);
                    break;
            }
        }
```

FIGURE 2.28
C code for the main function, bank model.

```
            /* Report results for the simulation with num_tellers tellers. */

            report();
        }

        fclose(infile);
        fclose(outfile);

        return 0;
}
```

FIGURE 2.28
(*continued*)

the model. So long as it is before 5 P.M. the close-doors event will be in the event list, and there will always be the next arrival scheduled, so the list will not be empty. At 5 P.M. the close-doors event will occur, removing this event record from the list, and the action of this event will be to remove (using simlib function event_cancel) the next-arrival event, thus "choking off" the arrival stream and removing this event from the event list from then on (since an arrival is scheduled only upon initialization and by the preceding arrival). The only other types of events are the departure events; and if there are any customers left in the bank at 5 P.M., some of them will be in service and their departure events will be in the event list. Eventually (or perhaps immediately at 5 P.M. if there happen to be no customers in the bank at that time), all the customers in the bank will receive service and depart, at which point the event list will become empty and the simulation will end. As long as the while loop is executing, we have the usual activities of invoking the timing function, passing control to arrive or depart for an arrival or departure event (note the int cast on the argument passed to depart, being the teller number in the third attribute of a departure event record), and executing the close-doors event, as described above, when it becomes time to do so. When the while loop ends, the current simulation is over and report is invoked to produce the output. After the for loop ends, all the simulations are over, so we close the input and output files and terminate the program.

A flowchart and the code for the arrival event are given in Figs. 2.29 and 2.30. The function begins by scheduling the next arrival event. Then a for loop begins, with index variable "teller" running over the teller numbers, and each teller is looked at in turn (list numbers $n + 1, n + 2, \ldots, 2n$) to see whether they are idle (i.e., whether list_size[num_tellers + teller] is equal to 0). As soon as an idle teller is found, the customer's delay of 0 is registered in sampst, the teller is made busy by filing a dummy record in that teller's list, and this customer's service-completion event is scheduled. Then the return statement transfers control back to the main function, and neither the rest of the arrive function nor the rest of this for loop (if any) is executed. The remainder of the function refers to the case when all tellers are busy, and the rest of this for loop refers to other, higher-numbered tellers at whom we don't want to look in any case due to the preference for the lowest-numbered idle teller. If this for loop is completed, then all tellers are busy, and the next for loop searches across the queues to find the shortest one, choosing the lowest-numbered one if there is a tie. This tie-breaking rule is implemented by the

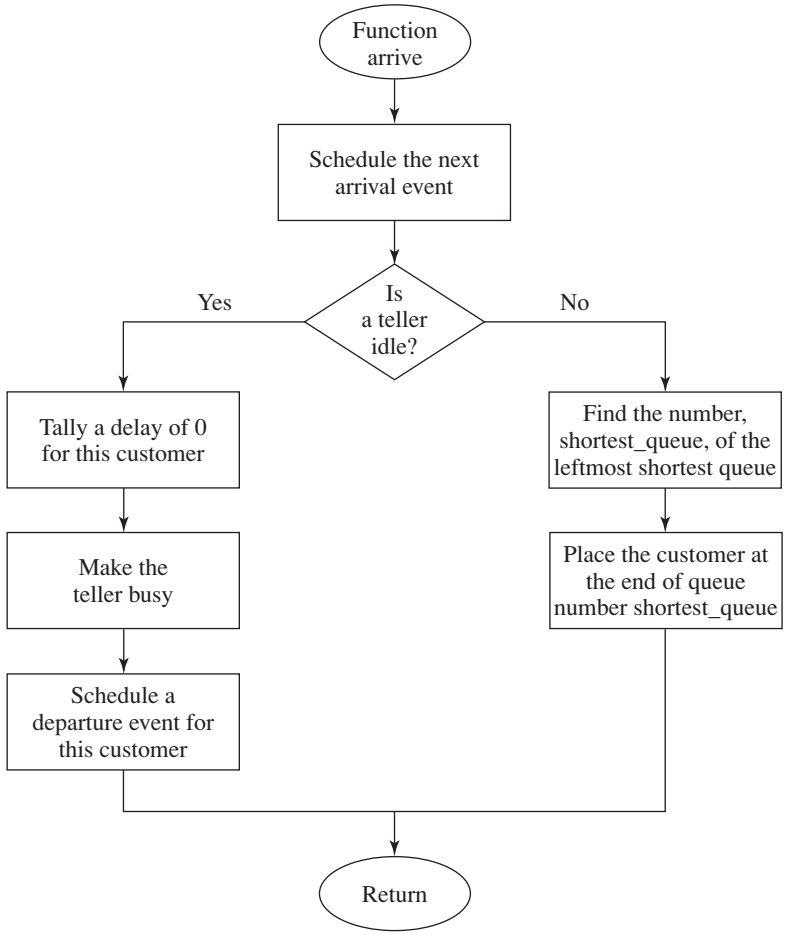


FIGURE 2.29
Flowchart for arrival function, bank model.

strict inequality (<) in the if statement in the for loop, meaning that in the left-to-right search a new choice of the queue would be taken only if the new queue is strictly shorter than the earlier choice. After finishing this for loop, the int shortest_queue will contain the queue number chosen, and the arriving customer is put at the end of that queue, with the time of arrival being the only attribute needed.

Event function depart, with the flowchart and code given in Figs. 2.31 and 2.32, is invoked from the main program when a customer completes service; the int argument "teller" is the number of the teller who is completing a service. If the queue for this teller is empty (list_size[teller] is 0), the teller is made idle by removing the dummy record from the corresponding list, and function jockey is invoked to determine whether a customer from another queue can jockey into service at teller number "teller", who just became idle. On the other hand, if the queue for this teller is not empty, the first customer is removed, his or her delay in queue

```
void arrive(void)  /* Event function for arrival of a customer to the bank. */
{
    int teller;

    /* Schedule next arrival. */

    event_schedule(sim_time + expon(mean_interarrival, STREAM_INTERARRIVAL),
                   EVENT_ARRIVAL);

    /* If a teller is idle, start service on the arriving customer. */

    for (teller = 1; teller <= num_tellers; ++teller) {

        if (list_size[num_tellers + teller] == 0) {

            /* This teller is idle, so customer has delay of zero. */

            sampst(0.0, SAMPST_DELAYS);

            /* Make this teller busy (attributes are irrelevant). */

            list_file(FIRST, num_tellers + teller);

            /* Schedule a service completion. */

            transfer[3] = teller;  /* Define third attribute of type-two event-
                                      list record before event_schedule. */

            event_schedule(sim_time + expon(mean_service, STREAM_SERVICE),
                           EVENT_DEPARTURE);

            /* Return control to the main function. */

            return;
        }
    }

    /* All tellers are busy, so find the shortest queue (leftmost shortest in
       case of ties). */

    shortest_length = list_size[1];
    shortest_queue  = 1;
    for (teller = 2; teller <= num_tellers; ++teller)
        if (list_size[teller] < shortest_length) {
            shortest_length = list_size[teller];
            shortest_queue  = teller;
        }

    /* Place the customer at the end of the leftmost shortest queue. */

    transfer[1] = sim_time;
    list_file(LAST, shortest_queue);
}
```

FIGURE 2.30
C code for function arrive, bank model.

(sim_time − transfer[1]) is registered in sampst, and the service-completion event is scheduled; for long simulations, sim_time and transfer might have to be made type double to avoid loss of precision in the subtraction to calculate the delay in queue. Note that it is our responsibility to define transfer[3] to be the teller number before invoking event_schedule, since this function only copies the time and type of event (attributes 1 and 2) into the transfer array before filing it into the event list. In any case, we must invoke jockey to see if any customers from *other* queues want to

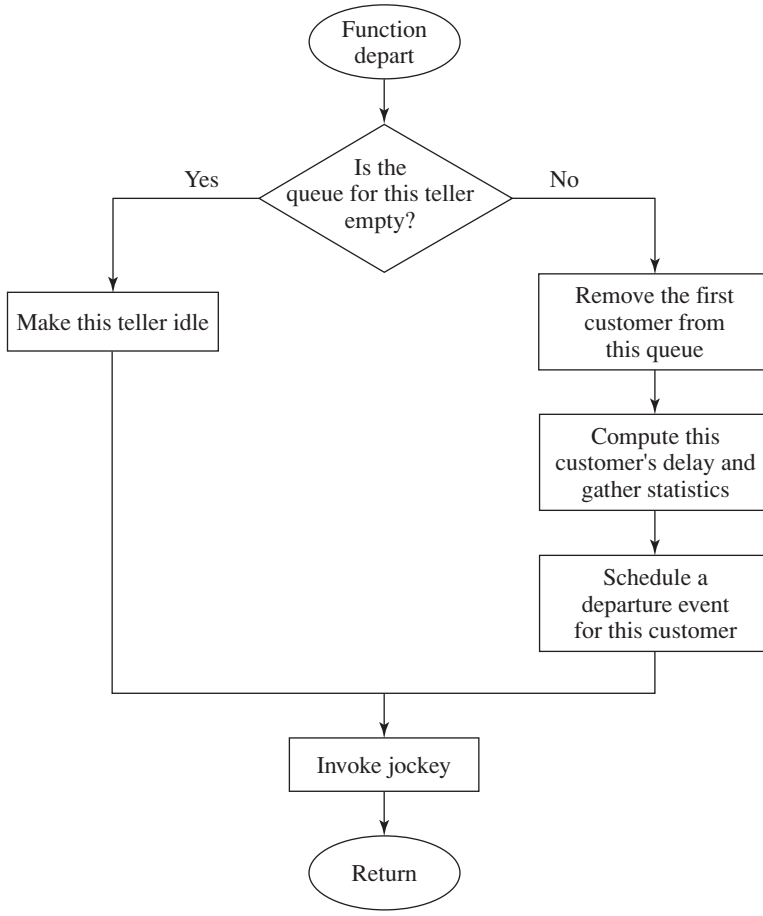


FIGURE 2.31
Flowchart for departure function, bank model.

jockey into this queue. (No customers should jockey after an arrival occurs, since this would not decrease their expected time to departure.)

The non-event function jockey is invoked with an int argument "teller" to see if a customer can jockey to the queue for teller "teller" from another (longer) queue, or possibly right into service at teller "teller" if he just became idle; its flowchart and code are shown in Figs. 2.33 and 2.34. The int variable jumper will hold the queue number of the jockeying customer, if any; it is set to zero initially and is made positive only if such a customer is found. The int variable min_distance is the (absolute) distance (in number of queues) of a potential jockeyer to the destination queue, and it is set to a large number initially, since we want to scan for the minimum such distance. The number of customers facing teller "teller" is the int variable ni, that is, ni $= n_i$ for $i =$ "teller". The for loop examines the queues (other_teller) to see if any of them satisfy the jockeying requirements, represented

```
void depart(int teller)  /* Departure event function. */
{
    /* Check to see whether the queue for teller "teller" is empty. */

    if (list_size[teller] == 0)

        /* The queue is empty, so make the teller idle. */

        list_remove(FIRST, num_tellers + teller);

    else {

        /* The queue is not empty, so start service on a customer. */

        list_remove(FIRST, teller);
        sampst(sim_time - transfer[1], SAMPST_DELAYS);
        transfer[3] = teller;  /* Define before event_schedule. */
        event_schedule(sim_time + expon(mean_service, STREAM_SERVICE),
                       EVENT_DEPARTURE);
    }

    /* Let a customer from the end of another queue jockey to the end of this
       queue, if possible. */

    jockey(teller);
}
```

FIGURE 2.32
C code for function depart, bank model.

here by the condition other_teller $\neq$ teller (since a customer would not jockey to his or her own queue) and nj $>$ ni $+$ 1, where nj is the number of customers facing teller other_teller, that is, nj $= n_j$ for $j =$ other_teller. If both of these conditions are satisfied, then the customer at the end of queue number other_teller would like to jockey, and this customer will (temporarily, perhaps) be issued a jockeying pass if she is strictly closer to the target queue than earlier customers who would also like to jockey (i.e., if the variable "distance", the number of queues this would-be jockeyer is away, is strictly less than the distance of the earlier closest would-be jockeyer). Note that in the case of two closest would-be jockeyers (one on the left and one on the right), we would jockey the one from the left since the one on the right would have to have been *strictly* closer. When this for loop ends, jumper will be zero if the other queue lengths were such that nobody wants to jockey, in which case control is passed back to the main function and no action is taken. If, however, jumper is positive, then it is equal to the queue number from which a customer will jockey, and that customer is removed from the end of his queue. A check is then made to see whether the teller who just finished service is busy (with the customer who was first in this teller's queue), in which case the jockeying customer just joins the end of his new queue. If this teller is idle, however, the jockeying customer jockeys right into service, so his delay is computed and registered, the server is made busy again, and the jockeying customer's service completion is scheduled.

The code for the report generator is in Fig. 2.35, and starts with a loop to add up the average numbers in the separate queues to get the average total number in queue, as explained earlier; this is then written out together with the number of

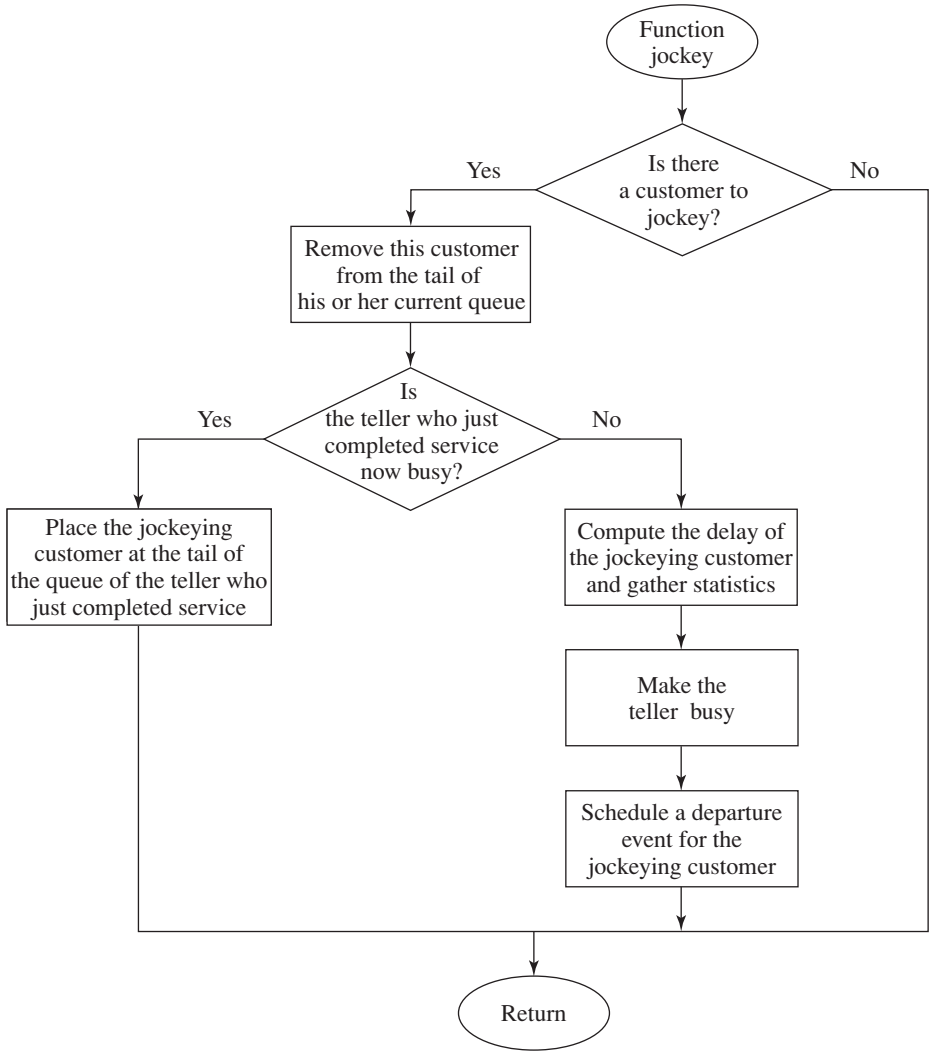


FIGURE 2.33
Flowchart for function jockey, bank model.

tellers in this model variant. Finally, the standard-format output function is invoked for the sole sampst variable to write out the average and maximum of the customer delays in queue(s).

2.6.3 Simulation Output and Discussion

Figure 2.36 contains the results (in the file mtbank.out) of the simulations. Compared to the current policy of five tellers, a reduction to four tellers would seem to penalize customer service quality heavily in terms of both delays in queue and the

```
void jockey(int teller)  /* Jockey a customer to the end of queue "teller" from
                            the end of another queue, if possible. */
{
    int jumper, min_distance, ni, nj, other_teller, distance;

    /* Find the number, jumper, of the queue whose last customer will jockey to
       queue or teller "teller", if there is such a customer. */

    jumper       = 0;
    min_distance = 1000;
    ni           = list_size[teller] + list_size[num_tellers + teller];

    /* Scan all the queues from left to right. */

    for (other_teller = 1; other_teller <= num_tellers; ++other_teller) {

        nj = list_size[other_teller] + list_size[num_tellers + other_teller];
        distance = abs(teller - other_teller);

        /* Check whether the customer at the end of queue other_teller qualifies
           for being the jockeying choice so far. */

        if (other_teller != teller && nj > ni + 1 && distance < min_distance) {

            /* The customer at the end of queue other_teller is our choice so
               far for the jockeying customer, so remember his queue number and
               its distance from the destination queue. */

            jumper       = other_teller;
            min_distance = distance;
        }
    }

    /* Check to see whether a jockeying customer was found. */

    if (jumper > 0) {

        /* A jockeying customer was found, so remove him from his queue. */

        list_remove(LAST, jumper);

        /* Check to see whether the teller of his new queue is busy. */

        if (list_size[num_tellers + teller] > 0)

            /* The teller of his new queue is busy, so place the customer at the
               end of this queue. */

            list_file(LAST, teller);

        else {

            /* The teller of his new queue is idle, so tally the jockeying
               customer's delay, make the teller busy, and start service. */

            sampst(sim_time - transfer[1], SAMPST_DELAYS);
            list_file(FIRST, num_tellers + teller);
            transfer[3] = teller;  /* Define before event_schedule. */
            event_schedule(sim_time + expon(mean_service, STREAM_SERVICE),
                           EVENT_DEPARTURE);
        }
    }
}
```

FIGURE 2.34
C code for function jockey, bank model.

```
void report(void)  /* Report generator function. */
{
    int   teller;
    float avg_num_in_queue;

    /* Compute and write out estimates of desired measures of performance. */

    avg_num_in_queue = 0.0;
    for (teller = 1; teller <= num_tellers; ++teller)
        avg_num_in_queue += filest(teller);
    fprintf(outfile, "\n\nWith%2d tellers, average number in queue = %10.3f",
            num_tellers, avg_num_in_queue);
    fprintf(outfile, "\n\nDelays in queue, in minutes:\n");
    out_sampst(outfile, SAMPST_DELAYS, SAMPST_DELAYS);
}
```

FIGURE 2.35
C code for function report, bank model.

```
Multiteller bank with separate queues & jockeying

Number of tellers               4 to  7

Mean interarrival time      1.000 minutes

Mean service time           4.500 minutes

Bank closes after           8.000 hours


With 4 tellers, average number in queue =     51.319

Delays in queue, in minutes:

 sampst                         Number
variable                          of
 number       Average           values          Maximum          Minimum
------------------------------------------------------------------------

    1         63.2229          501.000          156.363         0.000000
------------------------------------------------------------------------


With 5 tellers, average number in queue =      2.441

Delays in queue, in minutes:

 sampst                         Number
variable                          of
 number       Average           values          Maximum          Minimum
------------------------------------------------------------------------

    1         2.48149          483.000          21.8873         0.000000
------------------------------------------------------------------------


With 6 tellers, average number in queue =      0.718

Delays in queue, in minutes:

 sampst                         Number
variable                          of
 number       Average           values          Maximum          Minimum
------------------------------------------------------------------------

    1        0.763755          467.000          16.5103         0.000000
------------------------------------------------------------------------
```

FIGURE 2.36
Output report, bank model.

```
With 7 tellers, average number in queue =      0.179

Delays in queue, in minutes:

 sampst                          Number
variable                           of
 number       Average            values          Maximum          Minimum

    1        0.176180          493.000          6.97122          0.000000
```

FIGURE 2.36
(*continued*)

queue lengths. In the other direction, adding a sixth teller would bring a substantial improvement in customer service and average queue lengths; whether this is economically advisable would depend on how management values this improvement in customer service with respect to the cost of the extra teller. It seems unlikely in this example that adding a seventh teller could be justified, since the service improvement does not appear great relative to the six-teller system. Note also that we know how many customers were served during the day in each system variant, being the number of delays observed. There is little variation in this quantity across the system variants, since the arrival rate is the same and the lobby has unlimited space.

Problem 2.4(*b*) and (*c*) embellishes this model by adding new output measures (a measure of server utilization and the maximum total number of customers in the queues), and Prob. 2.4(*d*) further enhances the model by considering the realistic possibility of a limit on the size of the bank's lobby to hold the customers in the queues.

2.7 JOB-SHOP MODEL

In this section, we use simlib to simulate a model of a manufacturing system. This example, the most complex one we have considered, illustrates how simulation can be used to identify bottlenecks in a production process.

2.7.1 Problem Statement

A manufacturing system consists of five workstations, and at present stations 1, 2, . . . , 5 consist of 3, 2, 4, 3, and 1 identical machine(s), respectively, as shown in Fig. 2.37. In effect, the system is a network of five multiserver queues. Assume that jobs arrive at the system with interarrival times that are IID exponential random variables with mean 0.25 hour. There are three types of jobs, and arriving jobs are of type 1, 2, and 3 with respective probabilities 0.3, 0.5, and 0.2. Job types 1, 2, and 3 require 4, 3, and 5 tasks to be done, respectively, and each task must be

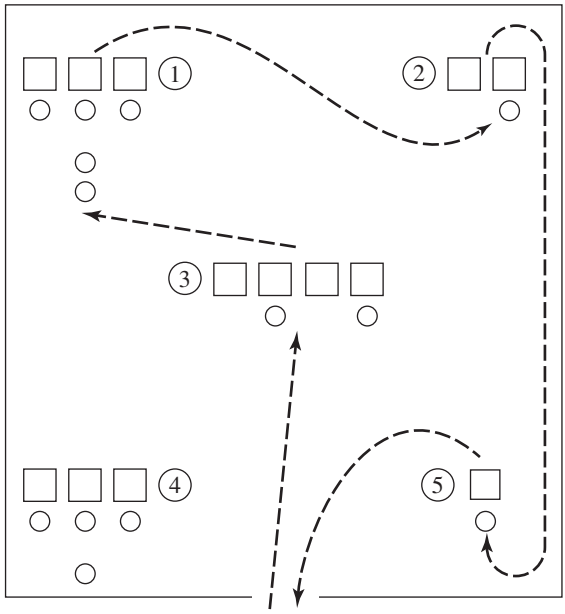


FIGURE 2.37
Manufacturing system with five workstations, showing the route of type 1 jobs.

done at a specified station and in a prescribed order. The routings for the different job types are:

Job type	Workstations in routing
1	3, 1, 2, 5
2	4, 1, 3
3	2, 5, 1, 4, 3

Thus, type 2 jobs first have a task done at station 4, then have a task done at station 1, and finally have a task done at station 3.

If a job arrives at a particular station and finds all machines in that station already busy, the job joins a single FIFO queue at that station. The time to perform a task at a particular machine is an independent 2-Erlang random variable whose mean depends on the job type and the station to which the machine belongs. (If X is a 2-Erlang random variable with mean r, then $X = Y_1 + Y_2$, where Y_1 and Y_2 are independent exponential random variables each with mean $r/2$. Alternatively X is known as a gamma random variable with shape parameter 2 and scale parameter $r/2$. See Sec. 6.2.2 for further details.) We chose the 2-Erlang distribution to represent service times because experience has shown that if one collects data on the time to perform some task, the histogram of these data will often have a shape similar to

that of the density function for an Erlang distribution. The mean service times for each job type and each task are:

Job type	Mean service times for successive tasks, hours
1	0.50, 0.60, 0.85, 0.50
2	1.10, 0.80, 0.75
3	1.20, 0.25, 0.70, 0.90, 1.00

Thus, a type 2 job requires a mean service time of 1.10 hours at station 4 (where its first task will be done).

Assuming no loss of continuity between successive days' operations of the system, we simulate the system for 365 eight-hour days and estimate the expected average total delay in queue (exclusive of service times) for each job type and the expected overall average job total delay. We use the true job-type probabilities 0.3, 0.5, and 0.2 as weights in computing the latter quantity. In addition, we estimate the expected average number in queue, the expected utilization (using simlib function timest), and the expected average delay in queue for each station.

Suppose that all machines cost approximately the same and that the system has the opportunity to purchase one new machine with an eye toward efficiency improvement. We will use the results of the above simulation to decide what additional simulation runs should be made. (Each of these new runs will involve a total of 14 machines, being 1 more than the original number.) From these additional runs, we will use the overall average job total delay to help decide what type of machine the system should purchase.

2.7.2 simlib Program

The events for this model are quite straightforward:

Event description	Event type
Arrival of a job to the system	1
Departure of a job from a particular station	2
End of the simulation	3

Note that for this model, the departure event refers to a job's departing from *any* station on its route, so does not represent the job's leaving the system unless the departure is from the final station on its route. An event graph for this model is given in Fig. 2.38.

We will use the following list structure:

List	Attribute 1	Attribute 2	Attribute 3	Attribute 4
1 through 5, queues	Time of arrival to station	Job type	Task number	—
25, event list	Event time	Event type	Job type	Task number

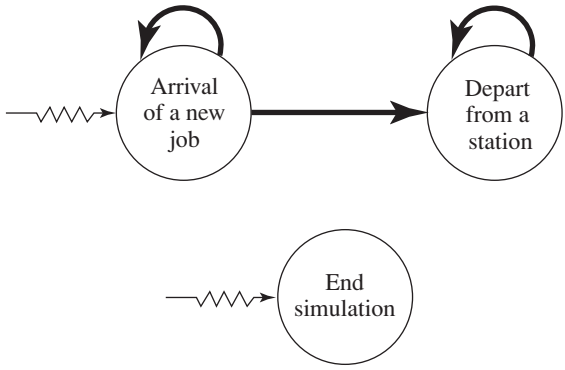


FIGURE 2.38
Event graph, job-shop model.

The "time of arrival" in attribute 1 of a queue list refers to the arrival time of the job to the station for *that* list, rather than the original arrival time to the system. The "task number" of a job represents how far along it is on its route, and will be equal to 1 for the first task, 2 for the second task, and so on; for example, task number 2 for a job of type 3 refers to its processing at station 5. Thus, the station for a job can be determined by knowing the job type and task number.

The delays of the jobs in the queues are used in different ways in this model, so the sampst variable structure is richer than in our previous models. We want the average delay in queue for each station (regardless of job type), and sampst variables 1 through 5 will be used for this. Also, we want to find the average delay in all the queues visited by each job type (regardless of station), for which sampst variables 6 through 8 will be used:

sampst variable number	Meaning
1	Delay in queue at station 1
2	Delay in queue at station 2
3	Delay in queue at station 3
4	Delay in queue at station 4
5	Delay in queue at station 5
6	Delay in queues for job type 1
7	Delay in queues for job type 2
8	Delay in queues for job type 3

Thus, each delay in each queue will be registered into two different sampst variables, one for the station and another for the job type.

For the continuous-time statistics, we will use filest as before and will now use timest as well. Since we have a list for each queue, we can easily get the time-average number in each of the queues by using filest. We also want to observe the utilization of each station; since there may be more than one machine in a station, this is defined as the time-average number of machines that are busy in the station, divided by the total number of machines in the station. To find the average number of busy machines in a station, we will keep our own (i.e., non-simlib) array num_machines_busy[j], which we will maintain as the number of machines

currently busy in station j, and timest will be invoked whenever this changes value for any station. Thus, we have the following timest variables:

timest variable number	Meaning
1	Number of machines busy in station 1
2	Number of machines busy in station 2
3	Number of machines busy in station 3
4	Number of machines busy in station 4
5	Number of machines busy in station 5

For this model, there are three types of random variables needed, to which we assign the following streams:

Stream	Purpose
1	Interarrival times
2	Job types
3	Service times

Stream 3 is used to generate the service times of all jobs, regardless of type; in some simulations we might want to dedicate a separate stream to generate the service time of each job type or at each station in order to control the exact characteristics of each job type or each station.

The external definitions for the program are in Fig. 2.39. After the required #include of simlib.h, we define symbolic constants for the event types, random-number stream numbers, as well as maximum values for the number of stations and job types; these maxima will be used to allocate space in arrays, and using them instead of putting the numbers directly in the array declarations makes the program more general. We next declare several ints and int arrays with names that are mostly self-explanatory; we will use i as a job-type index and j as a station or task-number index. The number of machines that exist in station j is num_machines[j], the total number of tasks (i.e., station visits) for a job of type i is num_tasks[i], and route[i][j] is the station for task j for a type i job. Several floats and float arrays are declared: mean_interarrival is in units of hours (being the time unit for the model), but length_simulation is the length of the simulation in 8-hour days (= 365) so a time-unit adjustment will have to be made; prob_distrib_job_type[i] is the probability that a job will be of type *less than or equal to* i; and mean_service[i][j] is the mean service time (in hours) of task j for a job of type i. The functions are then declared; note that an int argument new_job is passed into arrive, with a value of 1 if this is a new arrival to the system (in which case arrive will serve as an event function) and with a value of 2 if this is the non-event of a job's leaving one station and "arriving" at the next station along its route (in which case arrive will serve as a non-event "utility" function).

The main function, which is somewhat lengthy but of the usual form, is in Fig. 2.40. Note the correction done in the invocation of event_schedule for the end-simulation event to maintain consistency of time units.

```
/* External definitions for job-shop model. */

#include "simlib.h"              /* Required for use of simlib.c. */

#define EVENT_ARRIVAL         1  /* Event type for arrival of a job to the
                                    system. */
#define EVENT_DEPARTURE       2  /* Event type for departure of a job from a
                                    particular station. */
#define EVENT_END_SIMULATION  3  /* Event type for end of the simulation. */
#define STREAM_INTERARRIVAL   1  /* Random-number stream for interarrivals. */
#define STREAM_JOB_TYPE       2  /* Random-number stream for job types. */
#define STREAM_SERVICE        3  /* Random-number stream for service times. */
#define MAX_NUM_STATIONS      5  /* Maximum number of stations. */
#define MAX_NUM_JOB_TYPES     3  /* Maximum number of job types. */

/* Declare non-simlib global variables. */

int   num_stations, num_job_types, i, j, num_machines[MAX_NUM_STATIONS + 1],
      num_tasks[MAX_NUM_JOB_TYPES +1],
      route[MAX_NUM_JOB_TYPES +1][MAX_NUM_STATIONS + 1],
      num_machines_busy[MAX_NUM_STATIONS + 1], job_type, task;
float mean_interarrival, length_simulation, prob_distrib_job_type[26],
      mean_service[MAX_NUM_JOB_TYPES +1][ MAX_NUM_STATIONS + 1];
FILE  *infile, *outfile;

/* Declare non-simlib functions. */

void  arrive(int new_job);
void  depart(void);
void  report(void);
```

FIGURE 2.39
C code for the external definitions, job-shop model.

The function arrive, flowcharted in Fig. 2.41 and listed in Fig. 2.42, begins by checking new_ job to determine whether it is being used as an event function to process a new arrival to the system (new_ job = 1), or whether it is being used as the last part of a station-departure event to process an existing job's arrival at the next station along its route (new_job = 2). If this is a new arrival, the next arrival is scheduled and the job type of this new arrival is generated as a random integer between 1 and 3, using simlib function random_integer and the cumulative probabilities in prob_distrib_ job_type; finally, the task number for this new job is initialized to 1. As we will see in the discussion of depart below, if this is not a new arrival, its job type and task number will already have the correct values in the global variables job_type and task. Regardless of whether the job is new, the function continues by determining the station of the arrival from its job type and task number, by a lookup in the route array. Then a check is made to see whether all the machines in the station are busy. If so, the job is just put at the end of the station's queue. If not, the job has a zero delay here (registered in sampst for both the station and the job type), a machine in this station is made busy, and this is noted in the appropriate timest variable. Note the use of the float cast on the int array num_machines_busy to transform its value into a float, as required by timest. Finally, this job's exit from the station is scheduled, being careful to define event-record attributes beyond the first two before invoking event_schedule.

```
main()  /* Main function. */
{
    /* Open input and output files. */

    infile  = fopen("jobshop.in",  "r");
    outfile = fopen("jobshop.out", "w");

    /* Read input parameters. */

    fscanf(infile, "%d %d %f %f", &num_stations, &num_job_types,
           &mean_interarrival, &length_simulation);
    for (j = 1; j <= num_stations; ++j)
        fscanf(infile, "%d", &num_machines[j]);
    for (i = 1; i <= num_job_types; ++i)
        fscanf(infile, "%d", &num_tasks[i]);
    for (i = 1; i <= num_job_types; ++i) {
        for (j = 1; j <= num_tasks[i]; ++j)
            fscanf(infile, "%d", &route[i][j]);
        for (j = 1; j <= num_tasks[i]; ++j)
            fscanf(infile, "%f", &mean_service[i][j]);
    }
    for (i = 1; i <= num_job_types; ++i)
        fscanf(infile, "%f", &prob_distrib_job_type[i]);

    /* Write report heading and input parameters. */

    fprintf(outfile, "Job-shop model\n\n");
    fprintf(outfile, "Number of workstations%21d\n\n", num_stations);
    fprintf(outfile, "Number of machines in each station     ");
    for (j = 1; j <= num_stations; ++j)
        fprintf(outfile, "%5d", num_machines[j]);
    fprintf(outfile, "\n\nNumber of job types%25d\n\n", num_job_types);
    fprintf(outfile, "Number of tasks for each job type      ");
    for (i = 1; i <= num_job_types; ++i)
        fprintf(outfile, "%5d", num_tasks[i]);
    fprintf(outfile, "\n\nDistribution function of job types  ");
    for (i = 1; i <= num_job_types; ++i)
        fprintf(outfile, "%8.3f", prob_distrib_job_type[i]);
    fprintf(outfile, "\n\nMean interarrival time of jobs%14.2f hours\n\n",
            mean_interarrival);
    fprintf(outfile, "Length of the simulation%20.1f eight-hour days\n\n\n",
            length_simulation);
    fprintf(outfile, "Job type     Workstations on route");
    for (i = 1; i <= num_job_types; ++i) {
        fprintf(outfile, "\n\n%4d        ", i);
        for (j = 1; j <= num_tasks[i]; ++j)
            fprintf(outfile, "%5d", route[i][j]);
    }
    fprintf(outfile, "\n\n\nJob type     ");
    fprintf(outfile, "Mean service time (in hours) for successive tasks");
    for (i = 1; i <= num_job_types; ++i) {
        fprintf(outfile, "\n\n%4d    ", i);
        for (j = 1; j <= num_tasks[i]; ++j)
            fprintf(outfile, "%9.2f", mean_service[i][j]);
    }

    /* Initialize all machines in all stations to the idle state. */

    for (j = 1; j <= num_stations; ++j)
        num_machines_busy[j] = 0;

    /* Initialize simlib */

    init_simlib();
```

FIGURE 2.40
C code for the main function, job-shop model.

```
    /* Set maxatr = max(maximum number of attributes per record, 4) */

    maxatr = 4;  /* NEVER SET maxatr TO BE SMALLER THAN 4. */

    /* Schedule the arrival of the first job. */

    event_schedule(expon(mean_interarrival, STREAM_INTERARRIVAL),
                   EVENT_ARRIVAL);

    /* Schedule the end of the simulation.  (This is needed for consistency of
       units.) */

    event_schedule(8 * length_simulation, EVENT_END_SIMULATION);

    /* Run the simulation until it terminates after an end-simulation event
       (type EVENT_END_SIMULATION) occurs. */

    do {

        /* Determine the next event. */

        timing();

        /* Invoke the appropriate event function. */

        switch (next_event_type) {
            case EVENT_ARRIVAL:
                arrive(1);
                break;
            case EVENT_DEPARTURE:
                depart();
                break;
            case EVENT_END_SIMULATION:
                report();
                break;
        }

    /* If the event just executed was not the end-simulation event (type
       EVENT_END_SIMULATION), continue simulating.  Otherwise, end the
       simulation. */

    } while (next_event_type != EVENT_END_SIMULATION);

    fclose(infile);
    fclose(outfile);

    return 0;
}
```

FIGURE 2.40
(*continued*)

A flowchart and listing for event function depart are given in Figs. 2.43 and 2.44. The values of job_type and task for the departing job are obtained from the departure event record, which was just placed in the transfer array by timing, and the station "station" from which this job is leaving is then looked up in the route array. If the queue for this station is empty, a machine in this station is made idle, and timest is notified of this. If there is a queue, the first job is removed from it (having job type job_type_queue and task number task_queue, to maintain its distinction from the earlier job that is leaving this station), its delay is registered in the two appropriate sampst variables, and its departure from this station is scheduled; again, for long simulations both sim_time and transfer might have to be of

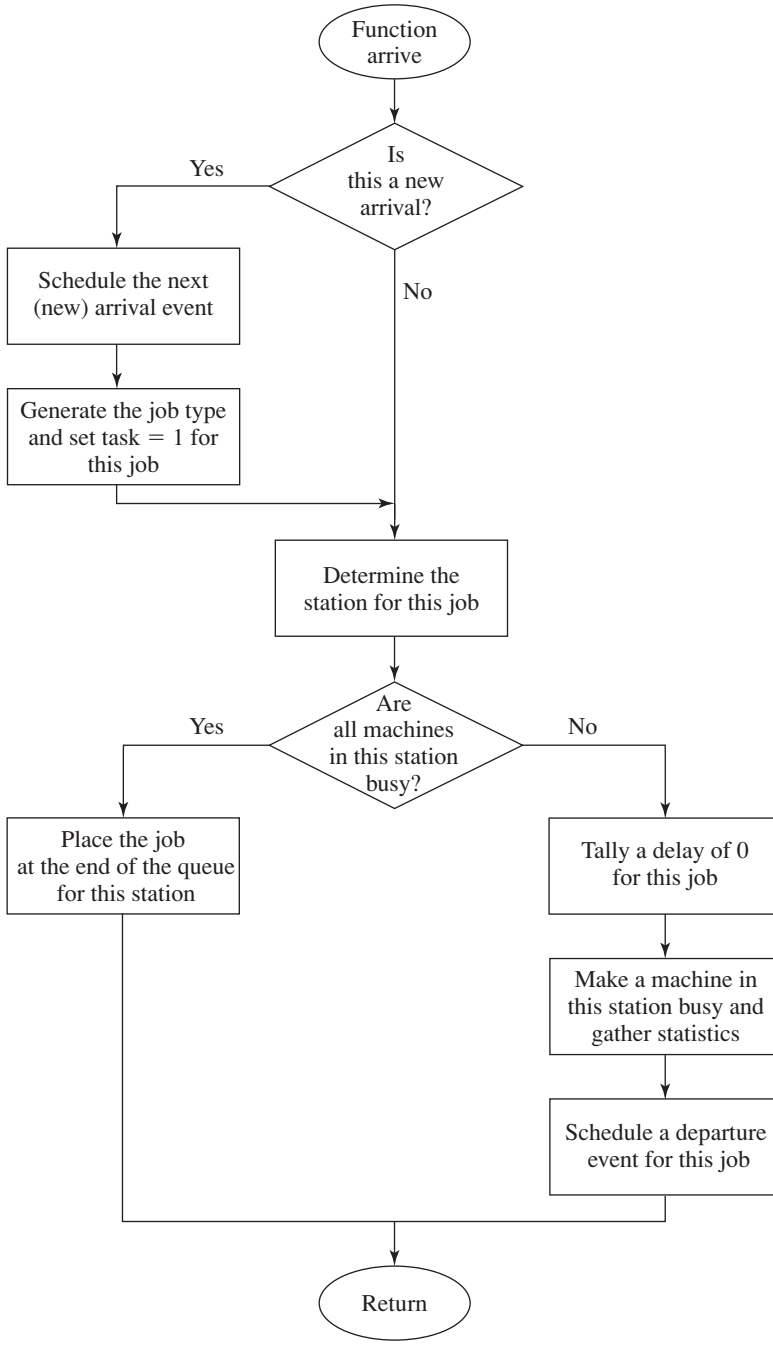


FIGURE 2.41
Flowchart for arrival function, job-shop model.

```
void arrive(int new_job)  /* Function to serve as both an arrival event of a job
                             to the system, as well as the non-event of a job's
                             arriving to a subsequent station along its
                             route. */
{
    int station;

    /* If this is a new arrival to the system, generate the time of the next
       arrival and determine the job type and task number of the arriving
       job. */


    if (new_job == 1) {

        event_schedule(sim_time + expon(mean_interarrival, STREAM_INTERARRIVAL),
                       EVENT_ARRIVAL);
        job_type = random_integer(prob_distrib_job_type, STREAM_JOB_TYPE);
        task     = 1;
    }

    /* Determine the station from the route matrix. */

    station = route[job_type][task];

    /* Check to see whether all machines in this station are busy. */

    if (num_machines_busy[station] == num_machines[station]) {

        /* All machines in this station are busy, so place the arriving job at
           the end of the appropriate queue. Note that the following data are
           stored in the record for each job:
             1. Time of arrival to this station.
             2. Job type.
             3. Current task number. */

        transfer[1] = sim_time;
        transfer[2] = job_type;
        transfer[3] = task;
        list_file(LAST, station);
    }

    else {

        /* A machine in this station is idle, so start service on the arriving
           job (which has a delay of zero). */

        sampst(0.0, station);                              /* For station. */
        sampst(0.0, num_stations + job_type);              /* For job type. */
        ++num_machines_busy[station];
        timest((float) num_machines_busy[station], station);

        /* Schedule a service completion.  Note defining attributes beyond the
           first two for the event record before invoking event_schedule. */

        transfer[3] = job_type;
        transfer[4] = task;
        event_schedule(sim_time
                       + erlang(2, mean_service[job_type][task],
                                STREAM_SERVICE),
                       EVENT_DEPARTURE);
    }
}
```

FIGURE 2.42
C code for the function arrive, job-shop model.

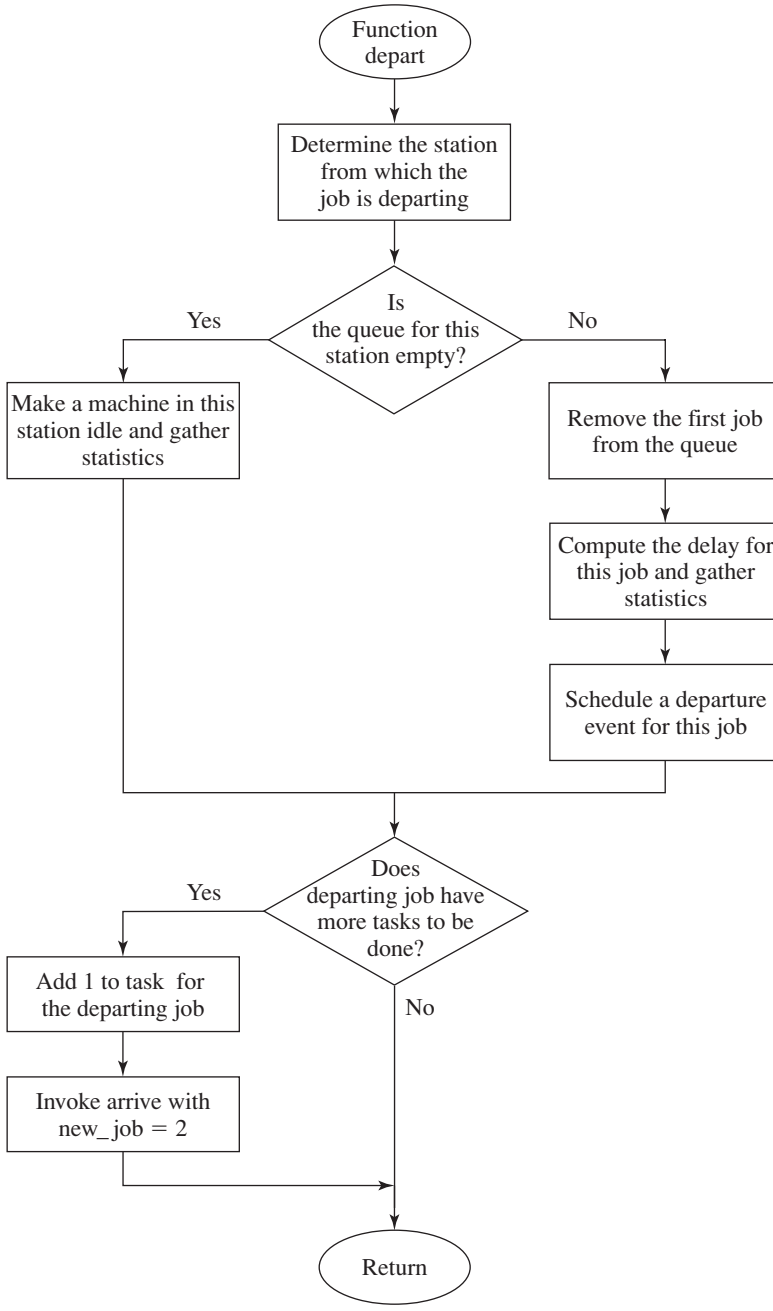


FIGURE 2.43
Flowchart for departure function, job-shop model.

```
void depart(void)  /* Event function for departure of a job from a particular
                      station. */
{
    int station, job_type_queue, task_queue;

    /* Determine the station from which the job is departing. */

    job_type = transfer[3];
    task     = transfer[4];
    station  = route[job_type][task];

    /* Check to see whether the queue for this station is empty. */

    if (list_size[station] == 0) {

        /* The queue for this station is empty, so make a machine in this
           station idle. */

        --num_machines_busy[station];
        timest((float) num_machines_busy[station], station);
    }

    else {

        /* The queue is nonempty, so start service on first job in queue. */

        list_remove(FIRST, station);

        /* Tally this delay for this station. */

        sampst(sim_time - transfer[1], station);

        /* Tally this same delay for this job type. */

        job_type_queue = transfer[2];
        task_queue     = transfer[3];
        sampst(sim_time - transfer[1], num_stations + job_type_queue);

        /* Schedule end of service for this job at this station.  Note defining
           attributes beyond the first two for the event record before invoking
           event_schedule. */

        transfer[3] = job_type_queue;
        transfer[4] = task_queue;
        event_schedule(sim_time
                       + erlang(2, mean_service[job_type_queue][task_queue],
                                STREAM_SERVICE),
                       EVENT_DEPARTURE);
}

    /* If the current departing job has one or more tasks yet to be done, send
       the job to the next station on its route. */

    if (task < num_tasks[job_type]) {
        ++task;
        arrive(2);
    }
}
```

FIGURE 2.44
C code for the function depart, job-shop model.

```
void report(void)  /* Report generator function. */
{
    int   i;
    float overall_avg_job_tot_delay, avg_job_tot_delay, sum_probs;

    /* Compute the average total delay in queue for each job type and the
       overall average job total delay. */

    fprintf(outfile, "\n\n\n\nJob type     Average total delay in queue");
    overall_avg_job_tot_delay = 0.0;
    sum_probs                 = 0.0;
    for (i = 1; i <= num_job_types; ++i) {
        avg_job_tot_delay = sampst(0.0, -(num_stations + i)) * num_tasks[i];
        fprintf(outfile, "\n\n%4d%27.3f", i, avg_job_tot_delay);
        overall_avg_job_tot_delay += (prob_distrib_job_type[i] - sum_probs)
                                     * avg_job_tot_delay;
        sum_probs = prob_distrib_job_type[i];
    }
    fprintf(outfile, "\n\nOverall average job total delay =%10.3f\n",
            overall_avg_job_tot_delay);

    /* Compute the average number in queue, the average utilization, and the
       average delay in queue for each station. */

    fprintf(outfile,
           "\n\n\n Work      Average number      Average       Average delay");
    fprintf(outfile,
             "\nstation       in queue       utilization        in queue");
    for (j = 1; j <= num_stations; ++j)
        fprintf(outfile, "\n\n%4d%17.3f%17.3f%17.3f", j, filest(j),
                timest(0.0, -j) / num_machines[j], sampst(0.0, -j));
}
```

FIGURE 2.45
C code for the function report, job-shop model.

type double to avoid excessive roundoff error in the subtraction for the delay calculation. Finally, if the job leaving this station still has more tasks to be done, its task number is incremented and it is sent on its way to the next station on its route by invoking arrive, now with new_job set to 2 to indicate that this is not a newly arriving job.

The code for the report-generator function is in Fig. 2.45. The first for loop computes the average total delay in all the queues for each job type i; the word "total" is used here to indicate that this is to be the average delay summed for *all* the queues along the route for each job type. We must multiply the average returned in sampst by the number of tasks for this job type, num_tasks[i], since sampst was invoked for each job of this type that left the system num_tasks[i] times rather than once, so that the denominator used by sampst to compute the average is num_tasks[i] times too large. We then weight these average total delays by the probabilities for the job types and add them up to get the overall average job total delay; we use these true (exact) probabilities of job types to obtain a more precise (less variable) estimate than if we simply averaged all the job total delays regardless of job type. Also, we must take successive differences in the prob_distrib_ job_type array to recover the probabilities of the job types' occurring, since this array contains the *cumulative* probabilities. (A technicality: The above multiplication of

the sampst average by num_tasks[i] is slightly incorrect. Since there will generally be some jobs left in the system at the end of the simulation that have not experienced their delays in all the queues, they should not have had *any* of their delays registered in sampst. However, since this simulation is $365 \times 8 = 2920$ hours long and since there are 4 job arrivals expected each hour, there will be an expected 11,680 job arrivals, so this error is likely to be minor. See Prob. 2.5 for an alternative way of collecting the total delays in queue by job type, which avoids this difficulty.) The function closes with a for loop to write out, for each station j, the time-average number in queue, the utilization (computed as the time-average number of machines busy divided by the number of machines in the station), and the average delay in queue.

The function to generate an m-Erlang random variable is in Fig. 2.66 in App. 2A (it is part of simlib, not this model), and follows the physical model for the Erlang distribution described earlier. Note that we must divide the desired expectation of the final Erlang random variable by m to determine the expectation of the component exponential random variables. Also, the user-specified stream number, "stream", is taken as input here and simply passed through to the exponential generator (Fig. 2.63 in App. 2A) and then on to the random-number generator lcgrand.

2.7.3 Simulation Output and Discussion

Figure 2.46 shows the output file (jobshop.out) for this simulation. Weighted by job type, the average time spent by jobs waiting in the queues was almost 11 hours; this

```
Job-shop model

Number of workstations                    5

Number of machines in each station        3    2    4    3    1

Number of job types                       3

Number of tasks for each job type         4    3    5

Distribution function of job types     0.300   0.800   1.000

Mean interarrival time of jobs          0.25 hours

Length of the simulation               365.0 eight-hour days


Job type     Workstations on route

   1            3    1    2    5

   2            4    1    3

   3            2    5    1    4    3
```

FIGURE 2.46
Output report, job-shop model.

Job type	Mean service time (in hours) for successive tasks				
1	0.50	0.60	0.85	0.50	
2	1.10	0.80	0.75		
3	1.20	0.25	0.70	0.90	1.00

Job type	Average total delay in queue
1	10.022
2	9.403
3	15.808

Overall average job total delay = 10.870

Work station	Average number in queue	Average utilization	Average delay in queue
1	12.310	0.969	3.055
2	11.404	0.978	5.677
3	0.711	0.719	0.177
4	17.098	0.961	6.110
5	2.095	0.797	1.043

FIGURE 2.46
(*continued*)

is not the average time in the system, since it does not include processing times at the stations (see Prob. 2.6). We might add that this model produced different numerical results on different computer systems due to its length and complexity, affording greater opportunity for roundoff error and change in order of use of the random-number stream.

Looking at the statistics by station, it appears that the bottlenecks are at stations 1, 2, and 4, although the order of their apparent severity depends on whether we look at average number in queue, utilization, or average delay in queue. Thus, we made three additional runs, adding a machine to each of these stations (stations 3 and 5 appear to be comparatively uncongested, so we did not consider them for a new machine) to see which type of new machine would have the greatest impact on the system's efficiency. Using the overall average job total delay as a single measure of performance, the results from these additional simulations are given in Table 2.1. From simply looking at these numbers, we see that a machine should apparently be added to station 4 to achieve the greatest reduction in overall average job total delay. As usual, however, this conclusion is rather tentative, since we have only a single simulation run of each model variant; this is especially true in this case, since the results for the three new machine configurations are really much too close to call.

TABLE 2.1
Estimated expected overall average job total delays for current and proposed machine configurations

Number of machines in stations	Overall average job total delay, in hours
3, 2, 4, 3, 1 (current configuration)	10.9
4, 2, 4, 3, 1 (add a machine to station 1)	8.1
3, 3, 4, 3, 1 (add a machine to station 2)	7.6
3, 2, 4, 4, 1 (add a machine to station 4)	7.5

2.8 EFFICIENT EVENT-LIST MANAGEMENT

Common to all dynamic simulations we have considered in this and the preceding chapter is the need to schedule events in some way, as well as to determine which of the events scheduled should occur next. We have looked at two different ways to handle the event list. In Chap. 1 it was stored sequentially, with the storage index number being the event type, and the next-event determination was made by searching the event list from top to bottom for the smallest event time. Then in this chapter, armed with simlib's ability to handle linked lists, we stored the event list as a doubly linked list ranked in increasing order on the event-time attribute while using another attribute for the event type; it was easy to determine the next event since it was always on top of the event list. Placing an event record on the list, however, was more work, since it involved searching for the correct location. In either case, a search of the event list is required, either when taking the event off the list or when putting it on.

The need for some sort of event-list processing in dynamic simulations has led a number of researchers to investigate whether other methods might be faster, at least for some types of simulations. For complex simulations involving a large number of events, much of the computer time required to perform the simulation can be expended on even-list processing. Comfort (1981) reported that for one example class of models, the number of instructions required to process the event list can comprise as much as 40 percent of the total number of instructions for the whole simulation. McCormack and Sargent (1981) provide additional evidence that the choice of event-processing algorithm can have a great impact on the simulation execution time. Henriksen (1983) used the term "spectacular failure" to describe the performance (in the case of one example model) of the simple top-to-bottom search to insert event records into the event list.

One way to improve the simlib method of determining the correct location of a new event record would be to use a more efficient data structure and search technique. One well-known approach uses a *median-pointer linked list*, which introduces a pointer to the median record in the event list (with respect to the number of records), in addition to the head and tail pointers. When a new record is to be placed on the list, the record corresponding to the median pointer is first examined to determine whether the new record should be placed in the first half or the second half of the event list. The appropriate half of the event list is then searched sequentially to determine the new

record's location, with McCormack and Sargent's empirical results suggesting that the first (second) half of the event list should be searched from the front (back). For simulations in which the event list can become very long (such as the time-shared computer model of Sec. 2.5 with a very large number of terminals), such a method could make a real difference in overall computation time (see Prob. 2.34).

Many other algorithms have been proposed for event-list management, including various types of heaps and trees [see Knuth (1998b)], calendar queues [Brown (1988)], ladder queues [Tang et al. (2005)], and Henriksen's algorithm (1977). Papers that give empirical evaluations of various event-list algorithms include Chung et al. (1993), Jones (1986), and Rönngren and Ayani (1997).

The choice of the best event-list-processing algorithm may depend on the type of simulation, the parameters and probability distributions used, and other factors that influence how the events are distributed in the event list. For example, in a simulation for which the time elapsing between when an event is scheduled (i.e., put on the event list) and when it occurs (i.e., taken off the event list) is more or less the same for all events, the events will tend to be inserted toward the end of the list in a linked-list data structure; in this case, it could be advantageous to search the event list from bottom to top, since in most cases the search would end quickly. Most modern simulation-software packages (see Chaps. 3 and 14) use efficient event-list-processing algorithms.

APPENDIX 2A
C CODE FOR simlib

The C code for the simlib functions is given in Figs. 2.47 through 2.66. The header file simlib.h, which the user must #include, is in Fig. 2.47; this file in turn #includes simlibdefs.h in Fig. 2.48. Figures 2.49 through 2.66, along with the random-number generator lcgrand in Fig. 7.5 in App. 7A, compose the file simlib.c. All this code can be downloaded from www.mhhe.com/law.

In timest in Fig. 2.57, a subtraction occurs at two points involving the simulation clock sim_time and the time of the last event involving this variable, tlvc[ ]. For long simulations, both could become very large relative to their difference, so might have to be made of type double to avoid loss of precision in this subtraction.

```
/* This is simlib.h. */

/* Include files. */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "simlibdefs.h"

/* Declare simlib global variables. */

extern int    *list_rank, *list_size, next_event_type, maxatr, maxlist;
extern float  *transfer, sim_time, prob_distrib[26];
```

FIGURE 2.47
Header file simlib.h.

```
extern struct master {
    float  *value;
    struct master *pr;
    struct master *sr;
} **head, **tail;

/* Declare simlib functions. */

extern void  init_simlib(void);
extern void  list_file(int option, int list);
extern void  list_remove(int option, int list);
extern void  timing(void);
extern void  event_schedule(float time_of_event, int type_of_event);
extern int   event_cancel(int event_type);
extern float sampst(float value, int varibl);
extern float timest(float value, int varibl);
extern float filest(int list);
extern void  out_sampst(FILE *unit, int lowvar, int highvar);
extern void  out_timest(FILE *unit, int lowvar, int highvar);
extern void  out_filest(FILE *unit, int lowlist, int highlist);
extern float expon(float mean, int stream);
extern int   random_integer(float prob_distrib[], int stream);
extern float uniform(float a, float b, int stream);
extern float erlang(int m, float mean, int stream);
extern float lcgrand(int stream);
extern void  lcgrandst(long zset, int stream);
extern long  lcgrandgt(int stream);
```

FIGURE 2.47
(*continued*)

```
/* This is simlibdefs.h. */

/* Define limits. */

#define MAX_LIST    25      /* Max number of lists. */
#define MAX_ATTR    10      /* Max number of attributes. */
#define MAX_SVAR    25      /* Max number of sampst variables. */
#define TIM_VAR     25      /* Max number of timest variables. */
#define MAX_TVAR    50      /* Max number of timest variables + lists. */
#define EPSILON      0.001  /* Used in event_cancel. */

/* Define array sizes. */

#define LIST_SIZE   26      /* MAX_LIST + 1. */
#define ATTR_SIZE   11      /* MAX_ATTR + 1. */
#define SVAR_SIZE   26      /* MAX_SVAR + 1. */
#define TVAR_SIZE   51      /* MAX_TVAR + 1. */

/* Define options for list_file and list_remove. */

#define FIRST        1      /* Insert at (remove from) head of list. */
#define LAST         2      /* Insert at (remove from) end of list. */
#define INCREASING   3      /* Insert in increasing order. */
#define DECREASING   4      /* Insert in decreasing order. */

/* Define some other values. */

#define LIST_EVENT  25      /* Event list number. */
#define INFINITY     1.E30  /* Not really infinity, but a very large number. */

/* Pre-define attribute numbers of transfer for event list. */

#define EVENT_TIME   1      /* Attribute 1 in event list is event time. */
#define EVENT_TYPE   2      /* Attribute 2 in event list is event type. */
```

FIGURE 2.48
Included file simlibdefs.h.

```
/* This is simlib.c (adapted from SUPERSIMLIB, written by Gregory Glockner). */

/* Include files. */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "simlibdefs.h"

/* Declare simlib global variables. */

int    *list_rank, *list_size, next_event_type, maxatr = 0, maxlist = 0;
float  *transfer, sim_time, prob_distrib[26];
struct master {
    float  *value;
    struct master *pr;
    struct master *sr;
} **head, **tail;

/* Declare simlib functions. */

void  init_simlib(void);
void  list_file(int option, int list);
void  list_remove(int option, int list);
void  timing(void);
void  event_schedule(float time_of_event, int type_of_event);
int   event_cancel(int event_type);
float sampst(float value, int variable);
float timest(float value, int variable);
float filest(int list);
void  out_sampst(FILE *unit, int lowvar, int highvar);
void  out_timest(FILE *unit, int lowvar, int highvar);
void  out_filest(FILE *unit, int lowlist, int highlist);
void  pprint_out(FILE *unit, int i);
float expon(float mean, int stream);
int   random_integer(float prob_distrib[], int stream);
float uniform(float a, float b, int stream);
float erlang(int m, float mean, int stream);
float lcgrand(int stream);
void  lcgrandst(long zset, int stream);
long  lcgrandgt(int stream);
```

FIGURE 2.49
External simlib definitions.

```
void init_simlib()
{

/* Initialize simlib.c.  List LIST_EVENT is reserved for event list, ordered by
   event time.  init_simlib must be called from main by user. */

    int list, listsize;

    if (maxlist < 1) maxlist = MAX_LIST;
    listsize = maxlist + 1;

    /* Initialize system attributes. */

    sim_time = 0.0;
    if (maxatr < 4) maxatr = MAX_ATTR;

    /* Allocate space for the lists. */
```

FIGURE 2.50
simlib function init_simlb.

```
    list_rank = (int *)            calloc(listsize,   sizeof(int));
    list_size = (int *)            calloc(listsize,   sizeof(int));
    head      = (struct master **) calloc(listsize,   sizeof(struct master *));
    tail      = (struct master **) calloc(listsize,   sizeof(struct master *));
    transfer  = (float *)          calloc(maxatr + 1, sizeof(float));

    /* Initialize list attributes. */

    for(list = 1; list <= maxlist; ++list) {
        head [list]     = NULL;
        tail [list]     = NULL;
        list_size[list] = 0;
        list_rank[list] = 0;
    }

    /* Set event list to be ordered by event time. */

    list_rank[LIST_EVENT] = EVENT_TIME;

    /* Initialize statistical routines. */

    sampst(0.0, 0);
    timest(0.0, 0);
}
```

FIGURE 2.50
(*continued*)

```
void list_file(int option, int list)
{

/* Place transfr into list "list".
   Update timest statistics for the list.
   option = FIRST place at start of list
            LAST  place at end of list
            INCREASING  place in increasing order on attribute list_rank(list)
            DECREASING  place in decreasing order on attribute list_rank(list)
            (ties resolved by FIFO) */

    struct master *row, *ahead, *behind, *ihead, *itail;
    int    item, postest;

    /* If the list value is improper, stop the simulation. */

    if(!((list >= 0) && (list <= MAX_LIST))) {
        printf("\nInvalid list %d for list_file at time %f\n", list, sim_time);
        exit(1);
    }

    /* Increment the list size. */

    list_size[list]++;

    /* If the option value is improper, stop the simulation. */

    if(!((option >= 1) && (option <= DECREASING))) {
        printf(
            "\n%d is an invalid option for list_file on list %d at time %f\n",
            option, list, sim_time);
        exit(1);
    }
```

FIGURE 2.51
simlib function list_file.

```
/* If this is the first record in this list, just make space for it. */

if(list_size[list] == 1) {

    row        = (struct master *) malloc(sizeof(struct master));
    head[list] = row ;
    tail[list] = row ;
    (*row).pr  = NULL;
    (*row).sr  = NULL;
}

else { /* There are other records in the list. */

    /* Check the value of option. */

    if ((option == INCREASING) || (option == DECREASING)) {
        item = list_rank[list];
        if(!((item >= 1) && (item <= maxatr))) {
            printf(
                "%d is an improper value for rank of list %d at time %f\n",
                item, list, sim_time) ;
            exit(1);
        }

        row    = head[list];
        behind = NULL; /* Dummy value for the first iteration. */

        /* Search for the correct location. */

        if (option == INCREASING) {
            postest = (transfer[item] >= (*row).value[item]);
            while (postest) {
                behind  = row;
                row     = (*row).sr;
                postest = (behind != tail[list]);
                if (postest)
                    postest = (transfer[item] >= (*row).value[item]);
            }
        }

        else {

            postest = (transfer[item] <= (*row).value[item]);
            while (postest) {
                behind  = row;
                row     = (*row).sr;
                postest = (behind != tail[list]);
                if (postest)
                    postest = (transfer[item] <= (*row).value[item]);
            }
        }

        /* Check to see if position is first or last.  If so, take care of
           it below. */

        if (row == head[list])

            option = FIRST;

        else

            if (behind == tail[list])

                option = LAST;
```

FIGURE 2.51
(*continued*)

```
                else { /* Insert between preceding and succeeding records. */

                    ahead        = (*behind).sr;
                    row          = (struct master *)
                                         malloc(sizeof(struct master));
                    (*row).pr    = behind;
                    (*behind).sr = row;
                    (*ahead).pr  = row;
                    (*row).sr    = ahead;
                }
        } /* End if inserting in increasing or decreasing order. */

        if (option == FIRST) {
            row         = (struct master *) malloc(sizeof(struct master));
            ihead       = head[list];
            (*ihead).pr = row;
            (*row).sr   = ihead;
            (*row).pr   = NULL;
            head[list]  = row;
        }
        if (option == LAST) {
            row         = (struct master *) malloc(sizeof(struct master));
            itail       = tail[list];
            (*row).pr   = itail;
            (*itail).sr = row;
            (*row).sr   = NULL;
            tail[list]  = row;
        }
    }

    /* Copy the row values from the transfer array. */

    (*row).value = (float *) calloc(maxatr + 1, sizeof(float));
    for (item = 0; item <= maxatr; ++item)
        (*row).value[item] = transfer[item];


    /* Update the area under the number-in-list curve. */

    timest((float)list_size[list], TIM_VAR + list);
}
```

FIGURE 2.51
(*continued*)

```
void list_remove(int option, int list)
{

/* Remove a record from list "list" and copy attributes into transfer.
   Update timest statistics for the list.
   option = FIRST remove first record in the list
            LAST  remove last record in the list */

    struct master *row, *ihead, *itail;

    /* If the list value is improper, stop the simulation. */

    if(!((list >= 0) && (list <= MAX_LIST))) {
        printf("\nInvalid list %d for list_remove at time %f\n",
               list, sim_time);
        exit(1);
    }
```

FIGURE 2.52
simlib function list_remove.

```
    /* If the list is empty, stop the simulation. */

    if(list_size[list] <= 0) {
        printf("\nUnderflow of list %d at time %f\n", list, sim_time);
        exit(1);
    }

    /* Decrement the list size. */

    list_size[list]--;

    /* If the option value is improper, stop the simulation. */

    if(!(option == FIRST || option == LAST)) {
        printf(
            "\n%d is an invalid option for list_remove on list %d at time %f\n",
            option, list, sim_time);
        exit(1);
    }

    if(list_size[list] == 0) {

        /* There is only 1 record, so remove it. */

        row        = head[list];
        head[list] = NULL;
        tail[list] = NULL;
    }

    else {

        /* There is more than 1 record, so remove according to the desired
           option. */

        switch(option) {

            /* Remove the first record in the list. */

            case FIRST:
                row         = head[list];
                ihead       = (*row).sr;
                (*ihead).pr = NULL;
                head[list]  = ihead;
                break;

            /* Remove the last record in the list. */

            case LAST:
                row         = tail[list];
                itail       = (*row).pr;
                (*itail).sr = NULL;
                tail[list]  = itail;
                break;
        }
    }

    /* Copy the data and free memory. */

    free((char *)transfer);
    transfer = (*row).value;
    free((char *)row);

    /* Update the area under the number-in-list curve. */

    timest((float)list_size[list], TIM_VAR + list);
}
```

FIGURE 2.52
(*continued*)

```
void timing()
{

/* Remove next event from event list, placing its attributes in transfer.
   Set sim_time (simulation time) to event time, transfer[1].
   Set next_event_type to this event type, transfer[2]. */

    /* Remove the first event from the event list and put it in transfer[]. */

    list_remove(FIRST, LIST_EVENT);

    /* Check for a time reversal. */

    if(transfer[EVENT_TIME] < sim_time) {
        printf(
            "\nAttempt to schedule event type %f for time %f at time %f\n",
            transfer[EVENT_TYPE], transfer[EVENT_TIME], sim_time);
        exit(1);
    }

    /* Advance the simulation clock and set the next event type. */

    sim_time        = transfer[EVENT_TIME];
    next_event_type = transfer[EVENT_TYPE];
}
```

FIGURE 2.53
simlib function timing.

```
void event_schedule(float time_of_event, int type_of_event)
{

/* Schedule an event at time event_time of type event_type.  If attributes
   beyond the first two (reserved for the event time and the event type) are
   being used in the event list, it is the user's responsibility to place their
   values into the transfer array before invoking event_schedule. */

    transfer[EVENT_TIME] = time_of_event;
    transfer[EVENT_TYPE] = type_of_event;
    list_file(INCREASING, LIST_EVENT);
}
```

FIGURE 2.54
simlib function event_schedule.

```
int event_cancel(int event_type)
{

/* Remove the first event of type event_type from the event list, leaving its
   attributes in transfer.  If something is cancelled, event_cancel returns 1;
   if no match is found, event_cancel returns 0. */

    struct       master *row, *ahead, *behind;
    static float high, low, value;

    /* If the event list is empty, do nothing and return 0. */

    if(list_size[LIST_EVENT] == 0) return 0;
```

FIGURE 2.55
simlib function event_cancel.

```
    /* Search the event list. */

    row   = head[LIST_EVENT];
    low   = event_type - EPSILON;
    high  = event_type + EPSILON;
    value = (*row).value[EVENT_TYPE] ;

    while (((value <= low) || (value >= high)) && (row != tail[LIST_EVENT])) {
        row   = (*row).sr;
        value = (*row).value[EVENT_TYPE];
    }

    /* Check to see if this is the end of the event list. */

    if (row == tail[LIST_EVENT]) {

        /* Double check to see that this is a match. */

        if ((value > low) && (value < high)) {
            list_remove(LAST, LIST_EVENT);
            return 1;
        }

        else /* no match */
            return 0;
    }

    /* Check to see if this is the head of the list.  If it is at the head, then
       it MUST be a match. */

    if (row == head[LIST_EVENT]) {
        list_remove(FIRST, LIST_EVENT);
        return 1;
    }

    /* Else remove this event somewhere in the middle of the event list. */

    /* Update pointers. */

    ahead        = (*row).sr;
    behind       = (*row).pr;
    (*behind).sr = ahead;
    (*ahead).pr  = behind;

    /* Decrement the size of the event list. */

    list_size[LIST_EVENT]--;

    /* Copy and free memory. */

    free((char *)transfer);       /* Free the old transfer. */
    transfer = (*row).value;      /* Transfer the data. */
    free((char *)row);            /* Free the space vacated by row. */

    /* Update the area under the number-in-event-list curve. */

    timest((float)list_size[LIST_EVENT], TIM_VAR + LIST_EVENT);
    return 1;
}
```

FIGURE 2.55
(*continued*)

```
float sampst(float value, int variable)
{

/* Initialize, update, or report statistics on discrete-time processes:
   sum/average, max (default -1E30), min (default 1E30), number of observations
   for sampst variable "variable", where "variable":
       = 0 initializes accumulators
       > 0 updates sum, count, min, and max accumulators with new observation
       < 0 reports stats on variable "variable" and returns them in transfer:
           [1] = average of observations
           [2] = number of observations
           [3] = maximum of observations
           [4] = minimum of observations */

    static int   ivar, num_observations[SVAR_SIZE];
    static float max[SVAR_SIZE], min[SVAR_SIZE], sum[SVAR_SIZE];

    /* If the variable value is improper, stop the simulation. */

    if(!(variable >= -MAX_SVAR) && (variable <= MAX_SVAR)) {
        printf("\n%d is an improper value for a sampst variable at time %f\n",
            variable, sim_time);
        exit(1);
    }

    /* Execute the desired option. */

    if(variable > 0) { /* Update. */
        sum[variable] += value;
        if(value > max[variable]) max[variable] = value;
        if(value < min[variable]) min[variable] = value;
        num_observations[variable]++;
        return 0.0;
    }

    if(variable < 0) { /* Report summary statistics in transfer. */
        ivar        = -variable;
        transfer[2] = (float) num_observations[ivar];
        transfer[3] = max[ivar];
        transfer[4] = min[ivar];
        if(num_observations[ivar] == 0)
            transfer[1] = 0.0;
        else
            transfer[1] = sum[ivar] / transfer[2];
        return transfer[1];
    }

    /* Initialize the accumulators. */

    for(ivar=1; ivar <= MAX_SVAR; ++ivar) {
        sum[ivar]              = 0.0;
        max[ivar]              = -INFINITY;
        min[ivar]              =  INFINITY;
        num_observations[ivar] = 0;
    }
}
```

FIGURE 2.56
simlib function sampst.

```
float timest(float value, int variable)
{

/* Initialize, update, or report statistics on continuous-time processes:
   integral/average, max (default -1E30), min (default 1E30)
   for timest variable "variable", where "variable":
       = 0 initializes counters
       > 0 updates area, min, and max accumulators with new level of variable
       < 0 reports stats on variable "variable" and returns them in transfer:
           [1] = time-average of variable updated to the time of this call
           [2] = maximum value variable has attained
           [3] = minimum value variable has attained
   Note that variables TIM_VAR + 1 through TVAR_SIZE are used for automatic
   record keeping on the length of lists 1 through MAX_LIST. */

    int          ivar;
    static float area[TVAR_SIZE], max[TVAR_SIZE], min[TVAR_SIZE],
                 preval[TVAR_SIZE], tlvc[TVAR_SIZE], treset;

    /* If the variable value is improper, stop the simulation. */

    if(!(variable >= -MAX_TVAR) && (variable <= MAX_TVAR)) {
        printf("\n%d is an improper value for a timest variable at time %f\n",
            variable, sim_time);
        exit(1);
    }

    /* Execute the desired option. */

    if(variable > 0) { /* Update. */
        area[variable] += (sim_time - tlvc[variable]) * preval[variable];
        if(value > max[variable]) max[variable] = value;
        if(value < min[variable]) min[variable] = value;
        preval[variable] = value;
        tlvc[variable]   = sim_time;
        return 0.0;
    }

    if(variable < 0) { /* Report summary statistics in transfer. */
        ivar         = -variable;
        area[ivar]   += (sim_time - tlvc[ivar]) * preval[ivar];
        tlvc[ivar]   = sim_time;
        transfer[1]  = area[ivar] / (sim_time - treset);
        transfer[2]  = max[ivar];
        transfer[3]  = min[ivar];
        return transfer[1];
    }

    /* Initialize the accumulators. */

    for(ivar = 1; ivar <= MAX_TVAR; ++ivar) {
        area[ivar]   = 0.0;
        max[ivar]    = -INFINITY;
        min[ivar]    =  INFINITY;
        preval[ivar] = 0.0;
        tlvc[ivar]   = sim_time;
    }
    treset = sim_time;
}
```

FIGURE 2.57
simlib function timest.

```
float filest(int list)
{

/* Report statistics on the length of list "list" in transfer:
       [1] = time-average of list length updated to the time of this call
       [2] = maximum length list has attained
       [3] = minimum length list has attained
   This uses timest variable TIM_VAR + list. */

    return timest(0.0, -(TIM_VAR + list));
}
```

FIGURE 2.58
simlib function filest.

```
void out_sampst(FILE *unit, int lowvar, int highvar)
{

/* Write sampst statistics for variables lowvar through highvar on file
   "unit". */

    int ivar, iatrr;

    if(lowvar>highvar || lowvar > MAX_SVAR || highvar > MAX_SVAR) return;

    fprintf(unit, "\n sampst                          Number");
    fprintf(unit, "\nvariable                           of");
    fprintf(unit, "\n number       Average            values          Maximum");
    fprintf(unit, "          Minimum");
    fprintf(unit, "\n_____________________________________");
    fprintf(unit, "_____________________________________");
    for(ivar = lowvar; ivar <= highvar; ++ivar) {
        fprintf(unit, "\n\n%5d", ivar);
        sampst(0.00, -ivar);
        for(iatrr = 1; iatrr <= 4; ++iatrr) pprint_out(unit, iatrr);
    }
    fprintf(unit, "\n_____________________________________");
    fprintf(unit, "_____________________________________\n\n\n");
}
```

FIGURE 2.59
simlib function out_sampst.

```
void out_timest(FILE *unit, int lowvar, int highvar)
{

/* Write timest statistics for variables lowvar through highvar on file
   "unit". */

    int ivar, iatrr;

    if(lowvar > highvar || lowvar > TIM_VAR || highvar > TIM_VAR ) return;


    fprintf(unit, "\n  timest");
    fprintf(unit, "\n variable       Time");
    fprintf(unit, "\n  number       average          Maximum          Minimum");
    fprintf(unit, "\n_____________________________________");
    for(ivar = lowvar; ivar <= highvar; ++ivar) {
        fprintf(unit, "\n\n%5d", ivar);
        timest(0.00, -ivar);
        for(iatrr = 1; iatrr <= 3; ++iatrr) pprint_out(unit, iatrr);
    }
    fprintf(unit, "\n_____________________________________");
    fprintf(unit, "\n\n\n");
}
```

FIGURE 2.60
simlib function out_timest.

```
void out_filest(FILE *unit, int lowlist, int highlist)
{

/* Write timest list-length statistics for lists lowlist through highlist on
   file "unit". */

    int list, iatrr;

    if(lowlist > highlist || lowlist > MAX_LIST || highlist > MAX_LIST) return;

    fprintf(unit, "\n  File         Time");
    fprintf(unit, "\n number       average          Maximum          Minimum");
    fprintf(unit, "\n_____________________________________________________________");
    for(list = lowlist; list <= highlist; ++list) {
        fprintf(unit, "\n\n%5d", list);
        filest(list);
        for(iatrr = 1; iatrr <= 3; ++iatrr) pprint_out(unit, iatrr);
    }
    fprintf(unit, "\n_____________________________________________________________");
    fprintf(unit, "\n\n\n");
}
```

FIGURE 2.61
simlib function out_filest.

```
void pprint_out(FILE *unit, int i) /* Write ith entry in transfer to file
                                      "unit". */
{
    if(transfer[i] == -1e30 || transfer[i] == 1e30)
        fprintf(unit," %#15.6G ", 0.00);
    else
        fprintf(unit," %#15.6G ", transfer[i]);
}
```

FIGURE 2.62
simlib function pprint_out.

```
float expon(float mean, int stream) /* Exponential variate generation
                                       function. */
{
    return -mean * log(lcgrand(stream));

}
```

FIGURE 2.63
simlib function expon.

```
int random_integer(float prob_distrib[], int stream) /* Discrete-variate
                                                        generation function. */
{
    int   i;
    float u;

    u = lcgrand(stream);

    for (i = 1; u >= prob_distrib[i]; ++i)
        ;
    return i;
}
```

FIGURE 2.64
simlib function random_integer.

```
float uniform(float a, float b, int stream) /* Uniform variate generation
                                               function. */
{
    return a + lcgrand(stream) * (b - a);
}
```

FIGURE 2.65
simlib function uniform.

```
float erlang(int m, float mean, int stream)  /* Erlang variate generation
                                                function. */
{
    int   i;
    float mean_exponential, sum;

    mean_exponential = mean / m;
    sum = 0.0;
    for (i = 1; i <= m; ++i)
        sum += expon(mean_exponential, stream);
    return sum;
}
```

FIGURE 2.66
simlib function erlang.

PROBLEMS

The following problems are to be done using simlib wherever possible.

2.1. For the single-server queue with simlib in Sec. 2.4, replace the dummy list for the server with a variable of your own representing the server status (busy or idle), and use timest instead of filest to get the server utilization.

2.2. For the time-shared computer model of Sec. 2.5, combine the end-simulation event with the end-run event. Redraw the event diagram, and alter and run the program with this simplified event structure.

2.3. For the time-shared computer model of Sec. 2.5, suppose that we want to collect the average response time for each terminal individually, as well as overall. Alter the simulation to do this, and run for the case of $n = 10$ terminals only. (*Hint*: You will have to add another attribute to represent a job's terminal of origin, and you will need to define additional sampst variables as well.)

2.4. For the multiteller bank model of Sec. 2.6, suppose that we want to know the maximum number of customers who are ever waiting in the queues. Do the following parts in order, i.e., with each part building on the previous ones.
(*a*) Explain why this cannot be obtained by adding up the maxima of the individual queues.
(*b*) Modify the program to collect this statistic, and write it out. Run for each of the cases of $n = 4$, 5, 6, and 7 tellers.
(*c*) Add to this an additional output measure, being the utilization of the servers. Since there are multiple servers, the utilization is defined here as the time-average

number of servers busy, divided by the number of servers. Note that this will be a number between 0 and 1.

(*d*) Now suppose that the bank's lobby is large enough to hold only 25 customers in the queues (total). If a customer arrives to find that there are already a total of 25 customers in the queues, he or she just goes away and the business is lost; this is called *balking* and is clearly unfortunate. Change the program to reflect balking, where the capacity of 25 should be read in as an input parameter. In addition to all the other output measures, observe the number of customers who balk during the course of the simulation.

2.5. In the manufacturing-system model of Sec. 2.7, correct the minor error described in the report generator regarding the collection of the total job delay in queue by job type. To do this, add an attribute to each job representing the cumulative delay in queue so far. When the job leaves the system, tally this value in sampst. Rerun the simulation for this alternative approach using the "current configuration" of the number of machines at each station.

2.6. For the manufacturing-system model of Sec. 2.7, estimate the expected overall average job time in system, being the weighted average of the expected times in system (delays in queue plus processing times) for the three job types, using the probabilities of occurrence of the job types as the weights. (*Hint*: You won't need a computer to do this.)

2.7. For the original configuration of the manufacturing system of Sec. 2.7, run the model for 100 eight-hour days, but use only the data from the last 90 days to estimate the quantities of interest. In effect, the state of the system at time 10 days represents the initial conditions for the simulation. The idea of "warming up" the model before beginning data collection is a common simulation practice, discussed in Sec. 9.5.1. (You may want to look at the code for simlib routine timest in Fig. 2.57, paying special attention to the variable treset, to understand how the continuous-time statistics will be computed.)

2.8. For the manufacturing-system model of Sec. 2.7, suggest a different definition of the attributes that would simplify the model's coding.

2.9. Do Prob. 1.15, except use simlib. Use stream 1 for interarrival times, stream 2 for service times at server 1, stream 3 for service times at server 2, and stream 4 for the travel times.

2.10. Do Prob. 1.22, except use simlib. Use stream 1 for the machine-up times and stream 2 for the repair times.

2.11. Do Prob. 1.24, except use simlib. Use stream 1 for interarrival times and stream 2 for service times. Note how much easier this model is to simulate with the list-processing tools.

2.12. Do Prob. 1.26, except use simlib. Use stream 1 for interarrival times, stream 2 for determining the customer type, stream 3 for service times of type 1 customers, and stream 4 for service times of type 2 customers.

2.13. Do Prob. 1.27, except use simlib. Use streams 1 and 2 for interarrival times and service times, respectively, for regular customers, and streams 3 and 4 for interarrival times and service times, respectively, of express customers.

2.14. Do Prob. 1.28, except use simlib. Use stream 1 for interarrival times for regular cars and stream 2 for service times for all cars.

2.15. Do Prob. 1.30, except use simlib. Use stream 1 for interarrival times, stream 2 for inspection times, stream 3 to decide whether a bus needs repair, and stream 4 for repair times.

2.16. For the inventory example of Sec. 1.5, suppose that the delivery lag is distributed uniformly between 1 and 3 months, so there could be between 0 and 3 outstanding orders at a time. Thus, the company bases its ordering decision at the beginning of each month on the sum of the (net) inventory level [denoted by $I(t)$ in Sec. 1.5] and the inventory on order; this sum could be positive, zero, or negative. For each of the nine inventory policies, run the model for 120 months and estimate the expected average total cost per month and the expected proportion of time there is a backlog. Note that holding and shortage costs are still based on the net inventory level. Use stream 1 for interdemand times, stream 2 for demand sizes, and stream 3 for delivery lags.

2.17. Problem 1.18 described a modification of the inventory system of Sec. 1.5 in which the items were perishable. Do this problem, using simlib, and in addition consider the case of LIFO (as well as FIFO) processing of the items in inventory. Use the same stream assignments as in Prob. 2.16, and in addition use stream 4 for the shelf lives.

2.18. For the time-shared computer model of Sec. 2.5, suppose that instead of processing jobs in the queue in a round-robin manner, the CPU chooses the job from the queue that has made the fewest number of previous passes through the CPU. In case of ties, the rule is FIFO. (This is equivalent to using the time of arrival to the queue to break ties.) Run the model with $n = 60$ terminals for 1000 job completions.

2.19. Ships arrive at a harbor with interarrival times that are IID exponential random variables with a mean of 1.25 days. The harbor has a dock with two berths and two cranes for unloading the ships; ships arriving when both berths are occupied join a FIFO queue. The time for one crane to unload a ship is distributed uniformly between 0.5 and 1.5 days. If only one ship is in the harbor, both cranes unload the ship and the (remaining) unloading time is cut in half. When two ships are in the harbor, one crane works on each ship. If both cranes are unloading one ship when a second ship arrives, one of the cranes immediately begins serving the second ship and the remaining service time of the first ship is doubled. Assuming that no ships are in the harbor at time 0, run the simulation for 90 days and compute the minimum, maximum, and average time that ships are in the harbor (which includes their time in berth). Also estimate the expected utilization of each berth and of the cranes. Use stream 1 for the interarrival times and stream 2 for the unloading times. [This problem is a paraphrasing of an example in Russell (1976, p. 134).]

2.20. Jobs arrive at a single-CPU computer facility with interarrival times that are IID exponential random variables with mean 1 minute. Each job specifies upon its arrival the maximum amount of processing time it requires, and the maximum times for successive jobs are IID exponential random variables with mean 1.1 minutes. However, if m is the specified maximum processing time for a particular job, the actual processing time is distributed uniformly between $0.55m$ and $1.05m$. The CPU will never process a job for more than its specified maximum; a job whose required processing time exceeds its specified maximum leaves the facility without completing service.

Simulate the computer facility until 1000 jobs have left the CPU if (*a*) jobs in the queue are processed in a FIFO manner, and (*b*) jobs in the queue are ranked in increasing order of their specified maximum processing time. For each case, compute the average and maximum delay in queue of jobs, the proportion of jobs that are delayed in queue more than 5 minutes, and the maximum number of jobs ever in queue. Use stream 1 for the interarrival times, stream 2 for the maximum processing times, and stream 3 for the actual processing times. Which operating policy would you recommend?

2.21. In a quarry, trucks deliver ore from three shovels to a single crusher. Trucks are assigned to specific shovels, so that a truck will always return to its assigned shovel after dumping a load at the crusher. Two different truck sizes are in use, 20 and 50 tons. The size of the truck affects its loading time at the shovel, travel time to the crusher, dumping time at the crusher, and return-trip time from the crusher back to its shovel, as follows (all times are in minutes):

	20-ton truck	**50-ton truck**
Load	Exponentially distributed with mean 5	Exponentially distributed with mean 10
Travel	Constant 2.5	Constant 3
Dump	Exponentially distributed with mean 2	Exponentially distributed with mean 4
Return	Constant 1.5	Constant 2

To each shovel are assigned two 20-ton trucks and one 50-ton truck. The shovel queues are all FIFO, and the crusher queue is ranked in decreasing order of truck size, the rule's being FIFO in case of ties. Assume that at time 0 all trucks are at their respective shovels, with the 50-ton trucks just beginning to be loaded. Run the simulation model for 8 hours and estimate the expected time-average number in queue for each shovel and for the crusher. Also estimate the expected utilizations of all four pieces of equipment. Use streams 1 and 2 for the loading times of the 20-ton and 50-ton trucks, respectively, and streams 3 and 4 for the dumping times of the 20-ton and 50-ton trucks, respectively. [This problem is taken from Pritsker (1995, pp. 153–158).]

2.22. A batch-job computer facility with a single CPU opens its doors at 7 A.M. and closes its doors at midnight, but operates until all jobs present at midnight have been processed. Assume that jobs arrive at the facility with interarrival times that are exponentially distributed with mean 1.91 minutes. Jobs request either express (class 4), normal (class 3), deferred (class 2), or convenience (class 1) service; and the classes occur with respective probabilities 0.05, 0.50, 0.30, and 0.15. When the CPU is idle, it will process the highest-class (priority) job present, the rule's being FIFO within a class. The times required for the CPU to process class 4, 3, 2, and 1 jobs are 3-Erlang random variables (see Sec. 2.7) with respective means 0.25, 1.00, 1.50, and 3.00 minutes. Simulate the computer facility for each of the following cases:

(*a*) A job being processed by the CPU is not preempted by an arriving job of a higher class.

(*b*) If a job of class i is being processed and a job of class j (where $j > i$) arrives, the arriving job preempts the job being processed. The preempted job joins the queue and takes the highest priority in its class, and only its remaining service time needs to be completed at some future time.

Estimate for each class the expected time-average number of jobs in queue and the expected average delay in queue. Also estimate the expected proportion of time that the CPU is busy and the expected proportion of CPU busy time spent on each class. Note that it is convenient to have one list for each class's queue and also an input parameter that is set to 0 for case (*a*) and 1 for case (*b*). Use stream 1 for the interarrival times, stream 2 for the job-class determination, and streams 3, 4, 5, and 6 for the processing times for classes 4, 3, 2, and 1, respectively.

2.23. A port in Africa loads tankers with crude oil for overwater shipment, and the port has facilities for loading as many as three tankers simultaneously. The tankers, which arrive at the port every 11 ± 7 hours, are of three different types. (All times given as a "$\pm$" range in this problem are distributed uniformly over the range.) The relative frequency of the various types and their loading-time requirements are:

Type	Relative frequency	Loading time, hours
1	0.25	18 ± 2
2	0.25	24 ± 4
3	0.50	36 ± 4

There is one tug at the port. Tankers of all types require the services of a tug to move from the harbor into a berth and later to move out of a berth into the harbor. When the tug is available, any berthing or deberthing activity takes about an hour. It takes the tug 0.25 hour to travel from the harbor to the berths, or vice versa, when not pulling a tanker. When the tug finishes a berthing activity, it will deberth the first tanker in the deberthing queue if this queue is not empty. If the deberthing queue is empty but the harbor queue is not, the tug will travel to the harbor and begin berthing the first tanker in the harbor queue. (If both queues are empty, the tug will remain idle at the berths.) When the tug finishes a deberthing activity, it will berth the first tanker in the harbor queue if this queue is not empty and a berth is available. Otherwise, the tug will travel to the berths, and if the deberthing queue is not empty, will begin deberthing the first tanker in the queue. If the deberthing queue is empty, the tug will remain idle at the berths.

The situation is further complicated by the fact that the area experiences frequent storms that last 4 ± 2 hours. The time between the end of one storm and the onset of the next is an exponential random variable with mean 48 hours. The tug will not start a new activity when a storm is in progress but will always finish an activity already in progress. (The berths will operate during a storm.) If the tug is traveling from the berths to the harbor without a tanker when a storm begins, it will turn around and head for the berths.

Run the simulation model for a 1-year period (8760 hours) and estimate:

(*a*) The expected proportion of time the tug is idle, is traveling without a tanker, and is engaged in either a berthing or deberthing activity

(*b*) The expected proportion of time each berth is unoccupied, is occupied but not loading, and is loading

(*c*) The expected time-average number of tankers in the deberthing queue and in the harbor queue

(*d*) The expected average in-port residence time of each type of tanker

Use stream 1 for interarrivals, stream 2 to determine the type of a tanker, stream 3 for loading times, stream 4 for the duration of a storm, and stream 5 for the time between the end of one storm and the start of the next.

A shipper considering bidding on a contract to transport oil from the port to the United Kingdom has determined that five tankers of a particular type would have to be committed to this task to meet contract specifications. These tankers would require 21 ± 3 hours to load oil at the port. After loading and deberthing, they would travel to the United Kingdom, offload the oil, return to the port for reloading, etc. The round-trip travel time, including offloading, is estimated to be 240 ± 24 hours. Rerun the simulation and estimate, in addition, the expected average in-port residence time of the proposed additional tankers. Assume that at time 0 the five additional tankers are in the harbor queue. Use the same stream assignments as before, and in addition use stream 6 for the oil-loading times at the port and stream 7 for the round-trip travel times for these new tankers. [This problem is an embellishment of one in Schriber (1974, p. 329).]

2.24. In Prob. 2.23, suppose that the tug has a two-way radio giving it the position and status of each tanker in the port. As a result, the tug changes its operating policies, as follows. If the tug is traveling from the harbor to the berths without a tanker and is less than halfway there when a new tanker arrives, it will turn around and go pick up the new tanker. If the tug is traveling from the berths to the harbor without a tanker and is less than halfway there when a tanker completes its loading, it will turn around and go pick up the loaded tanker. Run the simulation with the same parameters and stream assignments as before, under this new operating policy.

2.25. In Prob. 2.24, suppose in addition that if the tug is traveling from the harbor to the berths without a tanker and the deberthing queue is empty when a new tanker arrives, it will turn around and go pick up the new tanker, regardless of its position. Run the simulation with the same parameters and stream assignments as before, under this operating policy.

2.26. Two-piece suits are processed by a dry cleaner as follows. Suits arrive with exponential interarrival times having mean 10 minutes, and are all initially served by server 1, perhaps after a wait in a FIFO queue; see Fig. 2.67. Upon completion of service at server 1, one piece of the suit (the jacket) goes to server 2, and the other part (the pants) to server 3. During service at server 2, the jacket has a probability of 0.05 of being damaged, and while at server 3 the probability of a pair of pants being damaged is 0.10. Upon leaving server 2, the jackets go into a queue for server 4; upon leaving server 3, the pants go into a different queue for server 4. Server 4 matches and reassembles suit parts, initiating this when he is idle and two parts from the same suit are available. If both parts of the reassembled suit are undamaged, the suit is returned to the customer. If either (or both) of the parts is (are) damaged, the suit goes to customer relations (server 5). Assume that all service times are exponential, with the following means (in minutes) and use the indicated stream assignments:

Server number	Mean service time, in minutes	Stream
1	6	1
2	4	2
3	5	3
4	5 (undamaged)	4
4	8 (damaged)	5
5	12	6

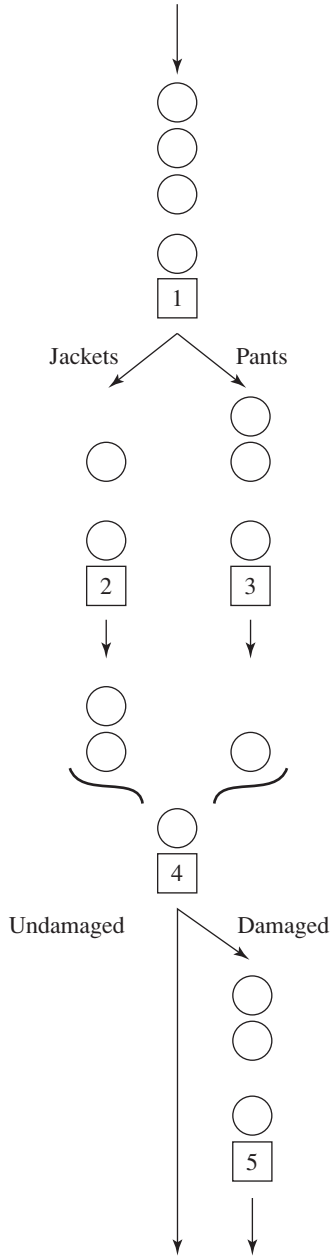


FIGURE 2.67
A dry-cleaning operation.

In addition, use stream 7 for interarrival times, and streams 8 and 9 for determining whether the pieces are damaged at servers 2 and 3, respectively. The system is initially empty and idle, and runs for exactly 12 hours. Observe the average and maximum time in the system for each type of outcome (damaged or not), separately, the average and maximum length of each queue, and the utilization of each server. What would happen if the arrival rate were to double (i.e., the interarrival-time mean were 5 minutes

instead of 10 minutes)? In this case, if you could place another person anywhere in the system to help out with one of the 5 tasks, where should it be?

2.27. A queueing system has two servers (A and B) in series, and two types of customers (1 and 2). Customers arriving to the system have their types determined immediately upon their arrival. An arriving customer is classified as type 1 with probability 0.6. However, an arriving customer may balk, i.e., may not actually join the system, if the queue for server A is too long. Specifically, assume that if an arriving customer finds m ($m \geq 0$) other customers already in the queue for A, he will join the system with probability $1/(m + 1)$, regardless of the type (1 or 2) of customer he may be. Thus, for example, an arrival finding nobody else in the queue for A (i.e., $m = 0$) will join the system for sure [probability $= 1/(0 + 1) = 1$], whereas an arrival finding 5 others in the queue for A will join the system with probability $\frac{1}{6}$. All customers are served by A. (If A is busy when a customer arrives, the customer joins a FIFO queue.) Upon completing service at A, type 1 customers leave the system, while type 2 customers are served by B. (If B is busy, type 2 customers wait in a FIFO queue.) Compute the average *total* time each type of customer spends in the *system*, as well as the number of balks. Also compute the time-average and maximum length of each queue, and both server utilizations. Assume that all interarrival and service times are exponentially distributed, with the following parameters:

- Mean interarrival time (for any customer type) = 1 minute
- Mean service time at server A (regardless of customer type) = 0.8 minute
- Mean service time at server B = 1.2 minutes

Initially the system is empty and idle, and is to run until 1000 customers (of either type) have left the system. Use stream 1 for determining the customer type, stream 2 for deciding whether a customer balks, stream 3 for interarrivals, stream 4 for service times at A (of both customer types), and stream 5 for service times at B.

2.28. An antiquated computer operates in a batch multiprocessing mode, meaning that it starts many (up to a fixed maximum of $k = 4$) jobs at a time, runs them simultaneously, but cannot start any new jobs until all the jobs in a batch are done. Within a batch, each job has its own completion time, and leaves the CPU when it finishes. There are three priority classes, with jobs of class 1 being the highest priority and class 3 jobs being the lowest priority. When the CPU finishes the last job in a batch, it first looks for jobs in the class 1 queue and takes as many as possible from it, up to a maximum of k. If there were fewer than k jobs in the class 1 queue, as many jobs as possible from the class 2 queue are taken to bring the total of class 1 and class 2 jobs to no more than the maximum batch size, k. If still more room is left in the batch, the CPU moves on to the class 3 queue. If the total number of jobs waiting in all the queues is less than k, the CPU takes them all and begins running this partially full batch; it cannot begin any jobs that subsequently arrive until it finishes all of its current batch. If no jobs at all are waiting in the queues, the CPU becomes idle, and the next arriving job will start the CPU running with a batch of size 1. Note that when a batch begins running, there may be jobs of several different classes running together in the same batch.

Within a class queue, the order of jobs taken is to be *either* FIFO or shortest job first (SJF); the simulation is to be written to handle either queue discipline by changing only an input parameter. (Thus, a job's service requirement should be generated

when it arrives, and stored alongside its time of arrival in the queue. For FIFO, this would not really be necessary, but it simplifies the general programming.) The service requirement of a class i job is distributed uniformly between constants $a(i)$ and $b(i)$ minutes. Each class has its own separate arrival process, i.e., the interarrival time between two successive class i jobs is exponentially distributed with mean $r(i)$ minutes. Thus, at any given point in the simulation, there should be three separate arrivals scheduled, one for each class. If a job arrives to find the CPU busy, it joins the queue for its class in the appropriate place, depending on whether the FIFO or SJF option is in force. A job arriving to find the CPU idle begins service immediately; this would be a batch of size 1. The parameters are as follows:

i	$r(i)$	$a(i)$	$b(i)$
1	0.2	0.05	0.11
2	1.6	0.94	1.83
3	5.4	4.00	8.00

Initially the system is empty and idle, and the simulation is to run for exactly 720 minutes. For each queue, compute the average, minimum, and maximum delay, as well as the time-average and maximum length. Also, compute the utilization of the CPU, defined here as the proportion of time it is busy regardless of the number of jobs running. Finally, compute the time-average number of jobs running in the CPU (where 0 jobs are considered running when the CPU is idle). Use streams 1, 2, and 3 for the interarrival times of jobs of class 1, 2, and 3, respectively, and streams 4, 5, and 6 for their respective service requirements. Suppose that a hardware upgrade could increase k to 6. Would this be worth it?

2.29. Consider a queueing system with a fixed number $n = 5$ of parallel servers fed by a single queue. Customers arrive with interarrival times that are exponentially distributed with mean 5 (all times are in minutes). An arriving customer finding an idle server will go directly into service, choosing the leftmost idle server if there are several, while an arrival finding all servers busy joins the end of the queue. When a customer (initially) enters service, her service requirement is distributed uniformly between $a = 2$ and $b = 2.8$, but upon completion of her initial service, she may be "dissatisfied" with her service, which occurs with probability $p = 0.2$. If the service was satisfactory, the customer simply leaves the system, but if her service was not satisfactory, she will require further service. The determination as to whether a service was satisfactory is to be made when the service is completed. If an unsatisfactory service is completed and there are no other customers waiting in the queue, the dissatisfied customer immediately begins another service time at her same server. On the other hand, if there is a queue when an unsatisfactory service is completed, the dissatisfied customer must join the queue (according to one of two options, described below), and the server takes the first person from the queue to serve next. Each time a customer *re*enters service, her service time and probability of being dissatisfied are lower; specifically, a customer who has *already* had i (unsatisfactory) services has a next service time that is distributed uniformly between $a/(i + 1)$ and $b/(i + 1)$, and her probability of being dissatisfied with this next service is $p/(i + 1)$. Theoretically, there is no upper limit on the number of times a given customer will have to be served to be finally satisfied.

There are two possible rules concerning what to do with a dissatisfied customer when other people are waiting in queue; the program is to be written so that respecifying a single input parameter will change the rule from (i) to (ii):

(i) A customer who has just finished an unsatisfactory service joins the end of the queue.

(ii) A customer who has just finished an unsatisfactory service rejoins the queue so that the next person taken from the (front of the) queue will be the customer who has already had the largest number of services; the rule is FIFO in case of ties. This rule is in the interest of both equity and efficiency, since customers with a long history of unsatisfactory service tend to require shorter service and also tend to be more likely to be satisfied with their next service.

Initially the system is empty and idle, and the simulation is to run for exactly 480 minutes. Compute the average and maximum total time in system [including all the delay(s) in queue and service time(s) of a customer], and the number of satisfied customers who leave the system during the simulation. Also compute the average and maximum length of the queue, and the time-average and maximum number of servers that were busy. Use stream 1 for interarrivals, stream 2 for all service times, and stream 3 to determine whether each service was satisfactory.

2.30. The student-center cafeteria at Big State University is trying to improve its service during the lunch rush from 11:30 A.M. to 1:00 P.M. Customers arrive together in groups of size 1, 2, 3, and 4, with respective probabilities 0.5, 0.3, 0.1, and 0.1. Interarrival times between groups are exponentially distributed with mean 30 seconds. Initially, the system is empty and idle, and is to run for the 90-minute period. Each arriving customer, whether alone or part of a group, takes one of three routes through the cafeteria (groups in general split up after they arrive):

- Hot-food service, then drinks, then cashier
- Specialty-sandwich bar, then drinks, then cashier
- Drinks (only), then cashier

The probabilities of these routes are respectively 0.80, 0.15, and 0.05; see Fig 2.68. At the hot-food counter and the specialty-sandwich bar, customers are served one at a time (although there might actually be one or two workers present, as discussed below). The drinks stand is self-service, and assume that nobody ever has to queue up here; this is equivalent to thinking of the drinks stand as having infinitely many servers. There are either two or three cashiers (see below), each having his own queue, and there is no jockeying; customers arriving to the cashiers simply choose the shortest queue. All queues in the model are FIFO.

In Fig. 2.68, ST stands for service time at a station, and ACT stands for the accumulated (future) cashier time due to having visited a station; the notation ~U(a, b) means that the corresponding quantity is distributed uniformly between a and b seconds. For example, a route 1 customer goes first to the hot-food station, joins the queue there if necessary, receives service there that is uniformly distributed between 50 and 120 seconds, "stores away" part of a (future) cashier time that is uniformly distributed between 20 and 40 seconds, then spends an amount of time uniformly distributed between 5 seconds and 20 seconds getting a drink, and accumulates an additional amount of (future) cashier time distributed uniformly between 5 seconds and 10 seconds. Thus, his service requirement at a cashier will be the sum of the U(20, 40) and U(5, 10) random variates he "picked up" at the hot-food and drinks stations.

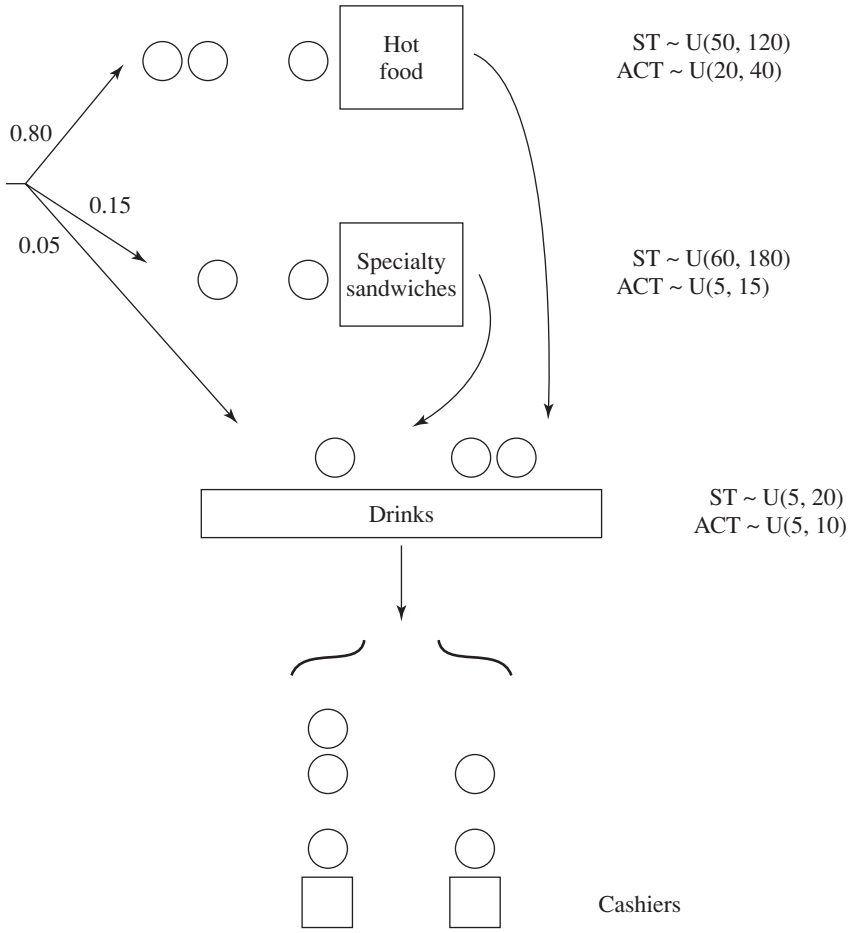


FIGURE 2.68
The BSU cafeteria.

Report the following measures of system performance:

- The average and maximum delays in queue for hot food, specialty sandwiches, and cashiers (regardless of which cashier)
- The time-average and maximum number in queue for hot food and specialty sandwiches (separately), and the time-average and maximum total number in all cashier queues
- The average and maximum total delay in all the queues for each of the three types of customers (separately)
- The overall average total delay for all customers, found by weighting their individual average total delays by their respective probabilities of occurrence
- The time-average and maximum total number of customers in the entire system (for reporting to the fire marshall)

There are several questions about the system's operation. For security reasons, there must be at least 2 cashiers, and the maximum number of cashiers is 3. Also, there must be at least one person working at each of the hot-food and specialty-sandwich stations.

Thus, the minimum number of employees is 4; run this as the "base-case" model. Then, consider adding employees, in several ways:

(*a*) Five employees, with the additional person used in one of the following ways:
  (i) As a third cashier
  (ii) To help at the hot-food station. In this case, customers are still served one at a time, but their service time is cut in half, being distributed uniformly between 25 seconds and 60 seconds.
  (iii) To help at the specialty-sandwich bar, meaning that service is still one at a time, but distributed uniformly between 30 seconds and 90 seconds

(*b*) Six employees, in one of the following configurations:
  (i) Two cashiers, and two each at the hot-food and specialty-sandwich stations
  (ii) Three cashiers, two at hot food, and one at specialty sandwiches
  (iii) Three cashiers, one at hot food, and two at specialty sandwiches

(*c*) Seven employees, with three cashiers, and two each at the hot-food and specialty-sandwich stations

Run the simulation for all seven expansion possibilities, and make a recommendation as to the best employee deployment at each level of the number of employees. In all cases, use stream 1 for the interarrival times between groups, stream 2 for the group sizes, stream 3 for an individual's route choice, streams 4, 5, and 6 for the STs at the hot-food, specialty-sandwich, and drinks stations, respectively, and streams 7, 8, and 9 for the ACTs at these respective stations.

2.31. Consolidated Corkscrews (CC) is a multinational manufacturer of precision carbon-steel corkscrews for heavy-duty, high-speed use. Each corkscrew is made on a metal lathe, and in order to meet rising consumer demand for their product, CC is planning a new plant with six lathes. They are not sure, however, how this new plant should be constructed, or how the maintenance department should be equipped. Each lathe has its own operator, who is also in charge of repairing the lathe when it breaks down. Reliability data on lathe operation indicate that the "up" time of a lathe is exponentially distributed with mean 75 minutes. When a lathe goes down, its operator immediately calls the tool crib to request a tool kit for repairs. The plant has a fixed number, m, of tool kits, so there may or may not be a kit in the crib when an operator calls for one. If a tool kit is not available, the operator requesting one is placed in a FIFO queue and must wait his or her turn for a kit; when one later becomes available, it is then placed on a conveyor belt and arrives t_i minutes later to lathe i, where t_i might depend on the lathe number, i, requesting the kit. If a kit is available, it is immediately placed on a conveyor belt and arrives at the broken lathe t_i minutes later; in this case the operator's queue delay is counted as 0. When an operator of a broken lathe receives a tool kit, he or she begins repair, which takes an amount of time distributed as a 3-Erlang random variable with mean 15 minutes. When the repair is complete, the lathe is brought back up and the tool kit is sent back to the tool crib, where it arrives t_i minutes later, if it is sent back from lathe i. Initially, assume that all lathes are up and have just been "freshly repaired," and that all m tool kits are in the crib. CC wants to know about the projected operation of the plant over a continuous 24-hour day by looking at:

- The proportion of time that each of the six lathes is down
- The time-average number of lathes that are down
- The time-average number of tool kits sitting idle in the crib
- The average delay in queue of operators requesting a tool kit

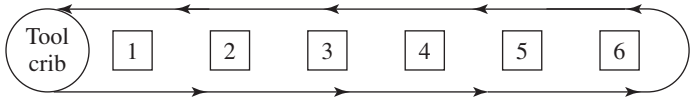


FIGURE 2.69
The linear design.

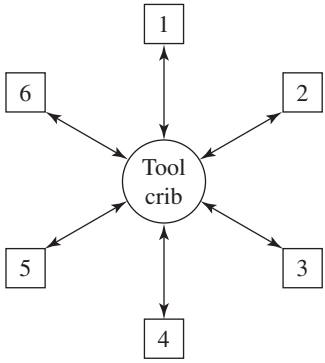


FIGURE 2.70
The circular design.

There are two major questions to be addressed:

(*a*) How should the plant be laid out? Two layouts are under consideration:
  (i) In the linear design (see Fig. 2.69), the lathes are placed in a straight line with the tool crib at the left end, and a single conveyor belt for the tool kits can reach all lathes. In this case, $t_i = 2i$ minutes, for $i = 1, 2, \ldots, 6$.
  (ii) In the circular design, the lathes are placed around the tool crib (see Fig. 2.70), and each lathe has its own conveyor belt to the crib; here, $t_i = 3$ for all lathe numbers i. This is a more expensive design, but results in shorter travel times for the kits.

(*b*) How many tool kits should there be? As tool kits are quite expensive, CC does not want to purchase more than necessary.

Carry out the necessary simulations and advise CC on questions (*a*) and (*b*). In all cases, use stream 1 for the lathe-up times, and stream 2 for repair times.

2.32. The engines on jet aircraft must be periodically inspected and, if necessary, repaired. An inspection/repair facility at a large airport handles seven different types of jets, as described in the table below. The times between successive arrivals of planes of type i (where $i = 1, 2, \ldots, 7$) are exponentially distributed with mean $a(i)$, as given in the table; all times are in days. There are n parallel service stations, each of which sequentially handles the inspection and repair of all the engines on a plane, but can deal with only one engine at a time. For example, a type 2 plane has three engines, so when it enters service, each engine must undergo a complete inspection and repair process (as described below) before the next engine on this plane can begin service, and all three engines must be inspected and (if necessary) repaired before the plane leaves the service station. Each service station is capable of dealing with any type of plane. As usual, a plane arriving to find an idle service station goes directly into service, while an arriving plane finding all service stations occupied must join a single queue.

Plane type (i)	Number of engines	$a(i)$	$A(i)$	$B(i)$	$p(i)$	$r(i)$	$c(i)$
1	4	8.1	0.7	2.1	0.30	2.1	2.1
2	3	2.9	0.9	1.8	0.26	1.8	1.7
3	2	3.6	0.8	1.6	0.18	1.6	1.0
4*	4	8.4	1.9	2.8	0.12	3.1	3.9
5	4	10.9	0.7	2.2	0.36	2.2	1.4
6	2	6.7	0.9	1.7	0.14	1.7	1.1
7*	3	3.0	1.6	2.0	0.21	2.8	3.7

Two of the seven types of planes are classified as widebody (denoted by an asterisk * in the above table), while the other five are classified as regular. Two disciplines for the queue are of interest:

(i) Simple FIFO with all plane types mixed together in the same queue

(ii) Nonpreemptive priority given to widebody jets, with the rule being FIFO within the widebody and regular classifications

For each engine on a plane (independently), the following process takes place (i denotes the plane type):

- The engine is initially inspected, taking an amount of time distributed uniformly between $A(i)$ and $B(i)$.
- A decision is made as to whether repair is needed; the probability that repair is needed is $p(i)$. If no repair is needed, inspection of the jet's next engine begins; or if this was the last engine, the jet leaves the facility.
- If repair is needed, it is carried out, taking an amount of time distributed as a 2-Erlang random variable with mean $r(i)$.
- After repair, another inspection is done, taking an amount of time distributed uniformly between $A(i)/2$ and $B(i)/2$ (i.e., half as long as the initial inspection, since tear-down is already done). The probability that the engine needs further repair is $p(i)/2$.
- If the initial repair was successful, the engine is done. If the engine still fails inspection, it requires further repair, taking an amount of time distributed as 2-Erlang with mean $r(i)/2$, after which it is inspected again, taking an amount of time distributed uniformly between $A(i)/2$ and $B(i)/2$; it fails this inspection with probability $p(i)/2$, and would need yet more repair, which would take a 2-Erlang amount of time with mean $r(i)/2$. This procedure continues until the engine finally passes inspection. The mean repair time stays at $r(i)/2$, the probability of failure stays at $p(i)/2$, and the inspection times stay between $A(i)/2$ and $B(i)/2$.

A cost of $c(i)$ (measured in tens of thousands of dollars) is incurred for every (full) day a type i plane is down, i.e., is in queue or in service. The general idea is to study how the total (summed across all plane types) average daily downtime cost depends on the number of service stations, n. Initially the system is empty and idle, and the simulation is to run for 365 round-the-clock days. Observe the average delay in queue for each plane type and the overall average delay in queue for all plane types, the time-average number of planes in queue, the time-average number of planes down for each plane type separately, and the total average daily downtime cost for all planes added together. Try various values of n to get a feel for the system's behavior. Recommend a choice for n, as well as which of the queue disciplines (i) or (ii) above appears to lead to the most cost-effective operation. Use streams 1 through 7 for the interarrival times of plane types $i = 1$ through $i = 7$, respectively, streams 8 through 14 for their

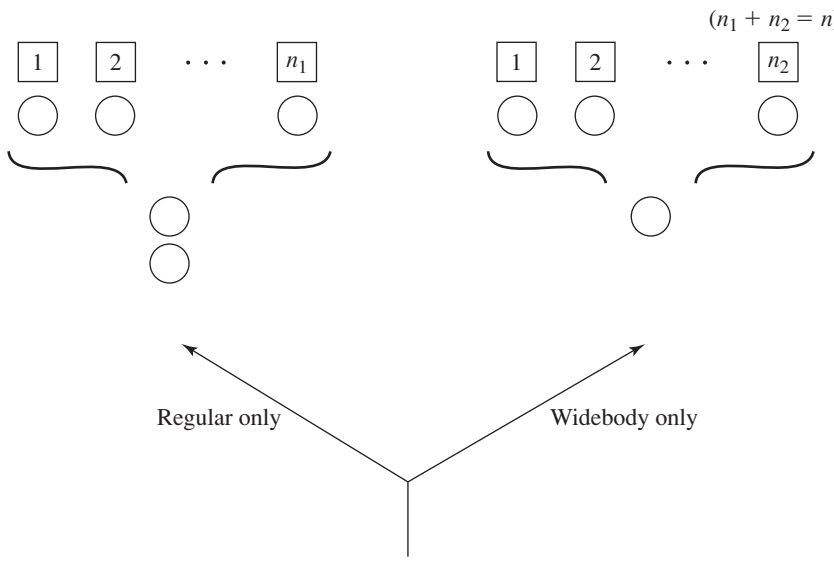


FIGURE 2.71
Alternative layout for the aircraft-repair facility.

respective inspection times (first or subsequent), streams 15 through 21 to determine whether they need (additional) repair, and streams 22 through 28 for their repair times (first or subsequent).

As an alternative to the above layout, consider separating entirely the service of the widebody and regular jets. That is, take n_2 of the n stations and send all the widebody jets there (with a single queue of widebodies feeding all n_2 stations), and the remaining $n_1 = n - n_2$ stations are for regular jets only; see Fig. 2.71. Do you think that this alternative layout would be better? Why? Use the same parameters and stream assignments as above.

2.33. Write a C function "delete" to delete the (logically) first record from list "list" with a value "value" (a float-valued representation of an integer, for example, 5.0 to represent 5) for attribute "attribute". Place the attributes of the deleted record in the transfer array. To delete a desired record, a statement of the form "delete(list, value, attribute)" should be executed. If there is an error condition (e.g., there is no matching record in the list), return a value of 0; otherwise return a value of 1. Also, update the statistics for list "list" by invoking timest (see the code for function remove in App. 2A).

2.34. Write a C function "insert" to insert a new event record into the event list, using the median-pointer algorithm discussed in Sec. 2.8. If two event records have the same event time, give preference to the event with the lowest-numbered event type.

2.35. For the bank model in Sec. 2.6, suppose that after a customer has waited in queue a certain amount of time, the customer *may* leave without being served; this is called *reneging*. Assume that the amount of time a customer will wait in queue before considering reneging is distributed uniformly between 5 and 10 minutes; if this amount of

time does elapse while the customer is in queue, the customer will actually leave with the following probabilities:

Position in queue when time elapses	1	2	3	≥ 4
Probability of reneging	0.00	0.25	0.50	1.00

Using the function "delete" from Prob. 2.33, run the simulation model with five tellers and estimate (in addition to what was estimated before) the expected proportion of customers who renege and the expected average delay in queue of the reneging customers. Use the same stream assignments as in Sec. 2.6, and in addition use stream 3 for the time a customer will wait in queue before considering reneging, and stream 4 for determining if he or she actually reneges if this time elapses.

2.36 A five-story office building is served by a single elevator. People arrive to the ground floor (floor 1) with independent exponential interarrival times having mean 1 minute. A person will go to each of the upper floors with probability 0.25. It takes the elevator 15 seconds to travel one floor. Assume, however, that there is no loading or unloading time for the elevator at a particular floor. A person will stay on a particular floor for an amount of time that is distributed uniformly between 15 and 120 minutes. When a person leaves floor i (where $i = 2, 3, 4, 5$), he or she will go to floor 1 with probability 0.7, and will go to each of the other three floors with probability 0.1. The elevator can carry six people, and starts on floor 1. If there is not room to get all people waiting at a particular floor on the arriving elevator, the excess remain in queue. A person coming down to floor 1 departs from the building immediately. The following control logic also applies to the elevator:

- When the elevator is going up, it will continue in that direction if a current passenger wants to go to a higher floor or if a person on a higher floor wants to get on the elevator.
- When the elevator is going down, it will continue in that direction if it has at least one passenger or if there is a waiting passenger at a lower floor.
- If the elevator is at floor i (where $i = 2, 3, 4$) and going up (down), then it will not immediately pick up a person who wants to go down (up) at that floor.
- When the elevator is idle, its home base is floor 1.
- The elevator decides at each floor what floor it will go to next. It will not change directions between floors.

Use the following random-number stream assignments:

1, interarrival times of people to the building
2, next-floor determination (generate upon arrival at origin floor)
3, length of stay on a particular floor (generate upon arrival at floor)

Run a simulation for 20 hours and gather statistics on:
(*a*) Average delay in queue in each direction (if appropriate), for each floor
(*b*) Average of individual delays in queue over all floors and all people
(*c*) Proportion of time that the elevator is moving with people, is moving empty, and is idle (on floor 1)
(*d*) Average and maximum number in the elevator
(*e*) Proportion of people who cannot get on the elevator since it is full, for each floor

Rerun the simulation if the home base for the elevator is floor 3. Which home base gives the smallest average delay [output statistic (*b*)]?

2.37. Coal trains arrive to an unloading facility with independent exponential interarrival times with mean 10 hours. If a train arrives and finds the system idle, the train is unloaded immediately. Unloading times for the train are independent and distributed uniformly between 3.5 and 4.5 hours. If a train arrives to a busy system, it joins a FIFO queue.

The situation is complicated by what the railroad calls "hogging out." In particular, a train crew can work for only 12 hours, and a train cannot be unloaded without a crew present. When a train arrives, the remaining crew time (out of 12 hours) is independent and distributed uniformly between 6 and 11 hours. When a crew's 12 hours expire, it leaves immediately and a replacement crew is called. The amount of time between when a replacement crew is called and when it actually arrives is independent and distributed uniformly between 2.5 and 3.5 hours.

If a train is being unloaded when its crew hogs out, unloading is suspended until a replacement crew arrives. If a train is in queue when its crew hogs out, the train cannot leave the queue until its replacement crew arrives. Thus, the unloading equipment can be idle with one or more trains in queue.

Run the simulation for 720 hours (30 days) and gather statistics on:

(*a*) Average and maximum time a train spends in the system
(*b*) Proportion of time unloading equipment is busy, idle, and hogged out
(*c*) Average and maximum number of trains in queue
(*d*) Proportion of trains that hog out 0, 1, and 2 times

Note that if a train is in queue when its crew hogs out, the record for this train must be accessed. (This train may be anywhere in the queue.) Use the C function "delete" from Prob. 2.33.

2.38. Consider a car-rental system shown in Fig. 2.72, with all distances given in miles. People arrive at location i (where $i = 1, 2, 3$) with independent exponential interarrival times at respective rates of 14, 10, and 24 per hour. Each location has a FIFO queue with unlimited capacity. There is one bus with a capacity of 20 people and a speed of 30 miles per hour. The bus is initially at location 3 (car rental), and leaves immediately in a counterclockwise direction. All people arriving at a terminal want to go to the car rental. All people arriving at the car rental want to go to terminals 1 and 2 with

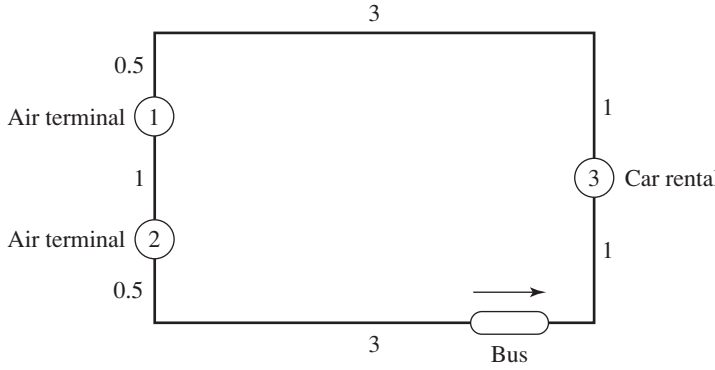


FIGURE 2.72
A car rental system.

respective probabilities 0.583 and 0.417. When a bus arrives at a location, the following rules apply:

- People are first unloaded in a FIFO manner. The time to unload one person is distributed uniformly between 16 and 24 seconds.
- People are then loaded on the bus up to its capacity, with a loading time per person that is distributed uniformly between 15 and 25 seconds.
- The bus always spends at least 5 minutes at each location. If no loading or unloading is in process after 5 minutes, the bus will leave immediately.

Run a simulation for 80 hours and gather statistics on:

(*a*) Average and maximum number in each queue
(*b*) Average and maximum delay in each queue
(*c*) Average and maximum number on the bus
(*d*) Average, maximum, and minimum time the bus is stopped at each location
(*e*) Average, maximum, and minimum time for the bus to make a loop (departure from the car rental to the next such departure)
(*f*) Average, maximum, and minimum time a person is in the system by arrival location

Use the following random-number stream assignments:

i, interarrival times at location *i* (where $i = 1, 2, 3$)
4, unloading times
5, loading times
6, determining destination of an arrival at the car rental